

Univerzitet u Beogradu
Fakultet organizacionih nauka
Katedra za elektronsko poslovanje

Diplomski rad iz predmeta:

Internet tehnologije

Tema: Razvoj veb aplikacije za studentsku službu

Mentor:

Zorica Bogdanović

Student:

Vladimir Jecić 79/17

Beograd, 2026.

Sadržaj

1. Uvod.....	8
2. Projektovanje arhitekture softvera	9
2.1 Analiza zahteva	10
2.1.1 Funkcionalni zahtevi	11
2.1.2 Atributi kvaliteta	14
2.1.2.1 arc42 Quality Model (Q42)	16
2.2 Projektovanje interfejsa.....	17
2.2.1 Paterni arhitekture API	17
2.2.2 API Arhikturalni stilovi i protokoli	19
2.3 Metodološki okviri za projektovanja arhitekture softvera	20
2.3.1 Acr42	20
2.3.2 4+1 View Model	25
2.3.3 Software Architecture Inception Canvas	26
2.4 Detaljno Projektovanje	28
2.4.1 Opis zahteva pomoću modela slučajeva korišćenja	28
2.4.2 Opis ponašanja sistema korišćenjem sequence dijagrama	28
3. Studijki primer - projektovanje rešenja	29
3.1 Uvod i ciljevi	29
3.1.1 Pregled zahteva	31
3.1.2 Ciljevi kvaliteta i Stejkholderi.....	31
3.2. Ograničenja	32
3.2.1 Tehnička ograničenja	32
3.2.2 Organizaciona ograničenja	32
3.2.3 Konvencije.....	32
3.3. Kontekst i opseg rešenja	33
3.3.1 Business Context.....	33
3.3.2 Deployment Context.....	34
3.4. Strategija rešenja	34
3.4.1 Nefunkcionalni zahtevi	34

3.4.2 Principi i Paterni	35
3.5 Pregled Modula(Building Block View)	35
3.5.1 Relacioni Model	35
3.5.2 Konceptualni Model.....	36
3.5.3 Pregled modula na klijentskoj strani	36
3.5.4 Pregled modula na serverskoj strani	38
3.6 Pregled Procesu (Runtime View)	39
3.7 Pregled Raspoređivanja (Deployment View)	41
3.8 Crosscutting Koncepti	41
3.8.1 Logovanje grešaka	41
3.8.2 Handlovanje grešaka.....	42
3.9 Odluke o dizajnu	43
3.10 Zahtevi kvaliteta.....	45
3.11 Rizici i tehnološki dug	46
3.12 Detaljno projektovanje	46
3.12.1 Analiza zahteva i projektovanje interfejsa	46
3.12.1.1. Slučaj korišćenja 1 – Prijavljivanje korisnika.....	47
3.12.1.2. Slučaj korišćenja 2 – Pregled svih ispita u aktuelnom ispitnom roku.....	48
3.12.1.3. Slučaj korišćenja 3 – Pregled studentovih ispita u aktuelnom ispitnom roku	49
3.12.1.4. Slučaj korišćenja 4 – Pregled uspešnih polaganja	50
3.12.1.5. Slučaj korišćenja 5 – Prijavljivanje polaganja ispita.....	52
3.12.1.6. Slučaj korišćenja 6 – Odjavljivanje polaganja ispita	54
3.12.1.7 Slučaj korišćenja 7 – Ocenjivanje prijave studenata	55
3.12.1.8. Slučaj korišćenja 8 – Odjavljivanje korisnika	57
3.12.1.9. Slučaj korišćenja 9 – Pregled izveštaja ispita	57
3.12.1.10. Slučaj korišćenja 10 – Pregled izveštaja predmeta u školskoj godini.....	60
4. Studijski primer - prikaz realizovanog rešenja	62
4.1 Implementacija na klijentskoj strani.....	62
4.2 Implementacija na serverskoj strani.....	69
4.2.1 Konfiguracija na serverskoj strani: konekcije prema bazi i migracija dummy podataka	71
4.3 Implementacija i korisničko uputstvo	73
4.3.1 Uputstvo SK1: Prijavljivanje korisnika.....	73
4.2.2 Uputstvo SK2: Pregled svih ispita u aktuelnom ispitnom roku	77

4.2.3 Uputstvo SK3: Pregled studentovih ispita u aktuelnom ispitnom roku	79
4.2.4 Uputstvo SK4: Pregled uspešnih polaganja ispita.....	81
4.2.5 Uputstvo SK5: Prijavljivanje polaganja ispita.....	86
4.2.6 Uputstvo SK6: Odjavljivanje polaganja ispita	88
4.2.7 Uputstvo SK7: Ocenjivanje prijave studenta	89
4.2.8 Uputstvo SK8: Odjavljivanje korisnika	92
4.2.9 Uputstvo SK9: Pregled izveštaja polaganja predmeta u ispitnom roku	93
4.2.10 Uputstvo SK10: Predgled izveštaja predmeta u školskoj godini.....	97
4.3. API	102
5. Zaključak	104
6. Reference.....	105
6.1. Linkovi ka Github repozitorijumima projekta	105
6.2 Korišćena Literatura.....	105

Slike

Slika 1: Primer SSA analize na 2 nivoa dekompozicije ^[6]	12
Slika 2: Primer UML Use Case dijagrama crtana po uzoru na ^[5]	13
Slika 3: Primer definisanja UserStorija kroz alat za kolaboraciju Jira	14
Slika 4: 8 ključnih sistemskih atributa, kao fokus 8 različitih uticajnih grupa, crtana po uzoru na ^[14]	16
Slika 5: Paterni arhitekture aplikacionih programskih interfejsa, preuzeto sa ^[10]	18
Slika 6: Primer Biznis Kontekst Diagrama – Uređaj proveru validonost HTML linkova (HTML Sanity Check), preuzeto sa ^[23]	22
Slika 8: Nekoliko izvršnih okruženja ugnježđenih na istom serverskom uređaju, preuzeto sa ^[16]	23
Slika 9: Primer specifikacije deployment artifakta, preuzeto sa ^[16]	23
Slika 10: Prožimajući koncepti kroz komponente, po uzoru na ^[25]	24
Slika 11: Koncepti koji se mogu pojaviti u poglavlju prožimajućih koncepata, slika radjena po uzoru na ^[24]	24
Slika 12: 4+1 View Model, crtana po uzoru na ^[17]	26
Slika 13: Kanvas softverske arhitekture, slika prevedena od ^[18]	27
Slika 14: Architecture Inception Canvas za projekat Studentska služba	30
Slika 15: Business Context Studentske službe	33
Slika 16: Deployment Context	34
Slika 17: Arhitektura na najvišem nivou apstrakcije.....	35
Slika 18: Konceptualni model (ER Diagram – Crow’s Foot Notacija)	36
Slika 19: Struktura sistema studentska služba sa klijentske strane.....	37

Slika 20: Struktura sistema studentska služba na serverskoj strani	38
Slika 21: Detaljni dijagram sekvenci kao primer interakcije komponenti prilikom slučaja korišćenja prijave ispita.....	40
Slika 22: Deployment Diagram	41
Slika 23: Omogućavanje opcije logovanja grešaka	42
Slika 24: Primer postavljanje greške prilikom pokušaja registracije korisnika sa indexom koji već postoji.....	42
Slika 25: Implementacija Custom Exception klase RegistrationAlreadyExistsException	43
Slika 26: Primer prikaza greške prilikom pokušaja registracije na ispit na koji je student već registrovan	43
Slika 27: Colorlib.com tema Sidebar #07 korišćena za dizajn GUI-a.....	44
Slika 28: Finalni izgled website-a sa primenjenom temom	44
Slika 29: Zahtevi kvaliteta, slika radjena po uzoru na sliku iz istoimenog poglavlja u DokChess dokumentaciji ^[15]	45
Slika 30: Dijagram slučajeva korišćenja	47
Slika 31: Dijagram sekvenci SK1, osnovni scenario.....	48
Slika 32: Dijagram sekvenci SK2, osnovni scenario.....	49
Slika 33: Dijagram sekvenci SK3, osnovni scenario.....	50
Slika 34: Dijagram sekvenci SK4, osnovni scenario.....	51
Slika 35: Detaljan dijagram sekvenci SK5, osnovni scenario	53
Slika 36: Dijagram sekvenci SK6, osnovni scenario.....	54
Slika 37: Dijagram sekvenci SK7, osnovni scenario.....	56
Slika 38: Dijagram sekvenci SK8, osnovni scenario.....	57
Slika 39: Dijagram sekvenci SK9, osnovni scenario.....	59
Slika 40: Dijagram sekvenci SK10, osnovni scenario.....	60
Slika 41: Konceptualni model softverskog sistema	62
Slika 42: App.js, UserProvider,AlertServiceProvider	64
Slika 43: AppLayout.....	65
Slika 44: pages folder	66
Slika 45: mapiranje ruta na page komponente	66
Slika 46: Pregled preostalih ispita studenta- ExamPeriodPage	67
Slika 47: Pregled preostalih ispita studenta – servis courseExam.ts	68
Slika 48: Implementacija GET funkcije korišćenjem axios biblioteke za HTTP pozive	69
Slika 49: Organizacija koda prema Hexagon arhitekturi.....	70
Slika 50: Spisak ruta na serverskoj strani.....	71
Slika 51: php/ext folder i php.ini	72
Slika 52: Konfiguracija Laravel - SQL Server konekcije	72
Slika 53: SK1:Izgled login forme	73
Slika 54: SK1: Uspešno logovanje	74
Slika 55: SK1, poruka: Došlo je do greške pri prijavljivanju	74
Slika 56: SK1, poruka: Pogrešan broj indeksa ili password	75
Slika 57: SK1, Implementacija login metode	76
Slika 58: SK2, Uspešno učitani svi ispiti u aktuelnom ispitnom roku.....	77
Slika 59: SK2, Controller metoda getExamPeriodsWithFilters	78
Slika 60: SK2, Vraćanje aktivnih rokova, metoda active().....	79
Slika 61: SK3, Uspešno učitani samo studentovi ispiti u aktuelnom ispitnom roku	80

Slika 62: SK3, Metoda getRemainingCourseExams	80
Slika 63: SK4, Navigacioni meni - student.....	81
Slika 64: SK4, Položeni ispiti studenta	82
Slika 65: SK4, Mogući parametri paginacije pretrage prijave ispita	83
Slika 66: SK4, Priprema parametara i poziv metode getAllExamRegistrationsWithFilters	84
Slika 67: SK4, Implementacija getAllExamRegistrationsWithFilters metode	85
Slika 68: SK5, Učitavanje ispita koje student može da polaže	86
Slika 69: SK5, Prijava ispita.....	87
Slika 70: SK5, Implementacija metode za kreiranje nove prijave.....	87
Slika 71: SK5, Poruka ispit je uspešno prijavljen	88
Slika 72: SK5 - Period za prijavu je istekao.....	88
Slika 73: SK6, implementacija odjave ispita.....	88
Slika 74: SK6, poruka, Ispit je uspešno odjavljen	89
Slika 75: SK6, Period za prijavu je istekao	89
Slika 76: SK7: Upis ocena	90
Slika 77: SK7, Pretraga po studentu i/ili ispitno roku i ispitu.....	90
Slika 78: SK7, Dodatni filteri.....	91
Slika 79: SK7: Unos ocene	91
Slika 80: SK7, poruka, Polaganje je uspešno ažurirano	92
Slika 81: SK7, implementacija updateExamRegistration, prvog unosa ili izmena ocene	92
Slika 82: SK8: Dugme izloguj me	93
Slika 83: SK9, Navigacioni meni – admin	94
Slika 84: SK9, Pretraga željenog izveštaja	94
Slika 85: SK9, Izveštaj ispita Statistika u roku septembar-2025	95
Slika 86: SK9, Excel izveštaj	95
Slika 87: Pripremanje CourseExamReportDTO i poziv generateCourseExamReport	96
Slika 88: SK09, kreiranje izveštaja na osnovu DTO-a i XML templejta.....	97
Slika 89: SK09, XML templejt za excel.....	97
Slika 90: SK10, Forma za pregledIzveštaj predmeta	98
Slika 91: SK10, Pretraga predmeta	98
Slika 92: SK10, izveštaj predmeta prikaz u dijalogu	99
Slika 93: SK10, Procenat prisustva po rokovima	99
Slika 94: SK10, Prolaznost po rokovima	100
Slika 95: SK10, Dugme za preuzimanje PDF-a i prosečna ocena po rokovima	100
Slika 96: SK10, PDF izveštaj	101
Slika 97: SK10, računanje prosečne prisutnosti,ocene i prolaznosti za izabrani predmet u svakom od rokovima	102
Slika 98: Spisak Swagger ruta.....	103

Sadržaj tabela

Tabela 1: Uticajne grupe i njihova očekivanja	16
Tabela 2: Oznaka kvaliteta/značenje	17
Tabela 3: Templejt tabele ciljevi kvaliteta u strategiji rešenja	22
Tabela 4: Tehnička ograničenja	32
Tabela 5: Organizaciona ograničenja	32
Tabela 6: Konvencije	32
Tabela 7: Ciljevi kvaliteta u strategiji rešenja	34

1. Uvod

Savremene obrazovne institucije se u sve većoj meri oslanjaju na informacione sisteme za upravljanje ključnim administrativnim procesima poput prijave ispita, evidencije polaganja i generisanja izveštaja. Kvalitet takvih softverskih rešenja direktno utiče na efikasnost rada studentskih službi, zadovoljstvo studenata i pouzdanost akademskih podataka. Međutim, razvoj ovakvih sistema u praksi je često praćen izazovima vezanim za nedostatak sistematičnog pristupa projektovanju arhitekture softvera, što dovodi do rešenja koja su teško održiva, slabo dokumentovana i nefleksibilna za buduće proširivanje [4]. Iz tog razloga, pitanje kako na adekvatan način projektovati arhitekturu sistema koji zadovoljava i funkcionalne i nefunkcionalne zahteve predstavlja značajan istraživački problem u oblasti softverskog inženjeringa.

Postojeća literatura pruža dobro definisane metodološke okvire za projektovanje arhitekture softvera, uključujući arc42 templejt dokumentacije [16], 4+1 View Model [17] i Software Architecture Inception Canvas [18]. Takođe, dobro su poznati principi analize zahteva [3], projektovanja interfejsa [8][9] i definisanja atributa kvaliteta [14]. Međutim, u dostupnim radovima nedostaje konkretan prikaz primene ovih metodologija na studijskom primeru iz domena upravljanja akademskim procesima, gde bi se od faze analize zahteva do implementacije pratila kompletna realizacija rešenja korišćenjem savremenih web tehnologija. Većina dostupnih primera, poput DokChess dokumentacije [15], demonstrira arc42 u okvirima koji nisu direktno primenljivi na specifičnosti studentskih servisa.

Cilj ovog rada je da se, primenom navedenih metodoloških okvira, projektuje i realizuje full-stack web aplikacija za studentsku službu, Estudent, koja obezbeđuje upravljanje ispitnim rokovima, prijavom i odjavom ispita, ocenjivanjem i generisanjem izveštaja. Očekuje se da rešenje zadovolji ključne attribute kvaliteta, uključujući pouzdanost, fleksibilnost, sigurnost i operativnost, uz primenu React.js na klijentskoj strani i Laravel-a na serverskoj strani sa SQL Server bazom podataka. Kroz detaljnu dokumentaciju projektantskih odluka, slučajeva korišćenja i dijagrama sekvenci, rad teži da demonstrira kako se teorijski principi projektovanja arhitekture softvera konkretno primenjuju u praksi.

Rad je podeljen u 6 poglavlja. U drugom poglavlju „Projektovanje arhitekture softvera” opisani su teoretski pristupi analizi zahteva, funkcionalni i nefunkcionalni zahtevi kvaliteta, projektovanje interfejsa sa osvrtom na API patterne, kao i tri metodološka okvira za projektovanje arhitekture: arc42, 4+1 View Model i Software Architecture Inception Canvas. Treće poglavlje predstavlja studijski primer projektovanja softverskog rešenja Estudent, prateći strukturu arc42 templejta. Kreće od definisanja poslovnog konteksta i ograničenja, preko pregleda modula i procesa, do detaljnog projektovanja 10 slučajeva korišćenja sa dijagramima sekvenci. Četvrto poglavlje prikazuje konkretnu realizaciju rešenja na klijentskoj i serverskoj strani, uključujući uputstva za sve slučajeve korišćenja sa komentarima implementacije i API dokumentaciju. Peto poglavlje sadrži zaključak, dok šesto navodi sve korišćene reference i linkove ka Github repozitorijumima projekta.

2. Projektovanje arhitekture softvera

Arhitektura softvera se odnosi na niz projektantskih odluka u vezi sa strukturom i ponašanjem softverskog sistema. Prilikom donošenja tih odluka fokus je na identifikaciji modula, komponenti i interfejsa preko kojih oni komuniciraju u cilju zadovoljavanja funkcionalnih i nefunkcionalnih zahteva sistema [\[4\]](#).

Moduli su logičke apstrakcije delova softverskog sistema koji obavljaju određen posao. Često su dizajnirani nezavisno od drugih modula kako bi minimizovali međuzavisnost (low coupling) i istovremeno enkapsulirali logički povezane podatke i funkcije (high cohesion). Primeri modula su exportovane funkcije, fajlovi, klase i paketi.

Komponente su apstrakcije ponašanja sistema i mogu povezivati više modula, međusobno su nezavisne i obavljaju složenije zadatke na nivou sistema. Primer moderne komponente je mikroservis koji obezbeđuje neku uslugu definisanu biznis logikom. Komponente izlažu interfejse koje druge komponente koriste za međusobnu komunikaciju.

Razlika između toga da li je nešto modul ili komponenta će jasnije biti objašnjena u poglavljima Pregled modula ([Building Block View](#)) i Pregled procesa ([Runtime View](#)).

U praksi su usvojene i najviše korišćene 3 paradigme razvoja softvera:

- paradigma razvoja proceduralnog softvera
- paradigma razvoja objektno-orjentisanog softvera
- paradigma razvoja procesno-orjentisanog softvera

Paradigma razvoja proceduralnog softvera nastala je šesdesetih godina prošlog veka zbog potrebe da se do tada pisani asemblerski kod učini čitljivijim i lakše održivim grupisanjem koda u procedure. Najznačajniji koncepti ove paradigme su top-down projektovanje, strukturno programiranje, moduli i koncept konkurentnog programiranja [\[4\]](#). Top-down projektovanje polazi od dekomponovanja sistema na glavne komponente imajući u vidu nefunkcionalne zahteve, tj. attribute kvaliteta. Potom alokira koje komponente će ispuniti koje funkcionalne zahteve i projektuje interfejse i komunikaciju komponenti [\[4\]](#). Iako je ova paradigma sama po sebi napravila veliki iskorak nad nestrukturiranim asemblerskim programima uvođenjem raznih apstrakcija nad podacima i ponašanjem sistema, taj nivo apstrakcije je idalje bio suviše nizak za predstavljanje koncepta realnog sveta u formi računarskog koda.

Paradigma razvoja objektno-orjentisanog softvera formirala se i razvijala uporedo sa paradigmom razvoja proceduralnog softvera. Zasniva se na objektima kao konceptima apstrakcije strukture realnog sveta. Objekti mogu biti čisto informacioni u kom slučaju sadrže samo attribute (promenljive) ili se mogu dopuniti sa procedurama koje predstavljaju apstrakciju ponašanja realnog sveta [\[4\]](#). Medjutim kritika ove paradigme je da se previše fokusira na definisanje strukture objekata, a manje na dinamiku između objekata, tj. njihovu interakciju. U paralelnom radu više procesa koji žele da pristupe istom objektu javlja se problem konkurentnog pristupa. Konkurentno programiranje rešava taj problem korišćenjem događa tj. uvođenjem kanala ili reda poruka (Message Queue) konačnog kapaciteta, a kasnije kada ni to nije bilo dovoljno uvođenjem koncepta Monitora [\[4\]](#), tj. sredstva sinhronizacije. Monitor je posebna klasa asocirana uz objekat u okviru koje se kao zasebne varijable mogu deklarirati sredstva operativnog

sistema. Kada nit uđe u metodu ili blok koda koji je sinhronizovan korišćenjem monitora, ona blokira pristup drugim nitima dok sama ne napusti tu sekciju.

Paradigma razvoja procesno-orijentisanog softvera prvo je zaživela u velikim kompanijama (Google, Amazon, Meta itd.) zbog toga što projekti rađeni po objektno-orijentisanoj paradigmi nisu mogli dovoljno brzo da odgovore na promene zahteva tržišta i korisnika. Na primer kod web servisa, često se broj klijenata i obim informacija u kratkom periodu mogao znatno povećati. Došlo je do promene načina razmišljanja pri razvoju i održavanja softvera. Kompanije su napustile projektni pristup i prešle na razvoj kroz upravljanje proizvodima. Osnovni zahtev u razvoju softvera postao je da bude napravljen tako da bez značajnijih izmena u kodu podrži brze promene na tržištu i promene u preduzeću [\[2\]](#). Skup zahteva koji su zajednički za većinu velikih preduzeća uključuju distribuiranost, linearnu skalabilnost i pouzdanost [\[4\]](#).

- Distribuiranost - Ovaj zahtev predviđa situaciju kada su delovi multinacionalnog preduzeća i klijenti u različitim državama. Podaci treba da se čuvaju i obrađuju na mestu gde su nastali. Ovo se radi zbog suvereniteta država nad podacima (zaštite osetljivih podataka), veće pouzdanosti, kao i zbog manje latencije, tj. ukupnog vremena koje je potrebno da zahtev stigne od klijentovog uređaja do servera koji može biti na velikoj fizičkoj udaljenosti – stotine kilometara.
- Linearna skalabilnost - Informacioni sistem treba da ima mogućnost jednostavnog povećanja snage obrade i usklađivanja potrebnih računarskih resursa, isključivo operativnim i administrativnim postupcima dodavanja i isključivanja infrastrukturnih kapaciteta, bez dodatnog kodiranja ili promena u softveru.
- Pouzdanost - Informacioni sistem treba da nastavi da radi s punom funkcionalnošću i u slučaju otkazivanja nekog njegovog dela.

Kako bi zadovoljile ove zahteve, organizacije se fokusiraju na kontinuiranu integraciju, kontinuiranu isporuku i kontinuirano postavljanje softvera u produkciono okruženje (Continuous Integration, Continuous Delivery, and Continuous Deployment). Promene koje je potrebno sprovesti opisuju se događajima, pa se proces može posmatrati i kao hronološki uređena sekvenca događaja i servisa koji su vezani za neku pojavu i njen životni ciklus.

2.1 Analiza zahteva

Analiza zahteva (Requirement Engineering) je grana softverskog inženjeringa koja se bavi ciljevima iz stvarnog sveta za funkcije i ograničenja softverskih sistema. Takođe se bavi preciznom specifikacijom ponašanja softvera kako bi ti ciljevi bili zadovoljeni [\[3\]](#).

Ne posvećivanje dovoljno pažnje ovoj fazi može da dovede do potrebe za produžavanjem rokova kasnijih faza projektovanja koje se oslanjaju na ovu kao i dorade konačnog rešenja zbog inicijalno loše definisanih zahteva.

U ovoj fazi potrebno je da se uzmu u obzir želje, interesi kao i ograničenja svih zainteresovanih strana u projektu. Na primer pored očekivanog ponašanja softvera koje je klijent tražio, postoje skrivene potrebe koje nije izrazio, a imaju veze sa atributima kvaliteta, kao i ograničenja zbog budžeta, sposobnosti tima i vremena koje je predviđeno za projekat. U poglavlju analiza zahteva će biti sagledana iz 2 aspekta:

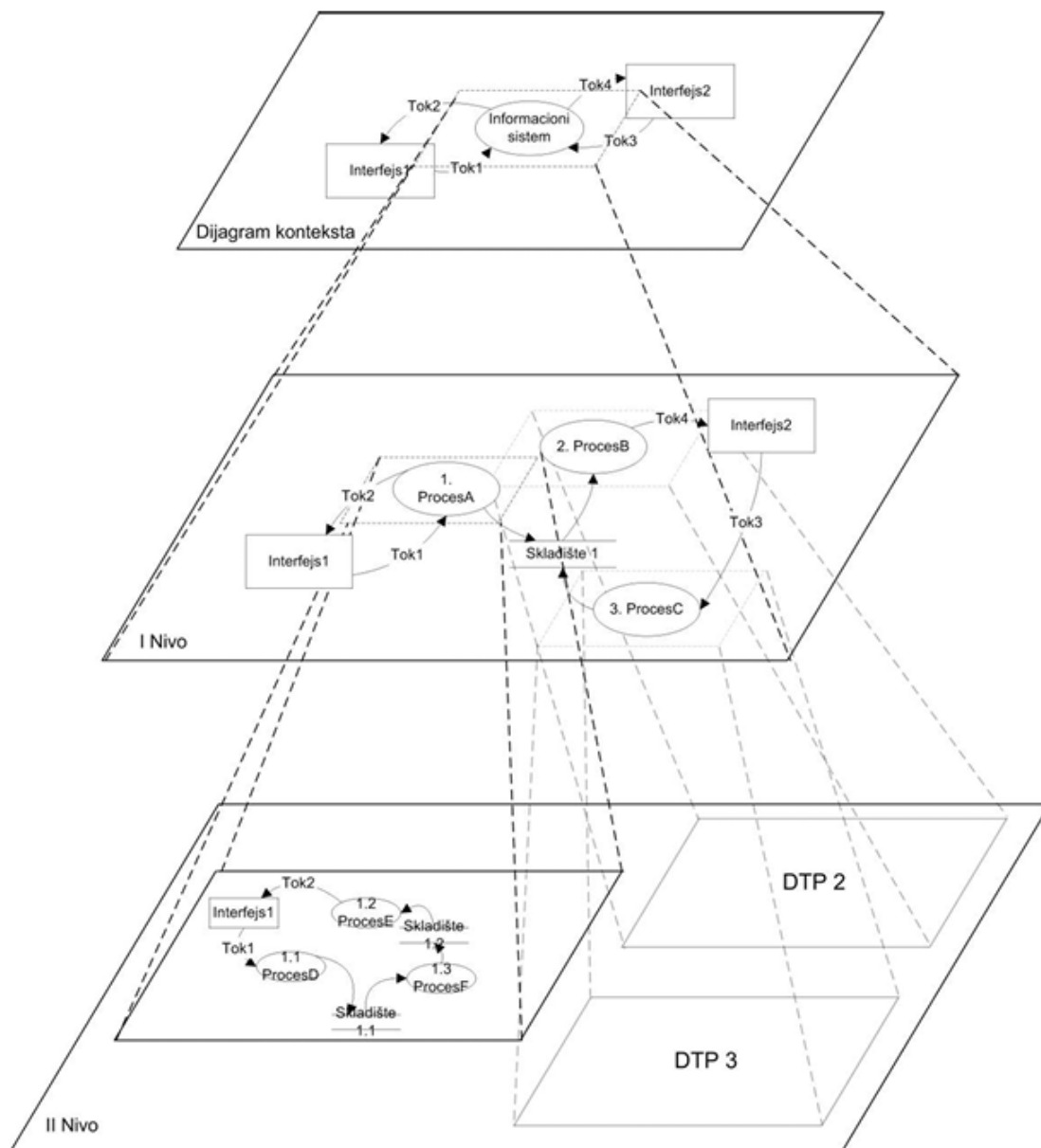
Funkcionalni zahtevi – odgovaraju na pitanje koje ciljeve rešenje treba da zadovolji

Nefunkcionalni zahtevi - odgovaraju na pitanje kako ti ciljevi treba da budu zadovoljeni, uzimajući u obzir da će inicijalni zahtevi evoluirati tokom vremena, odnose se pre svega na atribute kvaliteta koji moraju da budu zadovoljeni i načine na koje merimo tu zadovoljenost.

2.1.1 Funkcionalni zahtevi

Postojeći zahtevi izraženi tekstualnim opisom se prevode u jezik modela radi lakše identifikacije procesa koje rešenje treba da obuhvati. Primer takvih modela su npr dijagrami Strukturne Sistemske Analize(SSA) i Use Case dijagrami.

Strukturna sistemska analiza(SSA) se može koristiti za identifikovanje osnovnih procesa sistema na prvom nivou apstrakcije i takav dijagram se naziva dijagram konteksta. Međutim SSA se češće koristi u detaljnom projektovanju složenijih sistema zbog mogućnosti da se svaki proces dalje dekomponuje na procese nižeg nivoa. Osim procesa i aktora, koji se u SSA zovu interfejsi, na dijagramima se predstavljaju i tokovi podataka i skladišta(tokovi u mirovanju). Na osnovu tokova podataka i skladišta se pravi rečnik podataka koji može biti dobra osnova za projektovanje strukture softverskog sistema . Sa SSA se kroz više nivoa dekompozicije osnovni procesi sagledavaju do dalje nedeljivih, primitivnih procesa. Ovi primitivni procesi su u stvari slučajevi korišćenja.

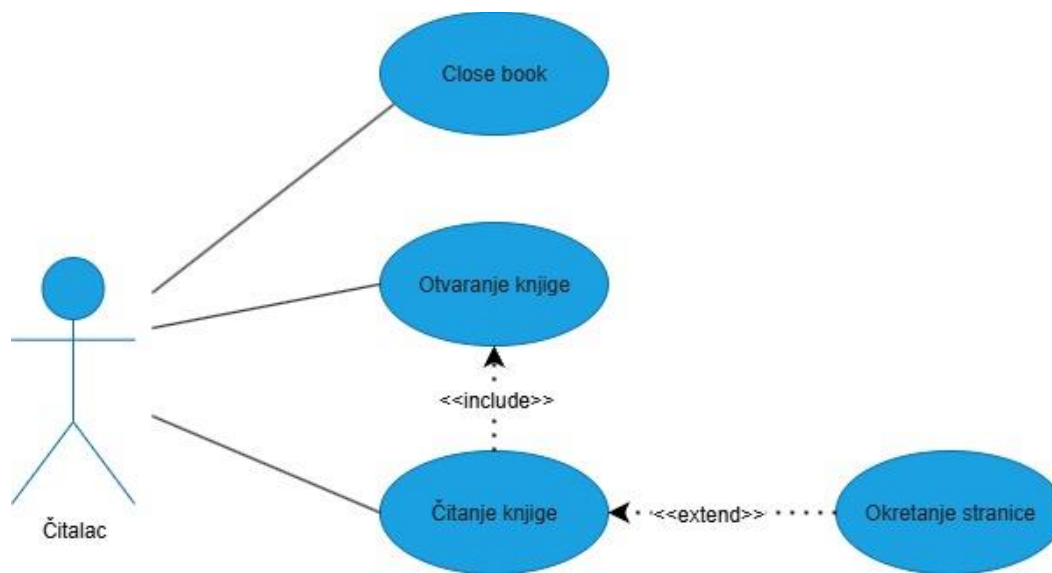


Slika 1: Primer SSA analize na 2 nivoa dekompozicije [\[6\]](#)

UML UseCase dijagrami ili dijagrami slučajeva korišćenja predstavljaju procese sistema iz perspektive interakcija korisnika. Kroz oblačiće se predstavljaju različite funkcionalnosti sistema kao i njihove međusobne zavisnosti. Korišćene notacije su:

<<includes>> kada jedan slučaj korišćenja uvek podrazumeva korišćenje drugog da bi mogao da se izvrši. Čitač mora otvoriti knjigu da bi je pročitao

<<extends>> kada jedan slučaj korišćenja neobavezno uključuje drugi kao opciju. Čitač može čitati knjigu, a zatim može i ne mora okrenuti stranu)

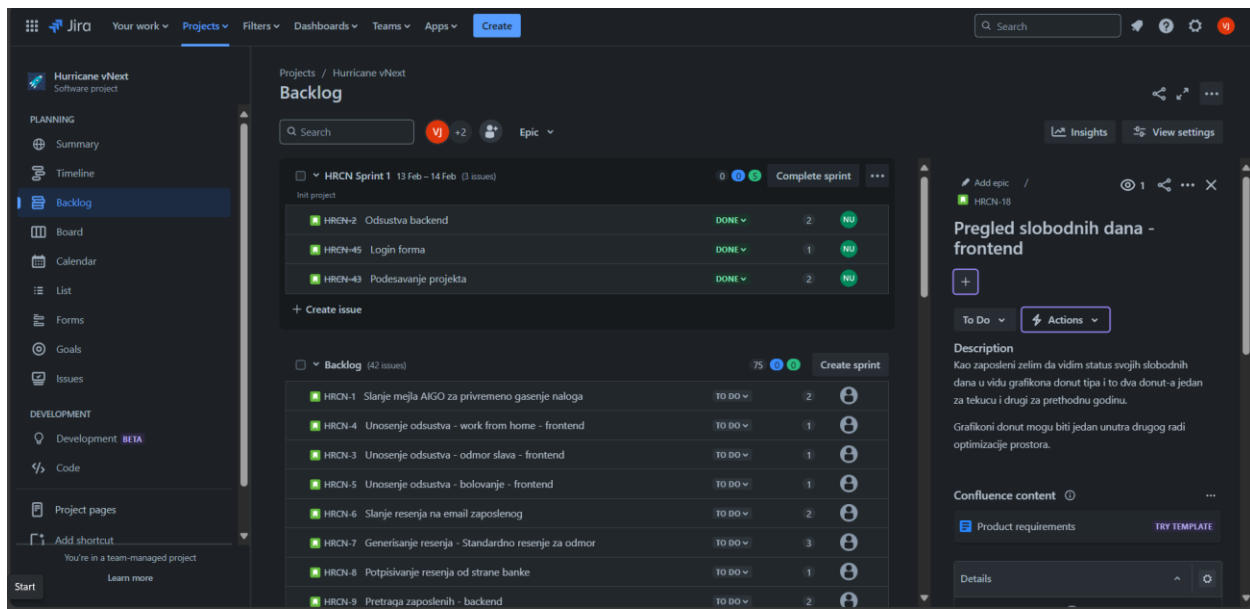


Slika 2: Primer UML Use Case dijagrama crtana po uzoru na [\[5\]](#)

Dijagram slučajeva korišćenja opisuje posao koji treba da bude urađen, međutim ne definiše ko će u razvojnom timu programera biti zadužen za rad na implementaciju kojeg procesa, kojim redosledom, koji je kriterijum ocene da je funkcionalnost ispravno implementirana i da će kvalitet zadovoljiti korisnike itd. Odgovor na ova pitanja daje metodologija razvoja kroz user storije.

User storiji predstavljaju templejt za zapisivanje scenarija iz ugla korisnika sa odgovarajućom ulogom. Opisuju mogućnost koju sistem pruža korisniku sa tom ulogom i kako on vidi vrednost koju mu sistem pruža, kao i zašto mu je ta odlika sistema bitna. Mapiraju se 1 na 1, ili više na 1 u odnosu na slučajeve korišćenja. Templejt je obično u formatu: Kao <ime uloge> očekujem da <mogu/želim da mogu> da<mogućnost/osobina>, tako da mogu da <dobijena vrednost>, zato što <razlog>.

Koriste se u okviru Agilnih metoda razvoja softvera kao taskovi koji se dodeljuju timovima i pojedincima u 2 nedeljnim periodima tj. sprintovima. Prvobitno se taksovi organizuju po bitnosti, a zatim se boduju po očekivanoj težini relativno jedni na druge, subjektivnom procenom, korišćenjem najčešće fibonačijevog niza brojeva 1, 2, 3, 5, 8, 13 itd. Na kraju svakog sprinta se sagledavaju rezultati, koliko bodova je ostvareno, kakvi problemi su postojali i kako se mogu prevazići u idućem sprintu. Nekada se obim sprinteva menja, zahtevi se menjaju ili se dodaju novi, zato se kaže da je ova metoda agilna i prilagođena promenama. Na većim projektima koji traju duže od nekoliko meseci kada se timovi na projektima uigraju, sprintevi za menadžere mogu predstavljati dobru metriku za procenu očekivanog trajanja projekta i u skladu sa tim mogu bolje da alociraju resurse ili obaveste klijente na vreme o potencijalnim problemima sa ispunjenjem obećanih rokova.



Slika 3: Primer definisanja UserStorija kroz alat za kolaboraciju Jira

2.1.2 Atributi kvaliteta

Krajnji korisnik kao i klijent često ne znaju kako da precizno definišu koliko brz, skalabilan ili siguran softver treba da bude, posebno u fazama razvoja. Softver koji funkcioniše dobro sa manjim brojem klijenata možda neće moći da usluži klijente dovoljno brzo kada njihov broj premaši određenu cifru.

Projekat koji je previše prilagođen za jednu bazu podataka i jedan operativni sistem može zahtevati kasnija proširenja, što može rezultovati dodatnim troškovima ako ti zahtevi nisu uzeti u obzir od samog početka. Na primer, softver koji radi isključivo sa MySQL bazom podataka na Windows sistemu može se suočiti sa značajnim izazovima ako se kasnije uvede zahtev za podršku PostgreSQL baze na Linux sistemu.

Sistem može raditi sa osetljivim podacima i potrebno je uzeti u obzir mogućnost otkaza u slučaju nestanka struje, internet konekcije ili greške. U takvim slučajevima, sistem treba biti distributivan, raspoređen na više mašina koje mogu preuzeti teret servera koji je otkazao.

Atributi kvaliteta su uvedeni sa procesno orijentisanom paradigmom i kao što je ranije navedeno, predviđaju softver koji je projektovan tako da podrži brze promene na tržištu i u preduzeću kroz administrativne postupke dodavanja i isključivanja infrastrukturnih kapaciteta, bez dodatnog kodiranja ili promena u softveru. Ovakvim problemima se bave DevOps inženjeri korištenjem alate poput Kubernetesa, Apache Kafke ili Nginx-a. Prvo pitanje koje se postavlja je kako se definišu i mere nefunkcionalni zahtevi? Odgovor su metrike. Za svaki od zahteva predviđa se najgori slučaj ili najveći mogući kapacitet koji sistem treba da podrži. U implementaciji ovakvih rešenja, pišu se ili sprovode testovi gde se simuliraju ovakve situacije [\[7\]](#).

Na primer, meri se da li je prosečno vreme odgovora servisa na zahtev manje od 1.2 sekunde u slučaju 10.000 istovremenih korisnika, ili da li poruka u čet aplikaciji stiže za manje od 0.2 sekunde u slučaju 100 ulogovanih korisnika ili koje vreme osvežavanje stranice internet prodavnice je prihvatljivo u slučaju

velikog broja korisnika (npr. za Crni petak). Ovakvi kriterijumi se nazivaju acceptance kriterijumi i često su deo user storija. Kriterijumi prihvatanja predstavljaju konačnu proveru kojom se smatra da je task u sprintu završen. U nastavku su neki od atributa i metrike koje se koriste.

- **Distribuiranost:** Ovaj atribut se odnosi na sposobnost sistema da ispunjava sve druge zahteve kvaliteta na fizički velikim udaljenostima, kada je sistem distribuiran na više servera. Na primer, globalni sistem za upravljanje zalihama može biti distribuiran kako bi obradio podatke u realnom vremenu iz skladišta širom sveta. Distribuiranost se može meriti metrikom kao što je latencija među distribuiranim serverima u različitim regionima i pouzdanost replikacije podataka, jer mikroservisi koji rade paralelno i pristupaju podacima u okviru latencije mogu čitati različite verzije tog podatka.
- **Performanse + Skalabilnost:** Aplikacija ima dobre performanse i skalabilna je kada ima kratak response time (vreme odziva) ili visok throughput tj. broj zahteva po sekundi koje sistem uslužuje uprkos latenciji i tokom perioda kada se očekuje najviše klijenata. Skalabilnost aplikacije bez menjanja koda se postiže korišćenjem load balansera tj. middleware-a koji teret ravnomerno raspoređuje na druge servere kako bi vreme odziva ostalo optimalno.
- **Pouzdanost (Reliability):** Ovo se odnosi na sposobnost sistema da funkcioniše bez grešaka tokom određenog perioda i izražava se kroz dostupnost sistema. Na primer, telekomunikaciona mreža može zahtevati 99.999% dostupnost, što znači da mreža može biti nedostupna maksimalno 5.26 minuta godišnje. Dostupnost se meri metrikama koje izražavaju vreme između otkaza (Mean Time Between Failures - MTBF) i vreme do oporavka (Mean Time to Recovery - MTTR).
- **Upotrebljivost (Usability):** Odnosi se na lakoću korišćenja sistema. Primeri uključuju prosečan broj klikova potrebnih da bi korisnik izvršio određenu operaciju, ocenu zadovoljstva korisnika iz anketiranja ili vremena potrebnog za obuku novog korisnika. Na primer, aplikacija za online bankarstvo treba da omogući korisnicima plaćanje računa u tri jednostavna koraka.
- **Prenosivost (Portability):** Odnosi se na sposobnost softvera da radi na različitim platformama. Metrika može biti procenat neprenosivog koda ili vreme potrebno za prilagođavanje softvera za novu platformu. Na primer, desktop aplikacija dizajnirana za Windows može biti prenosiva na macOS uz minimalne izmene koda.

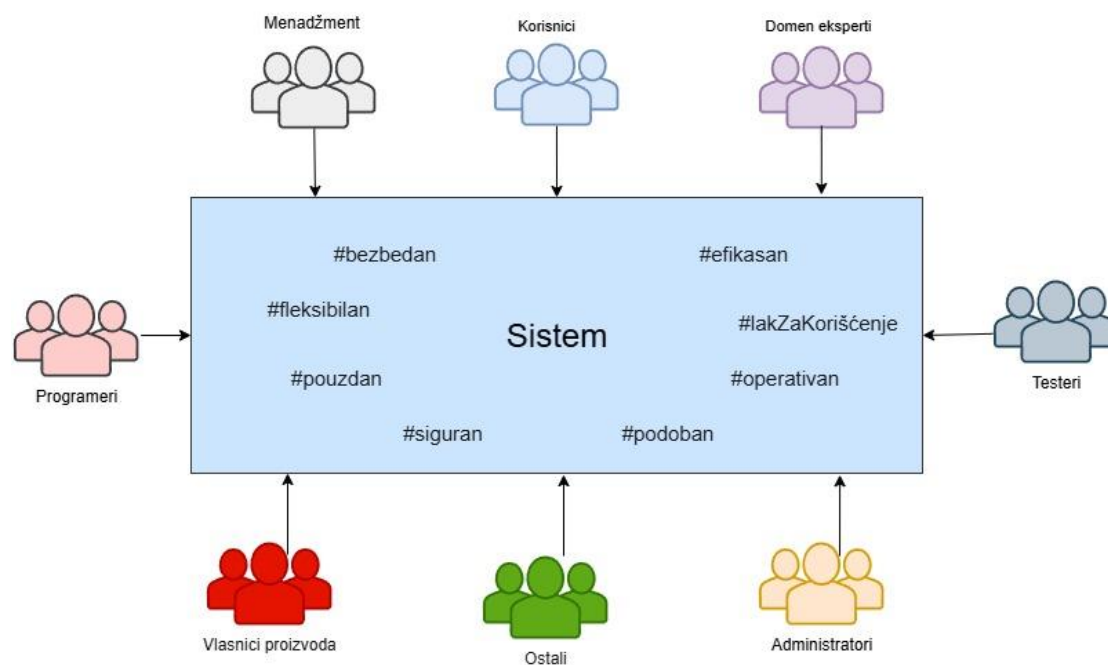
Nefunkcionalni zahtevi se često odnose na funkcionisanje aplikacije kao celine. Međutim, nekada je lakše započeti definisanje nefunkcionalnih zahteva samo za određenu bitnu komponentu ili grupu komponenti, gde se očekuju problemi. Ovi zahtevi bi trebalo da budu u vezi sa ciljevima biznisa. Potrebno je proceniti koji nefunkcionalni zahtevi su bitniji od drugih za zadovoljstvo korisnika, kao i kako se zahtevi uklapaju sa finansijskim budžetom i vremenom potrebnim za njihovu implementaciju. Dobra praksa prilikom implementiranja nove aplikacije ili njenog modula je proučiti postojeće standarde u delatnosti, kao i konkurente, za primere onog što bi moglo da se unapredi i kako se te metrike mogu implementirati. Na primer, bankarska industrija može koristiti standarde za bezbednost podataka kao što su PCI DSS za aplikacije koje obrađuju kreditne kartice [\[7\]](#).

Čak i u slučaju prototipa ili manjih aplikacija koje ne očekuju previše saobraćaja, osveščivanje značaja atributa kvaliteta programerima je u današnje vreme postalo esencijalno u svim ozbiljnim firmama. Firme

su uvidele da uključivanje nefunkcionalnih zahteva u projektno rešenje vodi ka zadovoljnjim klijentima kao i ka lakšem dugogodišnjem održavanju softverskih rešenja.

2.1.2.1 arc42 Quality Model (Q42)

Arc42 Quality Model predstavlja klasifikaciju najbitnijih atributa kvaliteta po stakeholderima. ISO-25010 predlaže fiksnu hijerarhiju kvaliteta, uređenu poput stabla, koja može delovati prekomplikovano za nekog ko želi da se fokusira na najbitnije kvalitete koje njegova aplikacija treba da poseduje. Q42 predlaže samo 8 tagova koji poput nadtipova zajedno obuhvataju više od 100 tradicionalnih zahteva kvaliteta [\[14\]](#).



Slika 4: 8 ključnih sistemskih atributa, kao fokus 8 različitih uticajnih grupa, crtana po uzoru na [\[14\]](#)

Tabela 1: Uticajne grupe i njihova očekivanja

Tip stakeholdera	Očekivanja
Korisnici	upotrebljiv u smislu pružanja očekivanih i željenih funkcija, lak i prijatan za korišćenje, pouzdan, dostupan kada je potreban, bezbedan
Menadžment	efikasan u smislu troškova razvoja i rada, pouzdan
Domene eksperti	pouzdan i efikasan u pružanju potrebnih funkcija, fleksibilan u pogledu dodavanja novih funkcija
Vlasnici proizvoda	fleksibilan za proširenje, dodavanje novih funkcionalnosti
Programeri	fleksibilan i pouzdan u smislu održavanja koda, lako testabilan
Testeri	testabilan uz odgovarajući trud, fleksibilan u pogledu automatizacije, pouzdan u smislu ponovljivih rezultata testa

Administratori	operativan i pouzdan u pogledu jednostavnog upravljanja sistemom, siguran i pouzdan prilikom integracije sa drugim sistemima, fleksibilan u pogledu radnih okruženja, siguran i bezbedan
Ostali	-

Tabela 2: Oznaka kvaliteta/značenje

Oznaka	Značenje
#reliable	dostupan, robusan, pouzdan, otporan na kvarove, tolerantan na greške, dosledan, oporavljiv, bezbedan, siguran
#flexible	fleksibilan tokom razvoja (lak za promene i proširenja), fleksibilan u pogledu infrastrukture i okruženja, fleksibilan u radu (konfigurativan, prilagodljiv, integrabilan)
#efficient	brz, responzivan, efikasan u pogledu memorije, CPU-a, niti, mreže i drugih resursa, efikasno održiv i operativan
#usable	lak i prijatan za korišćenje, atraktivan, estetski dopadljiv
#operable	lak za instalaciju, rad, nadzor i administraciju, sa adekvatnim operativnim troškovima
#suitable	pruža odgovarajuće karakteristike i funkcije
#secure	dosledan, odgovoran, poverljiv, sa integritetom
#safe	otporan na kvarove, izoluje greške, upozorava na opasnosti rano, prepoznaje rizike rano

2.2 Projektovanje interfejsa

Rezultat faze projektovanja interfejsa treba da definiše sledeće informacije: precizan opis događaja ili poruka iz okruženja na koje sistem treba da odgovori, precizan opis događaja i poruka koje sistem treba da kreira, specifikaciju podataka i formata podataka koji ulaze ili izlaze iz sistema, specifikaciju dinamike interakcija. [\[4\]](#). Kada se govori o projektovanju interfejsa ono se posmatra u glavnom iz dva aspekta:

- Eksternih interfejsa na nivou sistema
- Aplikacionih programskih interfejsa(API)

Projektovanje eksternih interfejsa odnosi se na specifikaciju interakcije sistema sa okruženjem. Na ovom nivou se zanemaruje interna struktura softvera, a sistem se tretira kao crna kutija. Fokus prilikom projektovanja je na komunikaciji sistema sa korisnicima i drugim aplikacijama ili uređajima.

Aplikacioni programski interfejs (Application Programming Interface, skr. API) je sredstvo komunikacije između komponenti, modula i servisa savremenih distribuiranih softverskih aplikacija. Predstavlja „most“ koji različitim programskim entitetima omogućava da efikasno komuniciraju i razmenjuju podatke [\[8\]](#).

2.2.1 Paterni arhitekture API

Kada se govori o generalnom pristupu prilagođavanja API-a postoje 2 glavna pristupa:

- Inside-out API dizajn - Počinje od serverske strane. Interna logika i mogućnosti sistema se izlažu kroz API, ovaj pristup je jednostavniji za implementirati i intuitivan programerima serverske strane.

Međutim, ovaj pristup često dovodi do tesno povezanih (tight-coupled) *API-ja* koji su složeniji za korišćenje i održavanja [\[9\]](#).

- Outside-in *API* dizajn - Prilagođava se potrebama i očekivanjima krajnjih korisnika *API-ja*. Fokusira se na jednostavnost, intuitivnost i apstrakciju interne kompleksnosti sistema. Rezultat su fleksibilni i lako održivi *API-ji* koji su manje zavisni od unutrašnje implementacije [\[9\]](#).

Osim ovog patterna *API* direction na slici postoji još dosta *API* patterna koji na najvišem nivou apstrakcije olakšavaju proces projektovanja dobro organizovanih i čitljivih *API-ja*. Podeljeni su u nekoliko grupa i svaka grupa rešava određen set problema



Slika 5: Paterni arhitekture aplikacionih programskih interfejsa, preuzeto sa [\[10\]](#)

- Vidljivost *API-ja* (*API Visibility*), definišu ko ima pristup *API-ju* i pod kojim uslovima. Fokus je na kontroli izloženosti funkcionalnosti sistema , da li su servisi namenjeni široj javnosti, ograničenim partnerima ili isključivo internim komponentama.
- Smer *API* komunikacije (*API Direction*) opisuje ulogu *API-ja* u komunikacionoj arhitekturi sistema, odnosno da li služi kao veza prema korisničkom interfejsu ili kao sredstvo integracije između serverskih komponenti. Fokus je na razdvajanju odgovornosti i optimizaciji komunikacije u zavisnosti od sloja sistema.
- Osnovna struktura poruke(*Basic Message Structure*) opisuje načine strukturiranja poruka u komunikaciji izmedju api-a. Strukture omogućavaju bolju preglednost i lakšu obradu, gde sam podatak u paru ključ-podatak može biti niz podataka ili drugi par ključ-podatak, tj stablo.

- Uloge endpoint-a i operacione odgovornosti (*Endpoint Roles and Operation Responsibilities*) Definiše koju funkcionalnu ulogu pojedinačni endpoint ima u sistemu i kakvu vrstu operacija izvršava. Fokus nije samo na tome šta endpoint radi, već i na jasnom razdvajanju odgovornosti. Da li api obrađuje podatke(CRUD operacije), izvršava akcije promene statusa u vezi sa poslovnom logikom ili omogućava pristup drugim apijima, poput posrednika.
- Dizajn zahteva i odgovora (*Designing Request and Response Message Representations*) Predlaže strukturu zahteva i odgovora radi standardizacije procesa kao što su identifikacija, obrada grešaka, kako bi komunikacija bila konzistentna, razumljiva i pouzdana.
- Unapređenje dizajna poruka (*Refine Message Design for Quality*) se bavi tehnikama dizajna zahteva koje poboljšavaju performanse i fleksibilnost razmene podataka. To ostvaruju kontrolom veličine odgovora, smanjenjem broja zahteva i prilagođavanje odgovora potrebama klijenta.
- Strategije evolucije API-ja (*Decide on API Evolution Strategies*) definiše kako se API razvija kroz vreme bez narušavanja postojećih integracija. Fokus je na verzionisanju endpointa, kako bi se omogućio kontinuirani razvoj sistema uz očuvanje kompatibilnosti sa postojećim korisnicima.
- Dokumentovanje API-a (*Document API Contract*) govori o tome šta svaka dokumentacija api-a treba da ima. Podrazume opis kako se API koristi, koje su njegovi očekivani odgovori tj. scenariji upotrebe i ograničenja, čime se omogućava pouzdana integracija i očekivano ponašanje sistema.

2.2.2 API Arhitekturalni stilovi i protokoli

Za razliku od paternarne arhitekture *API-ja*, koji rešavaju specifične probleme, *API* stilovi opširnije sagledavaju probleme adresirajući najbolje prakse iz više paternarne.

REST-Transfer stanja reprezentacijom resursa (*Representational state transfer*) je najpopularniji arhitekturalni stil koji je danas skoro postao standard pri dizajniranju klijent-server *API*-a. Prvi put spomenut u doktorskoj disertaciji Roy Fielding-a 2000. godine, predlaže *stateless* dizajn, tj. ne skladištenje podataka ili statusa između poziva. Koristi *http* protokol za prenos podataka i bazira se na konceptu resursa koji se identifikuju preko univerzalnog identifikatora (*URI*). Preporučuje keširanje odgovora na iste zahteve na strani klijenta i servera radi poboljšanja performansi. Takođe preporučuje korišćenje *HATEOAS*(Hypermedia as the engine of application state) koji je u praksi retko korišćen, jer developeri predviđaju strukturu poziva kroz strukturu *URI*-a. Po pravilu, endpointi (ili rute, adrese za usmeravanje) za pristup funkcionalnostima *API*-ja treba da budu precizno i nedvosmisleno definisane, korišćenjem *http* metoda (*get*, *post*, *put*, *delete*). Tu se pre svega misli na redosled resursa i operacija, sve rute koje se odnose na isti resurs treba da počinju na isti način, dok operacije nad resursima stoje na kraju. Odgovor takođe treba da bude dosledno definisan, najčešće u *JSON* ili *XML* formatu [\[9\]](#). Proces dokumentovanja i testiranja *API*-ja može se automatizovati korišćenjem alata poput *Swagger API*. U nastavku su definisani neki od češće korišćenih protokola u *http* komunikaciji:

- RPC Protokol za pozivanje udaljenih procedura (engl. Remote Procedure Call) koristi se u sinhronoj komunikaciji za slanje parametara distribuiranom servisu na obradu i prijem rezultata obrade. RPC pozivaju izvršne akcije ili procese na istom sistemu gađanjem imena ili adrese procedura ili na udaljenim sistemima kroz SOAP protokol specifikovanjem imena i parametra kroz XAML. Takođe postoji gRPC, koji je razvio google i on koristi http 2.0 protokol koji podržava serijalizaciju JSON dokumenata i najčešće se koristi u mikroservisnoj arhitekturi [\[4\]](#).
- WebSocket prokol se koristi za dvosmernu interaktivnu komunikaciju klijenta i servera u realnom vremenu (full-duplex). Značajno se razlikuje od ostalih prokola jer ne koristi više http zahteva već klijentsko/serverske zahteve i odgovore šalje preko jedne trajne konekcije uspostavljene preko web socket tehnologije.
- Webhook protokol je asinhroni event-driven mehanizam koji omogućava klijentu da pretplati na događaj nastalim u toku izvršavanja servisa na serveru, specifikovanjem callback URL-a na koji klijent želi da bude obavešten, tj. na kojem čeka odgovor[12]. Služi za prenošenje jednostavnih obaveštenja zasnovanih na događajima, ne podržava složene upite ili operacije.
- HTTP streaming je protokol prenosa multimedijalnih podataka putem http protokola u realnom vremenu.
- SOAP (engl. *Simple Object Access Protocol*) je http klijent-server arhitektura definisana korišćenjem XAML.
- GraphQL, upitni jezik koji je razvio fejsbuk.

2.3 Metodološki okviri za projektovanja arhitekture softvera

2.3.1 Acr42

Acr42 je templejt za dokumentaciju arhitekture softvera. Sastoji se od 12 poglavlja i kroz njih sažeto, ali jasno komunicira arhitekturu softvera iz različitih uglova gledanja.

Tih 12 poglavlja su:

1. Uvod i ciljevi
2. Ograničenja
3. Kontekst i opseg rešenja
4. Strategija rešenja
5. Pregled modula (Building Block View)
6. Pregled Procesu(Runtime View)
7. Pregled Raspoređivanja (Deployment View)
8. Crosscutting Koncepti
9. Odluke o arhitekturi (Architecture Decision)
10. Zahtevi kvaliteta

11. Rizici i tehnološki dug

12. Rečnik pojmova

Poglavlje 1: Uvod I ciljevi se dalje može podeliti na 3 podpoglavlja.

- 1.1 Pregled zahteva - Opisane su osnovne funkcionalnosti koje softver pruža kao i svrha koju treba da zadovolji
- 1.2 Ciljevi kvaliteta - služi kao vizija projekta, nešto čemu aplikacija teži npr. upotrebljivost, efikasnost, interoperabilnost, testabilnost itd.
- 1.3 Stejkholderi - svi ljudi koji imaju uticaja na rešenje. Obuhvata tim ljudi koji radi na izradi rešenja: developeri, softver arhitekta, project owneri, tester, ljudi koji poznaju biznis logiku procesa za koji se aplikacija pravi, partneri, klijenti itd.

Poglavlje 2: Ograničenja predstavlja ograničenja u smislu određenih softverskih alata, okruženja, hardvera ili ljudskih resursa koji su direktno uticali na arhitekturu. U nekim slučajevima može i ne mora dati razloge iza određenih odluka, predstavlja se u glavnom u tabelama sa 2 kolone: ograničenje i motivacija/razlog. Posmatra se iz 3 aspekta.

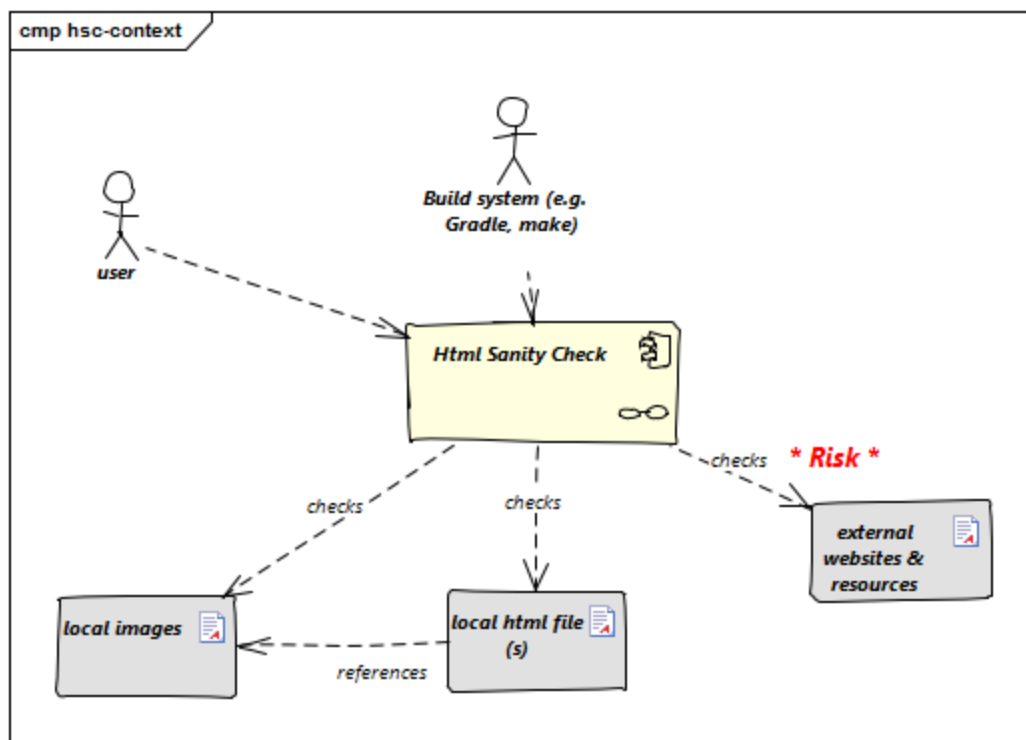
2.1 Tehnička ograničenja – predstavlja softverska, programerska i hardverska ograničenja

2.2 Organizaciona ograničenja – predstavlja odluke vezane za organizaciju ljudi koji će raditi na projektu, odluke o razvojnom okruženju. Metodologije i filozofije kojima je projekat vođen, npr risk driven arhitektura, iterativna, inkrementalna metoda kao i npr arc42 kao metodologija tj. template za dokumentovanje softvera.

2.3 Konvenciona ograničenja - Primer bi bio konvencija jezika koji je korišćen za implementaciju. Ako je jezik Java, treba pratiti konvenciju imenovanja promenljivih po camel notaciji, jezik koji je korišćen za komponente u projektu bi takođe trebalo da bude naveden ovde i da se developeri kasnije tog jezika i pridržavaju, npr ako je jezik engleski onda sve promenljive treba da budu na engleskom, a ne neke na srpskom, neke na engleskom. Takođe mogu biti navedene i neke konvencije specifične za ovaj projekat, npr posebna ekstenzija za fajlove koje aplikacija proizvodi.

Poglavlje 3: Kontekst i opseg rešenja odgovara na pitanje ko su korisnici sistema i od kojih drugih sistema zavisi. Posmatra se iz 2 aspekta.

3.1 Biznis kontekst – predstavlja sistem kao crnu kutiju i fokus je na tokovima informacija ka spoljnim sistemima, korisnicima ili uređajima i nazad.



Slika 6: Primer Biznis Kontekst Diagrama – Uređaj proveru validonost HTML linkova (HTML Sanity Check), preuzeto sa [23]

3.2 Tehnički kontekst – predstavlja predhodne uređaje, aktore i sisteme kao i na biznis kontekstu, ali ovaj put predstavljajući i protokole komunikacije kojim se primaju i šalju ulazni i izlazni podaci.

Poglavlje 4: Strategija rešenja prikazuje se često u formatu liste ili tabele i opisuje prvo ciljeve kvaliteta ili izazove sa kojima se projekat suočava, a zatim opisuje konkretne pristupe, alate, jezike, kojim planira da ih prevaziđe. Opis treba da bude kratak i po potrebi u koloni Link to Details da specificira link ka poglavlju dokumentacije gde čitalac može opširnije da se informiše o konkretnoj strategiji rešenja.

Tabela 3: Templejt tabele ciljevi kvaliteta u strategiji rešenja

Quality goal	Scenario	Solution approach	Link to Details
<Q-goal 1>	<Text>	<Text>	<Link>
<Q-goal 2>	<Text>	<Text>	<Link>

Poglavlje 5: Pregled modula (Building Block View) - Predstavlja dekompoziciju logičke strukture sistema na gradivne blokove (module, klase, pakete, funkcije) na više nivoa apstrakcije. Za prikaz se mogu koristiti UML Diagrami Komponenti, UML Dijagram Paketa, ER dijagram (Entity-Relationship Diagram) za grafički prikaz relacionog modela.

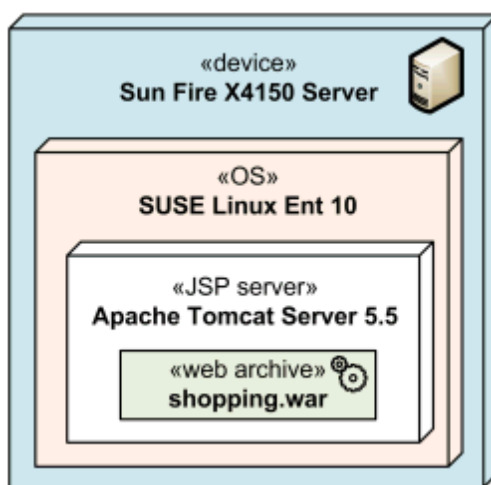
Poglavlje 6: Pregled procesa (Runtime View) - Predstavlja ponašanja sistema kroz procese između komponenti. Fokus ovog poglavlja je da ilustruju koje komponente inicijalizuju procese i šta su inputi i outputi koje svaka od komponenti proizvodi tokom procesa. Za prikaz se mogu koristiti UML Diagrami sekvenci ili UML Dijagram Aktivnosti.

Poglavlje 7: Pregled Raspoređivanja (Deployment View) - Predstavlja se Deployment Diagramom koji prikazuje fizički deployment softverskih artefakata na hardware nodovima.

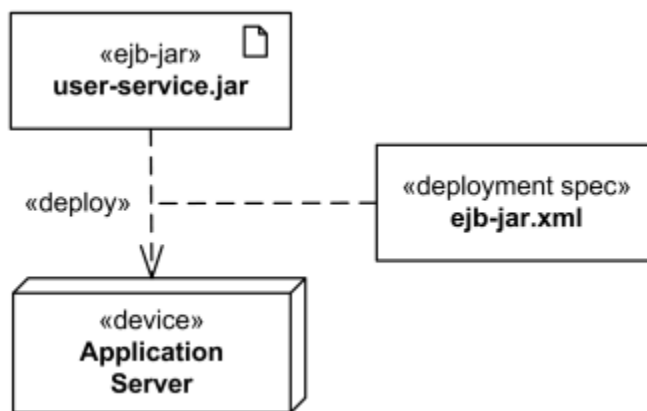
Artefakt (eng. *artifact*) – bilo koja vrsta informacije proizvedena kroz proces razvoja softvera, predstavlja implementaciju softverskih komponenti, npr. : egzekutabilni fajlovi, konfiguracioni fajlovi, skripte, tabela baze podataka.

Node (čvor) – hardverski uređaj ili softversko izvršno okruženje, mogu da ugnježdavaju (nestuju) druge nodove unutar sebe.

Deployment dijagram prikazuje nodove u okviru kojih se izvršavaju artefakti kao i veze asocijacije i zavisnosti između artefakta. Ovi dijagrami po potrebi mogu biti veoma detaljni i tačno specificovati tip procesora i operativni sistem svakog hardverskog noda, takođe ovo je mesto gde se specificiraju različita okruženja npr. development, testno i produkciono okruženje, load balanceri, ruteri, firewallovi itd.



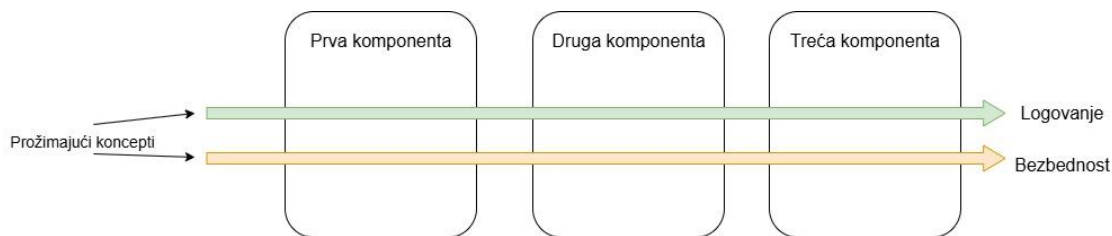
Slika 8: Nekoliko izvršnih okruženja ugnježđenih na istom serverskom uređaju, preuzeto sa ^[16]



Slika 9: Primer specifikacije deployment artifakta, preuzeto sa ^[16]

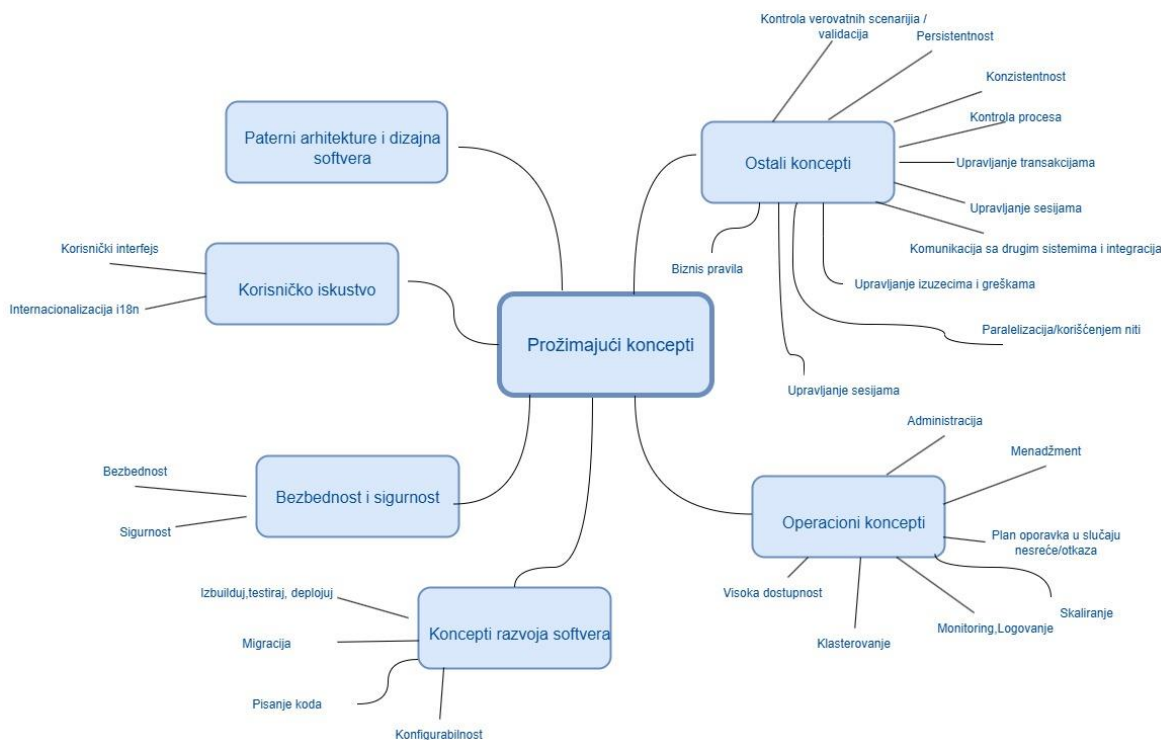
Na slici 9, dodavanje specifikacije pri deploymentu egzekutabilne .jar biblioteke na aplikacionom serveru je suština devops metoda koji su spominjane u poglavlju [2.1.2 Atributi kvaliteta](#). Kroz konfiguracione fajlove moguće je promeniti nivoe logovanja, promeniti dodeljene resurse ili konekcione parametre prema bazi bez potrebe da se modifikuje sam izvršni kod.

Poglavlje 8: Prožimajući (Crosscutting) koncepti, ova sekcija obuhvata širok spektar koncepata koji se prožimaju kroz celu arhitekturu i ne mogu se pripisati samo određenom delu aplikacije npr logovanje, testabilnost, security.



Slika 10: Prožimajući koncepti kroz komponente, po uzoru na [25]

Forma ovog poglavlja je prilagodljiva potrebama samog projekta i može obuhvatati po potrebi kratke opise svih koncepata koje autor želi detaljnije da objasni na jednom mestu jer se spominju na više mesta, ali nisu svojstveni ni za jedno drugo specifično poglavlje.



Slika 11: Koncepti koji se mogu pojaviti u poglavlju prožimajućih koncepata, slika radjena po uzoru na [24]

Poglavlje 9: Odluke o arhitekturi se dotiče odluka koje direktno utiču na performanse i kvalitet aplikacije, motivacije iza takvih odluka, tj. racionalizacijom zašto je baš to rešenje izabrano.

Poglavlje 10: Zahtevi kvaliteta, obuhvata sve zahteve kvaliteta popisane u formi stabla ili tabele sa kolonama zahteva kvaliteta i preduzetih mere da zahtev bude ispunjen. Zahtevi se mogu grupisati npr zahtevi koji se odnose na ispunjene ciljeva iz poglavlja 1.2 Ciljevi kvaliteta, npr kada pričamo o Security aspektu na primeru websajta za rad sa poreskim prijavama, gde obični radnici banke ne bi trebalo da mogu

da menjaju role drugim radnicima sem ako nemaju ulogu administratora, to se može postići kako ograničavanjem renderovanih komponenti na klijentskoj strani za korisnike sa nedovoljnom autorizacijom, kao i dodavanjem safe guardova na endpointe na serverskoj strani kao dodatni vid autorizacije.

Poglavlje 11: Rizici i tehnološki dug predstavlja listu identifikovanih tehničkih rizika i/ili tehničkih dugova, sortiranih po prioritetu.

Rizici su uglavnom izazvani faktorima koji su u trenutku projektovanja nepoznati timu. Mogu biti vezani za nedostatak dokumentacije biznis logike ili znanja određene tehnologije ili alata koja je predviđena za izradu projekta kao i njihove međusobne kompatibilnosti. Faktori rizika takođe mogu imati veze sa stakeholderima koji ne mogu da daju dovoljno jasne detalje željenog rešenja, zbog čega postoji rizik od izrade nezadovoljavajućeg rešenja.

Razlog za tehnološki dug mogu npr biti rokovi koji predviđaju demo ili produkciju verziju pre nego što su svi predviđeni featuri implementirani, u tom slučaju arhitekta ili developer može odlučiti da napravi kompromis predviđenoj arhitekturi, izostavljajući neke manje bitne feature i testove.

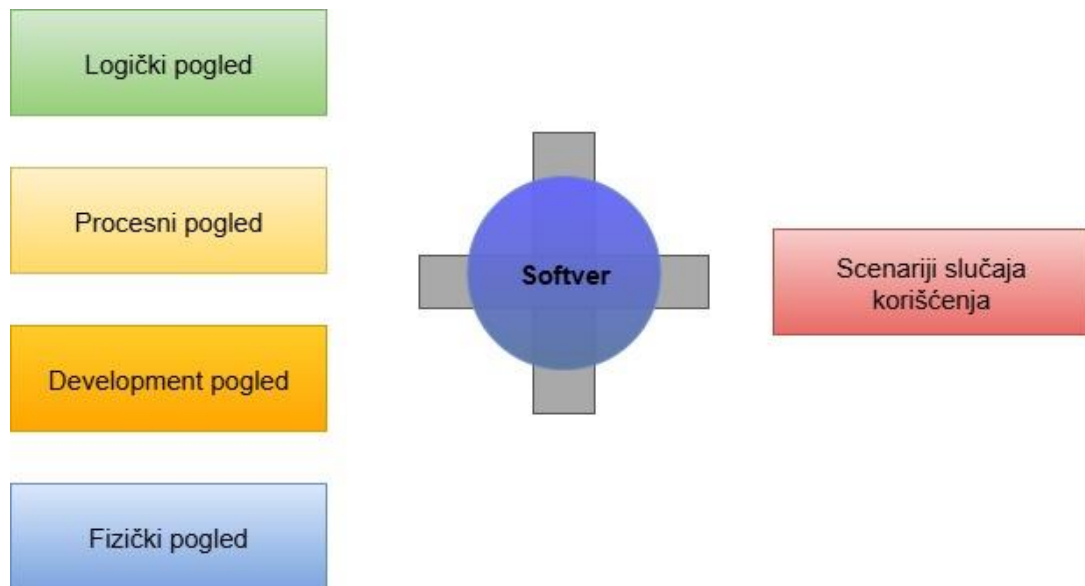
Poglavlje 12: Rečnik pojmova služi da objasni često spominjane tehničke termine u ostatku dokumentacije, kako bi svi stakeholderi koji čitaju ovaj dokument imali jasno, zajedničko razumevanje na šta se ti pojmovi odnose.

2.3.2 4+1 View Model

4+1 View Model je još jedan popularan templejt za dokumentovanje softvera. Fokus ovog templejta je na stakeholderima i kako oni vide softversko rešenje.

Filip Krušten (*Philip Kruchten*), autor ovog modela, objašnjava da je potreba za ovim modelom nastala zato što je na projektima na kojima je radio dolazio u situacije da dokumentacija bude puna oznaka koje krajnji korisnik ne razume, te se mučio da dođe do informacija koje bi se njega ticale.

Rešenje do kog je došao je napraviti model za svaku ulogu stakeholdera projekta sa detaljima koji se njega konkretno tiču. Ovaj model posmatra rešenje iz 4 ugla kranjih korisnika, project menadžera, programera, devops inženjera, na šta se dodaje 5. pogled koji sadrži neke, ali ne sve slučajeve korišćenja, kako bi objasnio kako su svi ostali pogledi povezani međusobno.^[17]



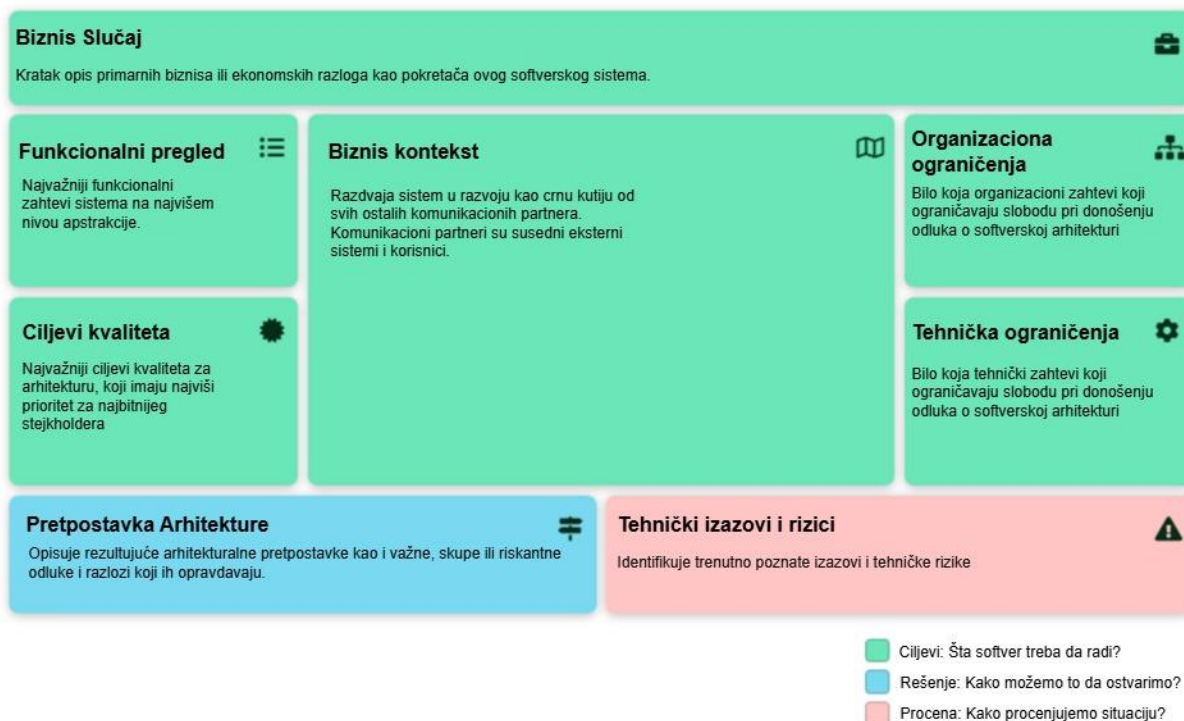
Slika 12: 4+1 View Model, crtana po uzoru na [\[17\]](#)

- Logički pogled predstavlja rešenje iz ugla kranjih korisnika, na veoma visokom nivou apstrakcije opisuje kako je sistem struktuiran da bi pružio funkcionalnosti korisniku. Za prikaz se mogu koristiti UML dijagram klasa i UML dijagram stanja.
- Procesni pogled se fokusira na ponašanje sistema u vreme izvršavanja, predstavlja procese u sistemu i kako oni međusobno komuniciraju. Za prikaz su prikladni UML dijagrami sekvenci, komunikacije i aktivnosti. Takođe adresira nefunkcionalne zahteve.
- Development pogled prikazuje sistem iz ugla programera i bavi se menadžmentom softvera . U tu svrhu, koristi se dijagrami komponenti i/ili package dijagrami.
- Fizički pogled posmatra rešenje na nižem nivou, nivou hardvera, opisuje mapiranje između softvera i hardvera i ekvivalentan je Deployment View-u u arc42 arhitekturi. Ovo je pogled iz ugla devops inženjera.

Scenariji slučaja korišćenja predstavljaju 5. pogled, kako korisnici interaguju sa sistemom. Za tu svrhu koristi UML use case dijagrame.

2.3.3 Software Architecture Inception Canvas

Software Architecture Inception Canvas je deo arc42 arhitekture templejta i predviđa sažet opis softverskog rešenja na jednoj slici nalik prezentacionom slajdu. Slika je podeljena u sekcije, gde svaka od sekcija opisuje jednu od vrsta vodećih motivacionih faktora rešenja. Sekcije su različito oboje i koriste ikonice kako bi dodatno olakšale razumevanje čitave slike. Ovo je inception canvas, što znači da je glavni fokus skica ili suština rešenja, ne treba da sadrži bilo kakve detalje implementacije arhitekture. [\[18\]](#)



Slika 13: Kanvas softverske arhitekture, slika prevedena od [18]

Biznis slučaj: počinje od pitanja zašto baš ovaj projekat? Definisanje razloga iza projekta daje orijentaciju produkcijom timu i odgovor na pitanje čemu će kranji proizvod služiti, kao i kolika je njegova bitnost. Dodatna vrednost je motivacija samog tima jer izvršiocima daje korelaciju između njihovih individualnih doprinosa i ekonomskih ciljeva preduzeća.

Funkcionalni pregled: daje odgovor na pitanje šta će naš softver raditi na najvišem nivou apstrakcije. Koje primarne probleme će ovaj softver rešiti.

Ciljevi kvaliteta: Sadrži 3 do 4 najbitnija atributa kvaliteta. Prilikom biranja ovih ciljeva kvaliteta bitno je izabrati atribute kvaliteta, imajući na umu da će oni biti vodilja svima koji će raditi na projektu (internim stakeholderima), kao i da to treba da budu atributi koji će najbolje zadovoljiti klijente i menadžment (eksterne stakeholdere).

Biznis Kontekst: Tretira rešenje kao crnu kutiju i fokusira se na susedne sisteme i stakeholdere koji zavise od ovog sistema ili od kojih ovaj sistem zavisi. Analiza susednih sistema i/ili stakeholdera može biti korisna u kasnijem otkrivanju tehničkih rizika u vezi tih sistema. Ako je u pitanju alat kao podrška već postojećoj softverskoj arhitekturi, ovo poglavlje treba da da odgovor na to koji deo procesa će ovaj alat automatizovati ili olakšati i sa kojim funkcionalnostima. Može se takođe predstaviti i ukratko samo sa jednim context dijagramom.

Organizaciona ograničenja: Odnosi se na ograničenja poput vremena i novca, takođe zahtevi nametnuti implementacijom tehnika poput npr SCRUM-a, utiču na definisanje jedinica posla.

Tehnička ograničenja: Odnosi se na konkretne tehnologije koje su izabrane za izradu rešenja. Npr u slučaju izrade web aplikacije, software arhitekta može izabrati određenu kombinaciju tehnologija, frameworka, alata i baze podataka koja sa sobom nosi određena ograničenja. Npr zastarele biblioteke i tehnologije često mogu biti ograničavajuć faktor, jer ograničavaju tim u korišćenju novije verzije biblioteke, jer ona nije kompatibilna sa tom zastarelom bibliotekom.

Pretpostavka arhitekture: Odnosi se na pretpostavke o arhitekturi na najvišim nivoima apstrakcije, bez ulaženja u detalje oko dizajna i implementacije. U praksi se često predstavlja skicom glavnih komponenti.

Tehnički izazovi i rizici: Može predstavljati bilo koju vrstu identifikovanog rizika koji ranije nije spomenut. Često ima veze sa nepoznatim mogućnostima tehnologija ili celokupnog biznis procesa koje rešenje treba da nadomesti u vreme pisanja canvasa. Tu takođe spadaju i ograničena mogućnost testiranja, mogućnost da aplikacija ne bude dobro prihvaćena od krajnjih korisnika iz nekih razloga itd.

2.4 Detaljno Projektovanje

U ovom poglavlju će biti reči o projektovanje softvera u tradicionalnom smislu (iz ugla objektno orjentisane paradigme).

2.4.1 Opis zahteva pomoću modela slučajeva korišćenja

Nakon identifikovanja zahteva klijenta kroz intervju i definisanja verbalnog opisa tih zahteva, izvlače se slučajevi korišćenja i predstavljaju na UML Use Case Diagramu.

Čovečuljci se koriste da označe različite uloge korisnika u sistemu. Ključne reči include i extend su objašnjene u poglavlju [2.1.1 Funkcionalni zahtevi](#). Svaki od oblačića predstavlja jedan slučaj korišćenja.

Sledeći korak je definisanje user storija i sprintova korišćenjem SCRUM Agilne metoda.

Jedan ili više slučajeva korišćenja usecase diagrama se mapiraju na jedan userstory.

User story su već ranije objašnjeni u poglavlju [2.1.1 Funkcionalni zahtevi](#).

Zahtevi se po potrebi ako je to potrebno, mogu opisati i veoma detaljno kroz interakcije sa sistemom i očekivanog odgovora u slučaju pozitivnog ishoda (glavni scenario) i u slučaju greške(alternativni scenario).

2.4.2 Opis ponašanja sistema korišćenjem sequence dijagrama

Nakon detaljnog definisanja slučajeva korišćenja, analiza scenarija podrazumeva transformaciju tekstualnih opisa u UML dijagrame sekvenci.

Uzimaju se samo delovi slučaja koji se odnose na pozivanje sistemskih operacija (APSO) i povratne argumente tih operacija (IA).

3. Studijki primer - projektovanje rešenja

U ovom poglavlju je predstavljen primer projektovanja rešenja na temu "Razvoj web aplikacije za studentsku službu". Kroz ovo poglavlje biće demonstrirana primena prethodno diskutovanih metodologija. Poglavlja će pratiti prvenstveno strukturu arc42 templejta, ali će u uvodu biti definisan software architecture inception canvas kao overview celog rešenja.

4+1 View Model nije eksplicitno korišćen, ali se može povući paralela između poglavlja.

- 3.3 Kontekst i opseg resenja -> Logical View
- 3.5 Pregled Module (Building Block View) -> Development View
- 3.6 Pregled Procesa (Runtime View) -> Process View
- 3.7 Deployment View -> Physical View
- U okviru poglavlja o detaljnom projektovanju
 - 3.12.1 Analiza zahteva i projektovanje interfejsa -> Scenario View

3.1 Uvod i ciljevi

Na slici 18 je prikazan Architecture Inception Canvas koji na jednoj slici u kratkim crtama skicira rešenje i u kratkim crtama daje uvid u funkcionalne i nefunkcionalne zahteve, organizaciona i tehnička ograničenja kao i glavne komponente softverskog sistema.

Biznis rešenje

Trenutno rešenje student, ne zadovoljava sve potrebe biznisa i FON kao jedan od vodećih fakulteta po praćenju trendova u IT-u, želi da demonstrira svojim studentima praktičnu implementaciju modernih tehnologija. Studentska služba je zamišljena kao prototip moderne web aplikacija za prijavu ispita i menadžment istih.

Funkcionalni pregled

Sofversko rešenje treba da omogući sledeće osnovne funkcionalnosti:

- +personalizovan nalog za svakog studenta i profesora
- +pregled svih ispita u aktuelnom ispitnom roku
- +prijavu ispita studentima
- +uvid u trenutno prijave studentima
- +opciono odustajanje od polaganja
- +uvid u rezultate polaganja studentima
- +zakazivanje rokova i polagaja ispita profesorima
- +ocenjivanje polaganja profesorima

Ciljevi kvaliteta

Definisani su sledeći ciljevi kvaliteta kojima će aplikacija težiti analizirajući željene kvaliteta iz 4 ugla: Korisnik, Programer, DevOps i Menadžment:

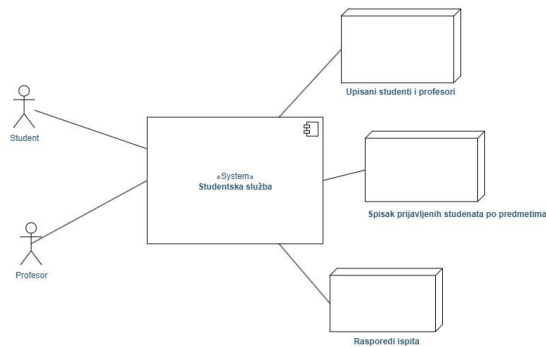
Korisnik: korisna, laka za razumeti i prijatna za korišćenje, pouzdana, sigurna.

Programer: fleksibilna u smislu održavanja koda i dodavanja novih featura, predvidjiva, interoperabilnost sa budućim rešenjima

DevOps: konfigurabilna, laka za postavljanje u produkciono okruženje, efikasna u korišćenju resursa.

Menadžment: efikasna u smislu cene razvoja, održavanja i vremena razvoja, pouzdana.

Biznis kontekst



Organizaciona ograničenja

Projekat treba da sadrži do 10 slučajeva korišćenja zbog ograničenog vremena koje student ima na raspolaganju i treba da služi više kao demonstracija rešenja i vežba, nego realno rešenje. Zbog gore navedenog ograničenja, dosta detalja poput editora rokova, editora ispita i prijavljivanje studenata na slušanje ispita biće izostavljeni kao funkcionalnosti.

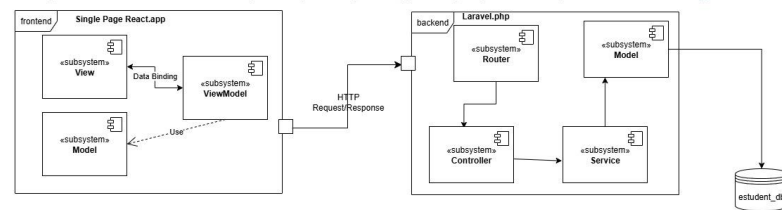
Poželjno je da se projekat razvija po SCRUM metodologiji

Tehnička ograničenja

Windows operativni sistem
React.js kao frontend framework
Laravel.php kao backend framework
SQL Server kao dbms

Pretpostavka arhitekture

Za dizajn arhitekture korisničke strane korišćen je MVVM pattern za podelu odgovornosti, dok je na serverskoj strani korišćen MVC i Hexagon



Tehnički izazovi i rizici

Nedostatak iskustva programera, ovo mu je prvi pravi projekat u Reactu i Laravelu. Teža procena potrebnog truda i vremena sa obzirom na veliki broj nepoznatih faktora u implementaciji.

Potencijalni problemi sa reaktivnošću, testiranjem, mapiranjem entiteta korišćenjem Laravel Eloquent ORM-a migracionim skriptama u Laravelu

Slika 14: Architecture Inception Canvas za projekat Studentska služba

3.1.1 Pregled zahteva

Funkcionalni zahtevi (verbalni opis)

Potrebno je napraviti softversko rešenje za menadžment ispita za Fakultet Organizacionih Nauka. Rešenje treba da omogući sledeće funkcionalnosti zaposlenima fakulteta i studentima: prijavljivanje i odjavljivanje na persolni nalog i uvid u trenutno aktivne ispitne rokove. Samo studentima treba omogućiti prijavu i odjavu ispita dok period prijave/odjave traje kao i uvid u uspešna polaganja ukupno ostvaren broj espb i prosečnu ocenu. Zaposlenima treba omogućiti ocenjivanje prijave na ispitima gde su prijavljeni kao ocenjivači, takodje potrebno je obezbediti mogućnost generisanja izveštaja posle održavanja ispita radi uvida u izlazno, prosečnu ocenu i prolaznost, takodje potrebno je omogućiti generisanje izveštaje sa istim metrikama ali za celu godine po rokovima.

Nefunkcionalni zahtevi (verbalni opis)

Prilikom dizajna obratiti pažnju na iduće attribute kvaliteta. Dizajn korisničkog interfejsa treba da bude prijatan oku i intuitivan za korišćenje korisnicima, vodjena principima material dizajna. Aplikacija treba da bude pouzdana, brza i sigurna od mogućih zloupotrebljavanja.

Business Case (sa Architecture Inception Canvasa)

Trenutno rešenje estudent, ne zadovoljava sve potrebe biznisa i FON kao jedan od vodećih fakulteta po praćenju trendova u IT-u, želi da demonstrira svojim studentima praktičnu implementaciju modernih tehnologija. Studentska služba je zamišljena kao prototip moderne web aplikacija za prijavu ispita i menadžment istih.

Functional Overview (sa Architecture Inception Canvasa)

Softversko rešenje treba da omogući sledeće osnovne funkcionalnosti:

- personalizovan nalog za svakog studenta i profesora
- pregled svih ispita u aktuelnom ispitnom roku
- prijavu ispita studentima
- uvid u trenutno prijave studentima
- opciono odustajanje od polaganja
- uvid u rezultate polaganja studentima
- zakazivanje rokova i polagaja ispita profesorima
- ocenjivanje polaganja profesorima

3.1.2 Ciljevi kvaliteta i Stejkholderi

Iz sekcije Quality Goals architecture inception canvasa. Definisani su sledeći ciljevi kvaliteta kojima će aplikacija težiti analizirajući željene kvaliteta iz 4 ugla: Korisnik, Programera, DevOps i Menadžement:

- Korisnik(Student): korisna, laka za razumeti i prijatna za korišćenje, pouzdana, sigurna.
- Programer: fleksibilna u smislu održavanja koda i dodavanja novih featura, predvidjiva, interoperabilnost sa budućim rešenjima
- DevOps: konfigurabilna, laka za postavljanje u produkciono okruženje, efikasna u korišćenju resursa.

- Menadžment: efikasna u smislu cene razvoja, održavanja i vremena razvoja, pouzdana.

3.2. Ograničenja

3.2.1 Tehnička ograničenja

Tabela 4: Tehnička ograničenja

Ograničenje	Razlog i /ili motivacija
Softver predviđen za rad na Windows operativnim sistemima	Zbog velike distribuiranosti ovog sistema i toga što je predviđeno rešenje u formi web aplikacije, iako je implementacija u Linuxu i MacOS poželjna u ovom slučaju nije obavezna
React.js kao framework na klijentskoj strani	Izabran zbog popularnosti ovog js frameworka u trenutku pisanja i kao vežba programeru radi usavršavanja znanja u ovom okruženju
Bootstrap 5	Standardna biblioteka sa predefinisanim css klasama za stilizovanje komponenti
Laravel.php	Stabilan i popularan framework. Postojala je opcija da se rešenje piše i u Java SpringBoot frameworku, ali je Laravel.php izabran radi usavršavanja i vežbe prethodno stečenih znanja iz php-a na fakultetu

3.2.2 Organizaciona ograničenja

Tabela 5: Organizaciona ograničenja

Ograničenje	Razlog i /ili motivacija
Projekat treba da sadrži ne više od 10 slučajeva korišćenja	Zbog ograničenog vremena developera i cilja rešenja, tj. demonstracija potencijalnog realnog rešenja
SCRUM metodologija	Podela posla na manje celine, user storije olakšava konzistentan rad u 2 nedeljnim sprintovima
Jedan neiskusni developer	Vladimir Jecić, ovo mu je prvi projekat u Laravelu i Reactu, teža procena potrebnog truda, potencijalno kašnjenje taskova zbog potrebe da se douče određeni koncepti Reacta i Laravela

3.2.3 Konvencije

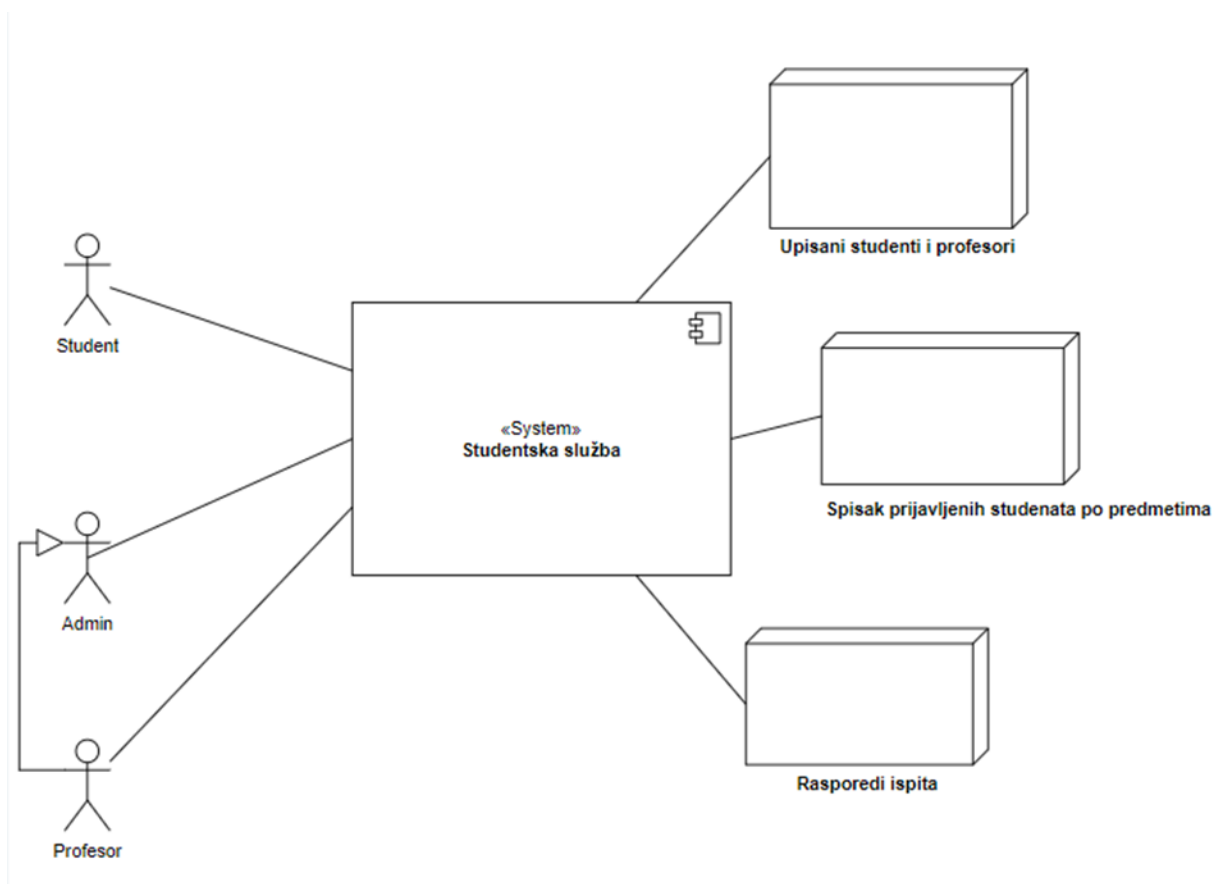
Tabela 6: Konvencije

Ograničenje	Razlog i /ili motivacija
MVVM arhitektura	za podelu odgovornosti komponenti na klijentskoj strani

Hexagon arhitektura	za podelu odgovornosti komponenti na serverskoj strani
REST api stil projektovanja	Popularna konvencija, pogodna za dokumentovanje i interoperabilnost api
Kamel notacija imenovanja promenljivih	prihvaćena u php i javascript jeziku
Primarni jezik korišćen za kodiranje (nazivi promenljivih, klasa itd.) - engleski	prostiji jezik za imenovanja, preferenca

3.3. Kontekst i opseg rešenja

3.3.1 Business Context



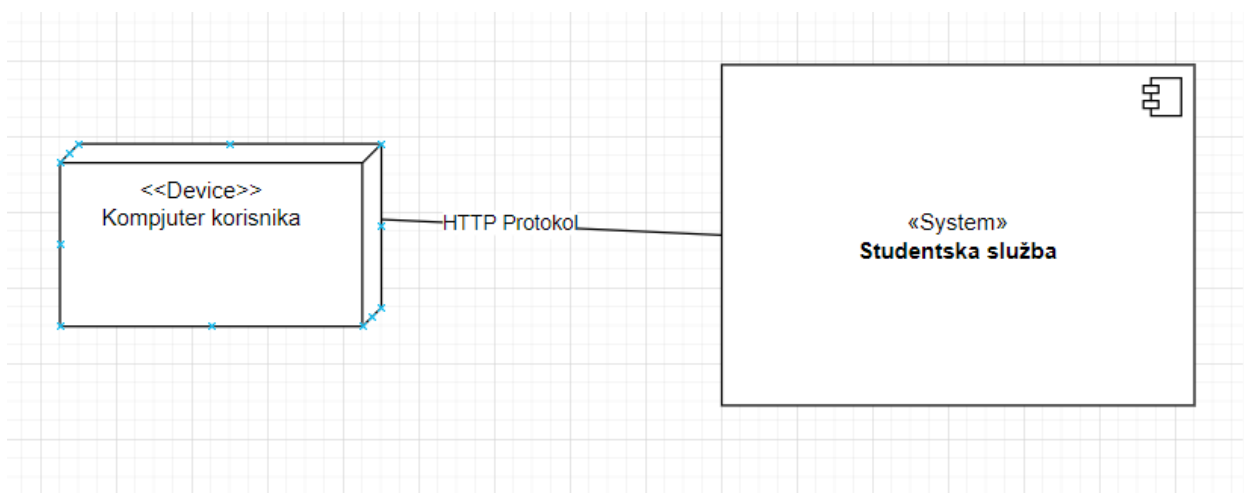
Slika 15: Business Context Studentske službe

Raspored Ispita, Spisak prijavljenih studenata po predmetima, upisani studenti i profesori eksterni sistemi ili baze znanja

Korisnici:

- Student - ima ograničene mogućnosti koje se njega tiču, student može da pregleda prethodna polaganja, aktuelne rokove, prijavi polaganje ili odjavi polaganje ispita.
- Admin – ima prava da objavljuje nove rokove, dodaje studente na određene predmete,
- Profesor - istovremeno ima i rolu admina i takođe ima prava da oceni polaganja, pregleda izveštaje ispita (izlaznost, prolaznost, prosečna ocena) ili statistiku predmeta tokom cele školske godine itd.

3.3.2 Deployment Context



Slika 16: Deployment Context

3.4. Strategija rešenja

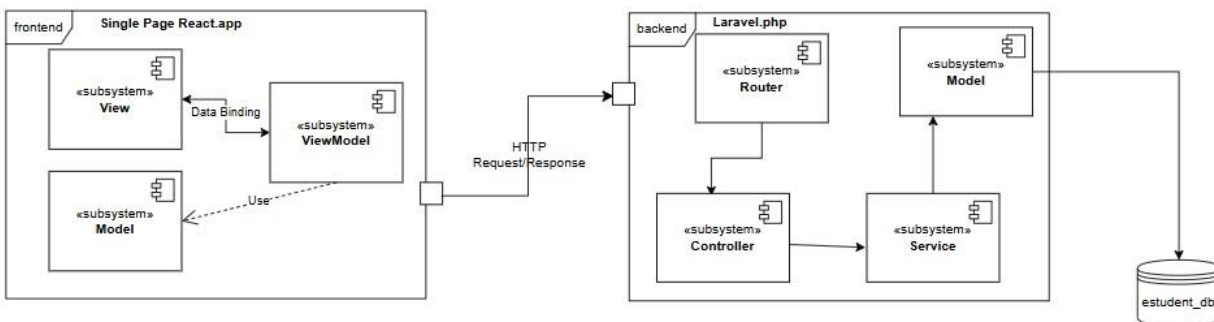
3.4.1 Nefunkcionalni zahtevi

Tabela 7: Ciljevi kvaliteta u strategiji rešenja

Quality goal	Scenario	Solution approach
Laka za razumeti, prijatna za korišćenje Laka navigacija	Student želi da pregleda aktuelne rokove	Single Page aplikacija, Korišćenje adekvantne font-awesome ikone i jasnog layouta, minimalistički dizajn
fleksibilna u smislu održavanja koda i dodavanja novih featura	Programer zeli da doda novi feature za profesore, retroaktivno menjanje unetih detalja polaganja	REST api, MVC i Hexagon pattern na serverskoj strani, jasna organizacija foldera i open-closed principle, migracione skripte omogućavaju intuitivno proširenje rešenja sa novim modelima i slučajevima korišćenja u budućnosti
Pouzdana, sigurna, kontrolisan pristup u zavisnosti od uloge	Korisnik želi da njegovi podaci budu zaštićeni lozinkom, dok biznis takođe želi i da ograniči pristup određenim rutama na serverskoj strani(npr da student ne bi sam sebi upisivao ocene)	Login strana sa enkriptovanim passwordom za autentikaciju i OAuth 2.0 za autorizaciju

Testabilna	Tokom razvoja developer želi da testira prijavljivanje određenog ispita u određenom roku od određenog studenta	Factory u Laravelu(za definisanje fake podataka) i Seeder za generisanje konkretnih ispita npr seed 20 random ispita, kao i unit testovi za simulaciju slučaja korišćenja
------------	--	---

3.4.2 Principi i Paterni



Slika 17: Arhitektura na najvišem nivou apstrakcije

Arhitektura predviđena strategijom na slici 14 predviđa raslojavanje aplikacije klijentske strane na 3 sloja. Korisnik unosi podatke kroz View i okida funkcije u ViewModel-u. Funkcija u ViewModelu vrši validaciju na klijentskoj strani i kreira zahtev i prosleđuje ga HTTP protokolom serveru na endpoint u ruteru definisan po REST api stilu. Ruter dalje poziva odgovarajuću metodu u Controller-u. Controller ima ulogu stateless servisa koji vrši određene validacije i izvršava jednu od ili kombinaciju CRUD operacija nad Modelom. Model u Laravel arhitekturi nije ekvivalent POJO objektu u javi, već nasleđuje Model klasu u php-u koja u sebi ima predefinisane ORM metode koje rade direktno sa bazom. U suštini pozivanjem metode Modela, Laravel Eloquent ORM-om(Object Relationship Mappingom), promene u modelu se perzistuju nad odgovarajućom relacijom u bazi podataka. Model na klijentskoj strani je klasa koja služi kao posrednica u mapiranju DTO(Data Transfer Objekata) u JSON formatu, tj. Resursa po Laravel nomenklaturi u odgovarajući objekat pogodan za prikaz u ViewModelu. Nakon uspešne operacije ViewModel zove injected metodu iz View-a koja korišćenjem useState() react hooka osvežava trenutni prikaz u View sa projekcijom trenutnog stanja u ViewModelu.

3.5 Pregled Modula(Building Block View)

3.5.1 Relacioni Model

User(user_id,indexNum,name,email,role,password)

Course(course_id,name,semester,espb)

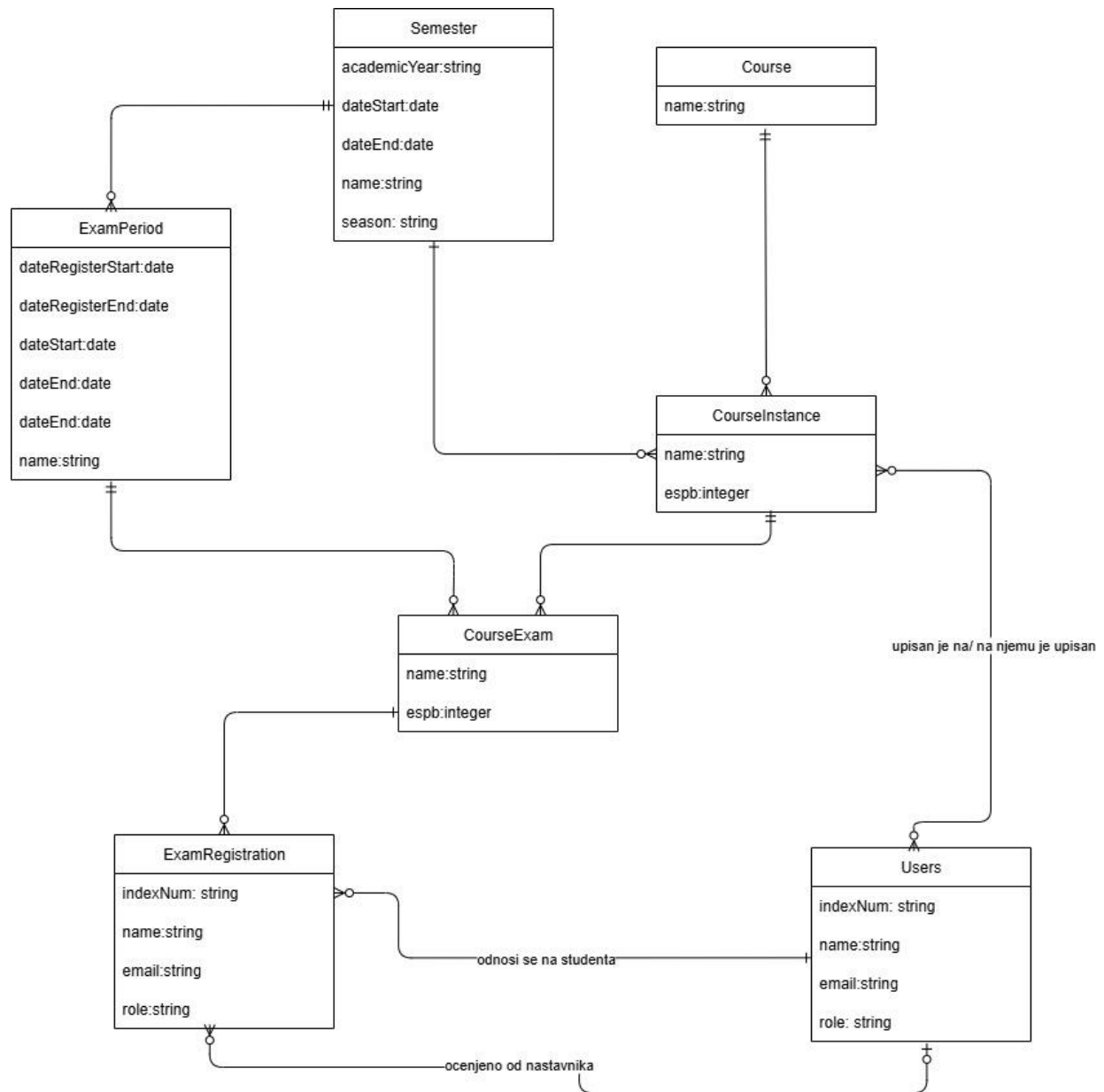
CourseUser(user_id,course_id)

ExamPeriod(exam_period_id,name,dateRegisterStart,dateRegisterEnd,dateStart,dateEnd)

CourseExam(course_id,exam_period_id,examDateTime,hall)

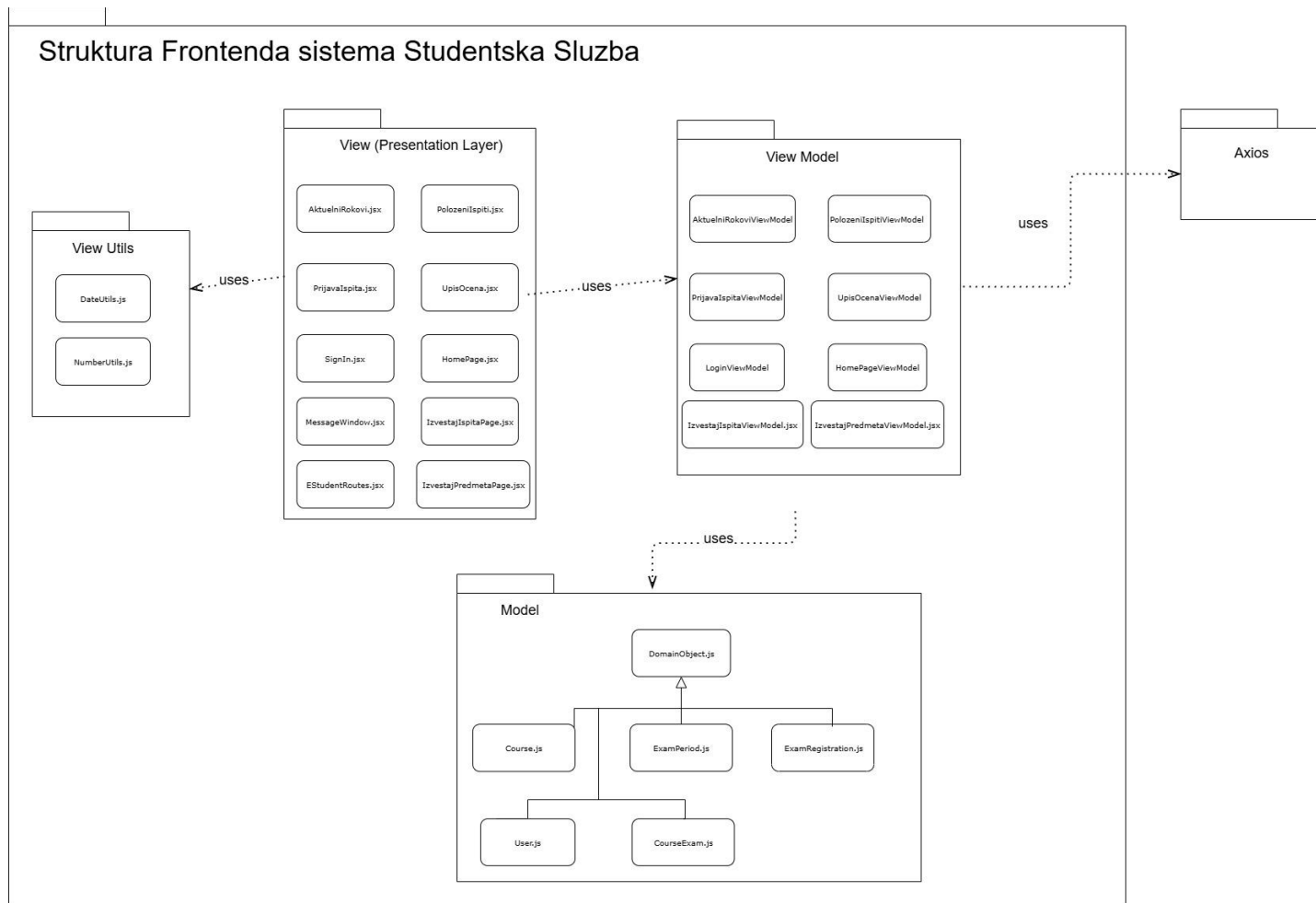
ExamRegistration(course_id,exam_period_id,student_id, mark, comment, signed_by_id)

3.5.2 Konceptualni Model



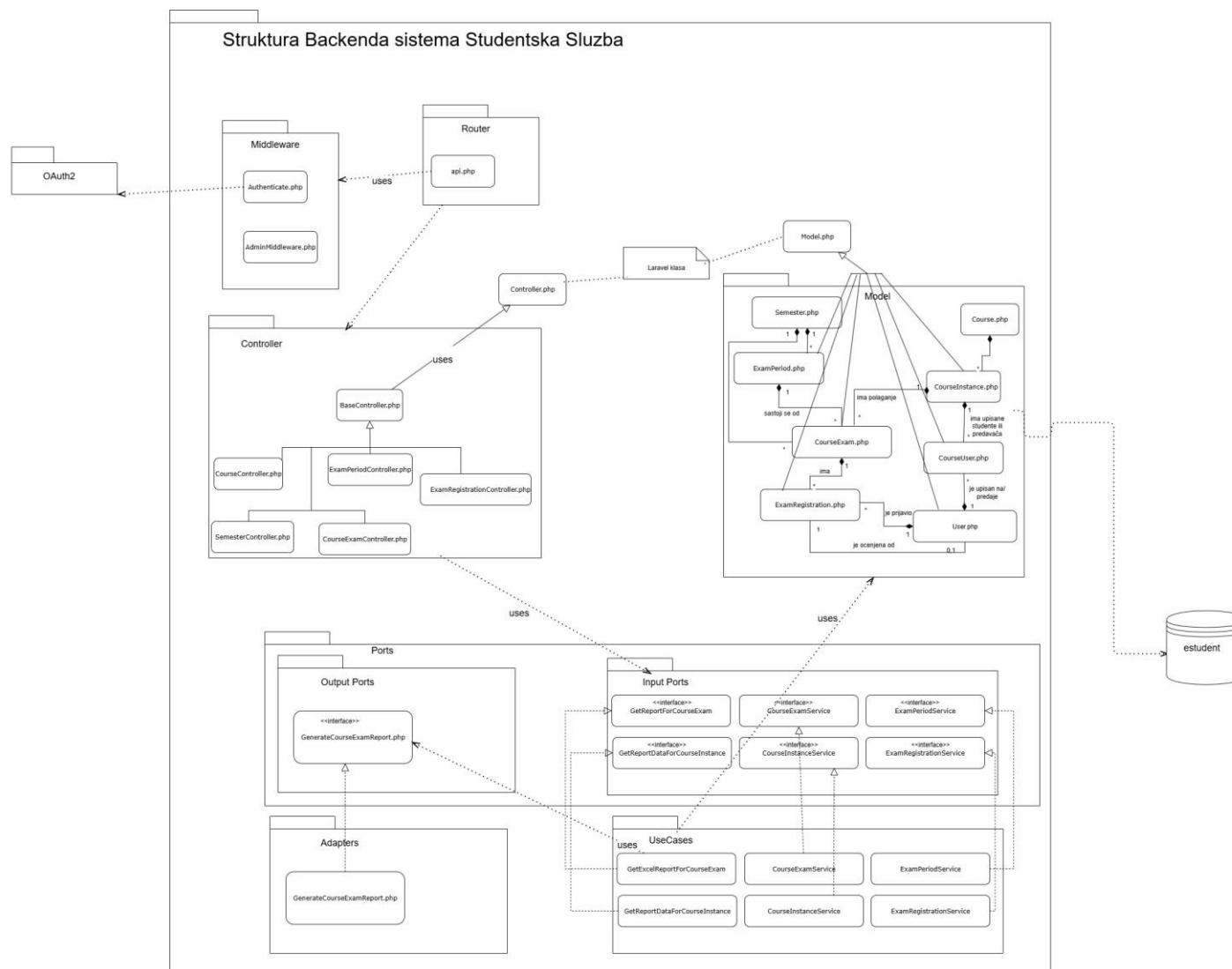
Slika 18: Konceptualni model (ER Diagram – Crow's Foot Notacija)

3.5.3 Pregled modula na klijentskoj strani



Slika 19: Struktura sistema studentska služba sa klijentske strane

3.5.4 Pregled modula na serverskoj strani



Slika 20: Struktura sistema studentska služba na serverskoj strani

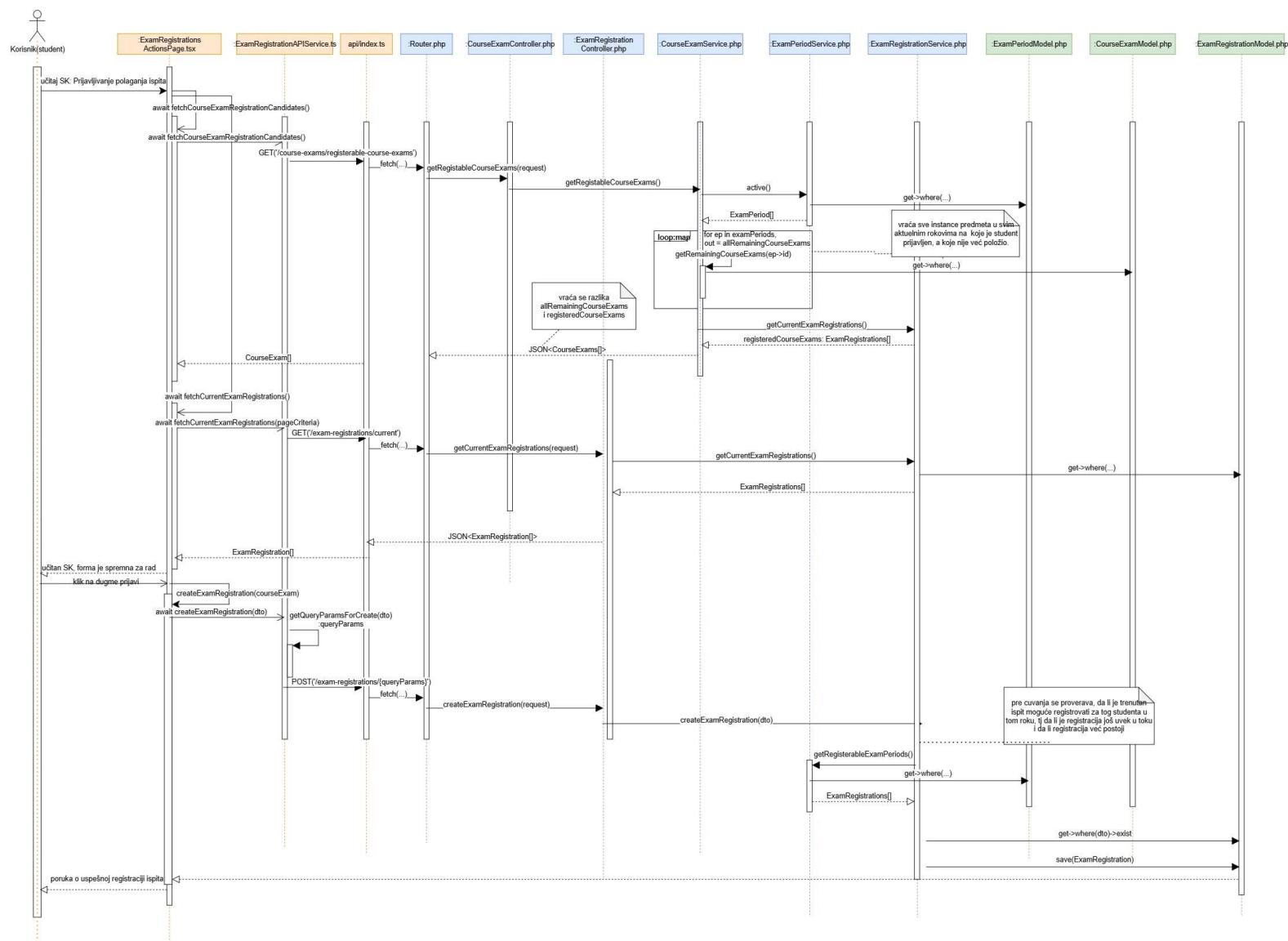
3.6 Pregled Procesa (Runtime View)

Zbog velikom broja interakcija između modela na sledećem dijagramu će biti ilustrovane najvažnije interakcije komponenti [SK5: Prijavljivanje Ispita](#), za pregled ostalih identifikovanih operacija, videti poglavlje [3.12 Detaljno projektovanje](#).

Interakcije na dijagramu sekvenci na slici ispod se mogu podeliti u 3 celine: učitavanje neprijavljenih ispita u aktuelnom roku kao ispita koja student može da prijavi ako je registracija u toku, učitavanje već registrovanih ispita i sam proces registracije studenta na ispit.

Ovaj slučaj je izabran kao primer zbog složenije biznis logike na serverskoj strani. Ispiti koje student još uvek nije prijavio se pronalaze tako što se prvo pronadju svi aktivni ispitni rokovi pozivom metode `active`, klase `ExamPeriodService`. Ispitni rok je aktivan ako je današnji dan posle početka registracije, a pre kraja održavanja ispita, tj. ispitnog roka. Zatim se kroz iteraciju za svaki od rokova zove metod `getRemainingCourseExams`. Metod `getRemainingCourseExams` za prosledjeni rok učitava prvo sve ispite u tom roku i od njih oduzima one za koje već postoji prijava sa ocenom većom od 5 za tog studenta. Kao rezultat ove operacije se dobije lista ispita koje student sigurno nije položio, na dijagramu `allRemainingCourseExams`. Međutim od ovog skupa treba oduzeti ispite koje je student već prijavio kako ne bi prikazivali ispit ispit i u neprijavljenim i prijavljenim. Tako da je konačna lista koja se vraća korisničkoj strani aplikacije zapravo razlika skupova `allRemainingCourseExams` i liste `registeredCourseExams`.

Nakon što je forma spremna, student inicira prijavljivanje ispita, gde se za svaki slučaj proverava da li je trenutno autentikovao korisnik isti korisnik koji prijavljuje ispit, jer student može da prijavi ispit samo za sebe, kao i da li je registracija još uvek u toku i da li već postoji. Posle uspešne prijave, u osnovnom scenariju korisniku se kao potvrda o uspešnoj registraciji ispita, prikazuje poruka.

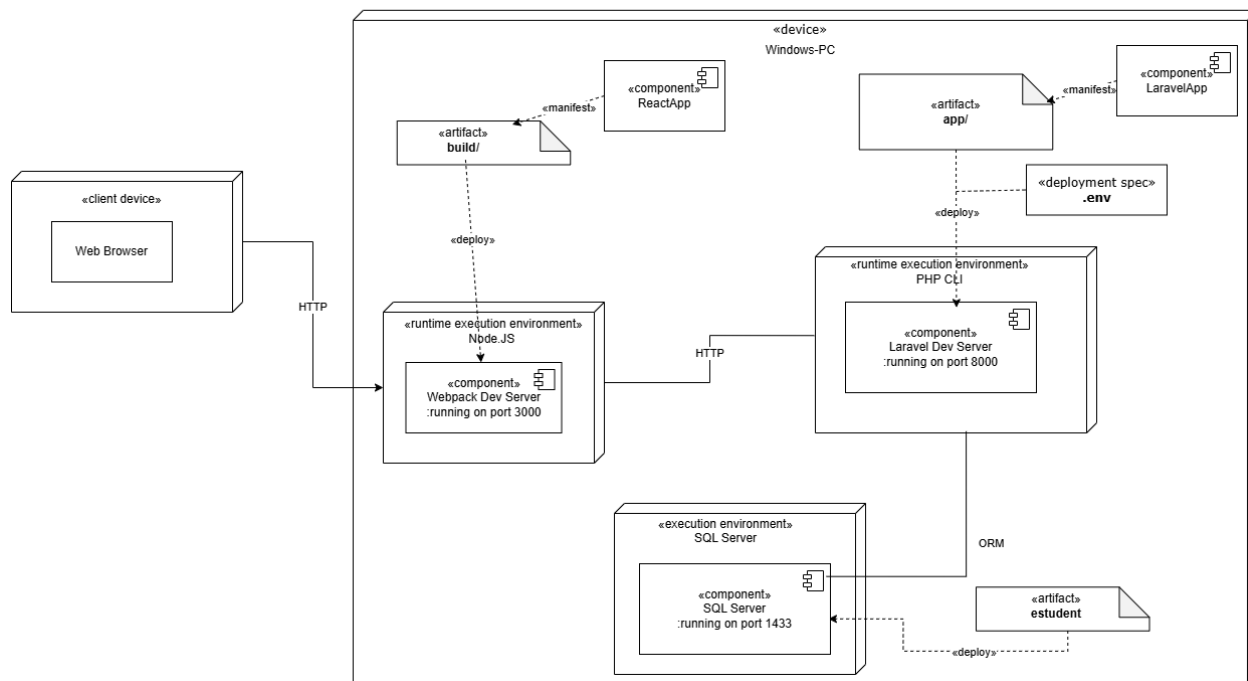


Slika 21: Detaljni dijagram sekvenci kao primer interakcije komponenti prilikom slučaja korišćenja prijave ispita

3.7 Pregled Raspoređivanja (Deployment View)

Softver nema produkciono okruženje, već samo razvojno i ono je predstavljeno na slici ispod. Kao development server za React je korišćen Webpack Dev Server u izvršnom okruženju Node.js, dok je za servesku stranu korišćen PHP build-in server u izvršnom okruženju PHP CLI.

Webpack procesira sav javascript u bundle.js u folderu build/ i taj folder je potrebno deployovati, dok se u slučaju laravel projekta ceo app/ folder deploy-uje. Na ovom primeru su i aplikacija klijentske i serverske strane i baza na istoj mašini, međutim u praksi bi ovo rešenje moglo da se rasporedi na 3 različite mašine.

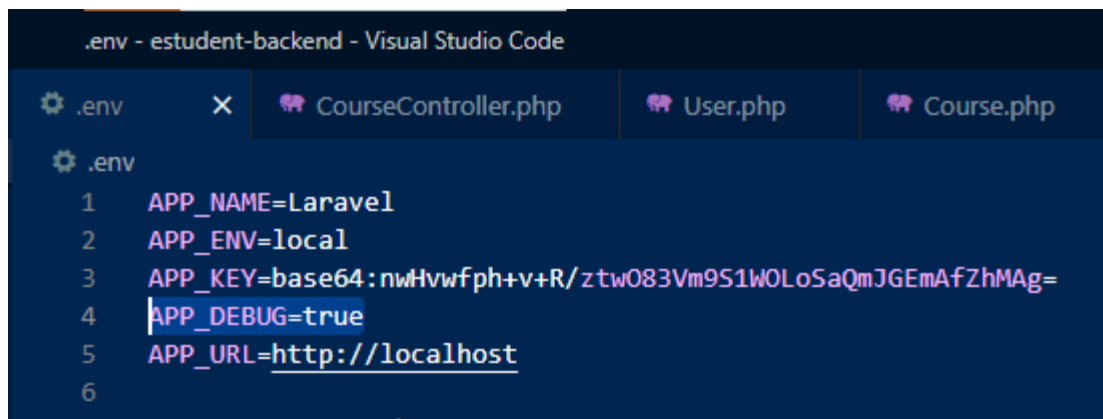


Slika 22: Deployment Diagram

3.8 Crosscutting Koncepti

3.8.1 Logovanje grešaka

Logovanje grešaka na serveskoj strani je predviđeno samim Laravel frameworkom. Omogućava se postavljanjem variable APP_DEBUG=true u konfiguracionom .env fajlu i u tada se logovi čuvaju u fajlu storage/logs/laravel.log.



Slika 23: Omogućavanje opcije logovanja grešaka

3.8.2 Handlovanje grešaka

Kada greške nastale na serverskoj strani, rešenje predviđa postavljanje poruke koja će biti vraćena klijentu, kao i HTTP kod. Na primeru ispod se vidi postavljanje greške u slučaju pokušaja registracije za taj ispit i studenta koja već postoji, u tom slučaju se vraća poruka i HTTP greška 409 – Conflict.

```

1 reference | 0 overrides
2 public function createExamRegistration(SubmitExamRegistrationDTO $dto): ExamRegistration
3 {
4     $courseExam = CourseExam::with(relations: 'examPeriod')->find(id: $dto->courseExamId);
5     $registerableExamPeriods = $this->examPeriodService->registerable();
6     if (!$registerableExamPeriods->contains(key: $courseExam->examPeriod)) {
7         throw new RegistrationNotInProgressException(message: 'Exam registration is not in progress for this exam period.');
```

Slika 24: Primer postavljanje greške prilikom pokušaja registracije korisnika sa indexom koji već postoji

```

EStudentException.php
1 <?php
2
3 namespace App\estudent\domain\exceptions;
4
5 use Exception;
6
7 11 references | 5 implementations
8 abstract class EStudentException extends Exception
9 {
10     6 references
11     protected int $statusCode = 500;
12
13     1 reference | 0 overrides
14     public function getStatusCode(): int
15     {
16         return $this->statusCode;
17     }
18 }

RegistrationAlreadyExistsException.php
1 <?php
2
3 namespace App\estudent\domain\exceptions;
4
5 2 references | 0 implementations
6 class RegistrationAlreadyExistsException extends EStudentException
7 {
8     1 reference
9     protected $message = 'Registration already exists';
10    1 reference
11    protected int $statusCode = 409;
12
13    5 references | 0 overrides
14    public function __construct(string $message = null)
15    {
16        parent::__construct(message: $message ?? $this->message, code: $this->statusCode);
17    }
18 }

```

Slika 25: Implementacija Custom Exception klase RegistrationAlreadyExistsException

```

POST http://localhost:3000/estudent/api/exam-registrations?course-exam-i... 409 (Conflict)

[NEW] Explain Console errors by using Copilot in Edge: click to explain an error. Learn more

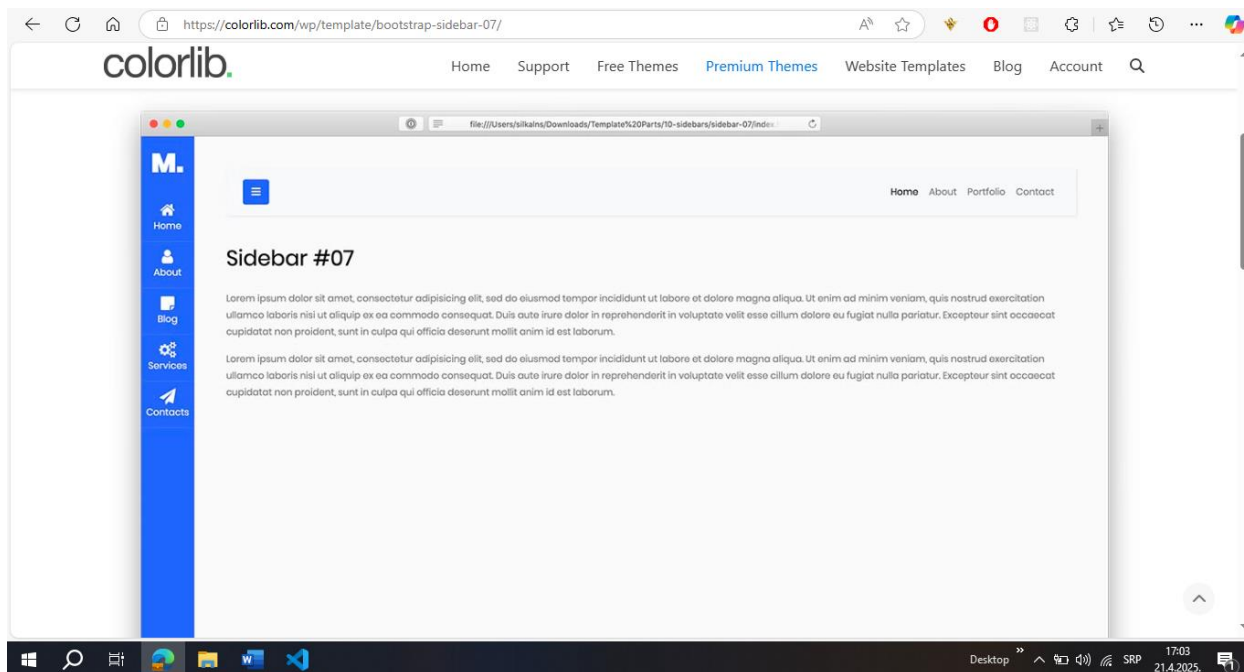
EStudentError: Exam registration already exists for this student and course_exam.
    at handleResponse (index.ts:146:13)
    at async Object.POST (index.ts:72:12)
    at async ExamRegistrationAPIService.createExamRegistration (examRegistrations.ts:96:5)
    at async saveExamRegistration (ExamRegistrationsActionsPage.tsx:85:7)

```

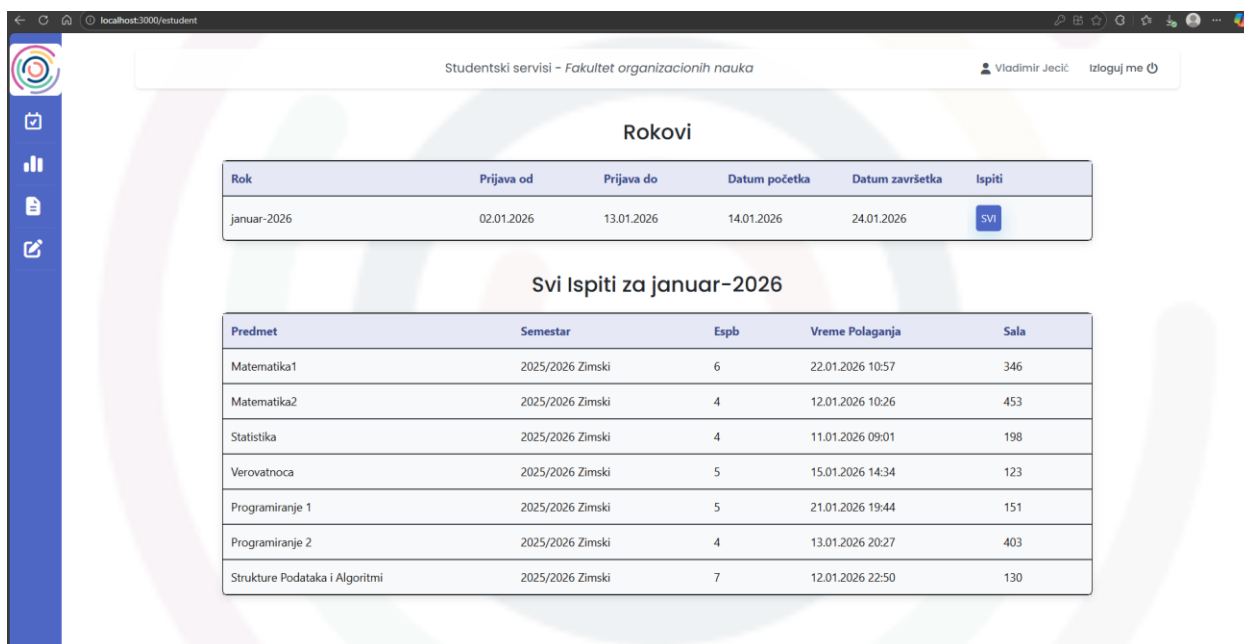
Slika 26: Primer prikaza greške prilikom pokušaja registracije na ispit na koji je student već registrovan

3.9 Odluke o dizajnu

Za dizajn izgleda GUI-a korišćena je tema sa sajta Colorlib.com kao uzor, sa dosta kustomizacije korišćenjem custom css i bootstrapa.



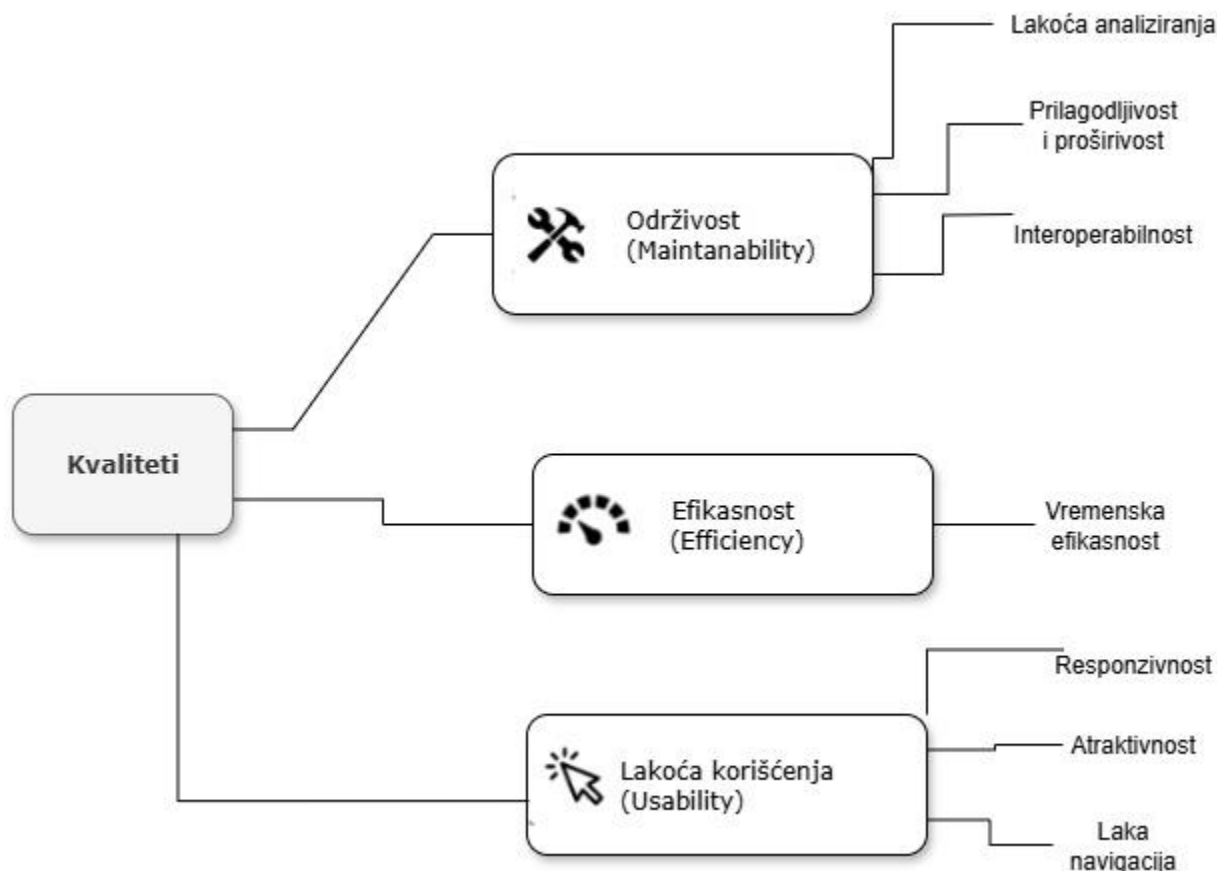
Slika 27: Colorlib.com tema Sidebar #07 korišćena za dizajn GUI-a



Slika 28: Finalni izgled website-a sa primenjenom temom

Klijentska strana aplikacije je osmišljena kao jednostrana (single page) aplikacija. Ovaj pristup omogućava bolje korisničko iskustvo kroz bržu i lakšu navigaciju, jer se samo deo sadržaja strane menja u virtualnom DOM-u dok korisnik u suštini ostaje na istoj HTML strani.

3.10 Zahtevi kvaliteta



Slika 29: Zahtevi kvaliteta, slika radjena po uzoru na sliku iz istoimenog poglavlja u DokChess dokumentaciji [\[5\]](#)

- Lakoća analiziranja – Odnosi se na kvalitet lakoće analiziranja namene i načina funkcionisanja određenih celina projekta. Moduli su logički organizovan po folderima u skladu sa njihovim odgovornostima, takođe prilikom pisanja koda, posebna pažnja je obraćana na doslednost prilikom davanja naziva promenljivih i ruta. Ovome doprinosi i ova dokumentacija u skladu sa arc42 templejtom.
- Prilagodljivost i proširivost - zbog razdvajanja biznis logike od logike apija, moguće je prilagoditi resurse na serverskoj strani ili dodati nove endpointe bez menjanja postojećeg modela.
- Interoperabilnost –REST API endpointi omogućavaju integraciju sa eksternim sistemima nezavisno od platforme.
- Vremenska efikasnost – kratko vreme potrebno za izvršavanje bilo koje operacije, npr za učitavanje aktuelnih rokova i svih ispita studenta potrebno je samo 2-3 sekunde.
- Responzivnost – omogućen je jasan pregled websajta i na mobilnim uređajima
- Atraktivnost – moderan dizajn, intenzivne boje i samosakrivajući side-bar
- Laka navigacija – u navigacionom baru se nalaze prečice do operacija, te u svega 3 klika korisnik može prijaviti ispit koji želi

3.11 Rizici i tehnološki dug

Scenario 1: Testiranje je rađeno samo sa jednim korisnikom i malom količinom podataka (do 10 ispita, prijava i studenata), u slučaju produkcione baze, učitavanje bi moglo znatno da uspori.

Potencijalno rešenje: Primeniti paginaciju prilikom učitavanja resursa, npr ako bude postojao scenario gde profesor prijavljuje studente na određeni kurs, učitalo bi se samo prvih 10 studenata i omogućio full text search po delu imena ili prezimena.

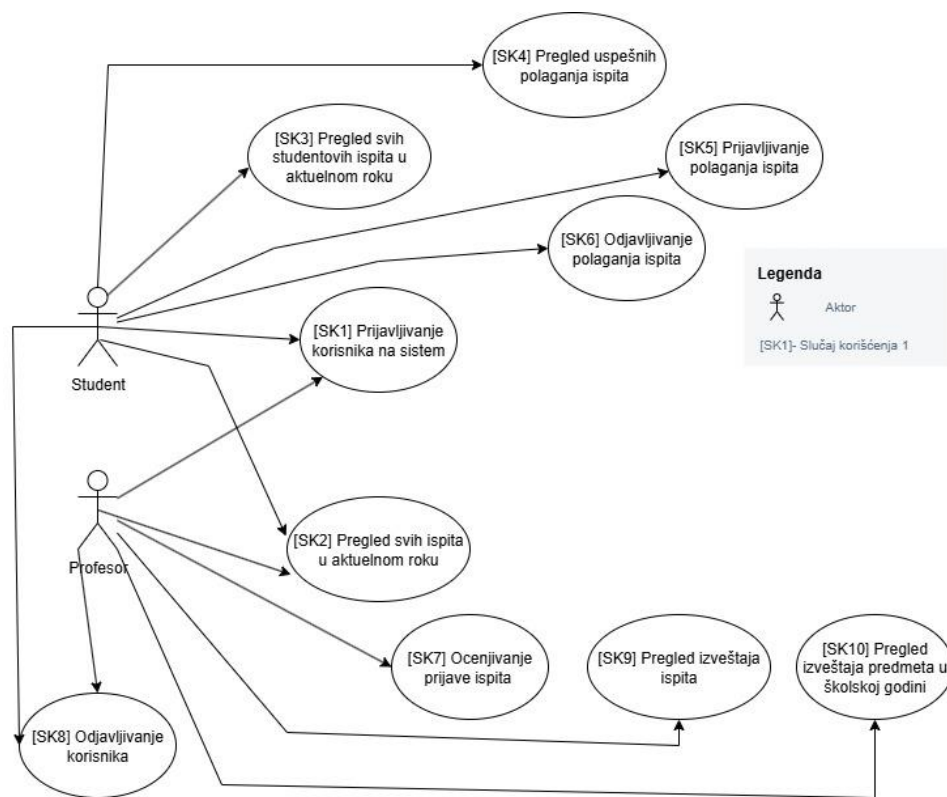
Scenario 2: Moguće je da dođe do nestanka struje ili do otkaza serverske mašine ili operativnog sistema.

Potencijalno rešenje: Rizik se može ublažiti redovnim backupovanjem baze.

3.12 Detaljno projektovanje

3.12.1 Analiza zahteva i projektovanje interfejsa

1. Prijavljivanje korisnika
2. Pregled svih ispita u aktuelnom ispitnom roku
3. Pregled studentovih ispita u aktuelnom ispitnom roku
4. Pregled uspešnih polaganja ispita
5. Prijavaljivanje polaganja ispita
6. Odjavljivanje polaganja ispita
7. Ocenjivanje prijave studenta
8. Odjavljivanje korisnika
9. Pregled izveštaja polaganja predmeta u ispitnom roku
10. Pregled izveštaja predmetu u školskoj godini



Slika 30: Dijagram slučaja korišćenja

3.12.1.1. Slučaj korišćenja 1 – Prijavljivanje korisnika

Naziv SK

Prijavljivanje *korisnika*

Akteri SK

Korisnik

Učesnici SK: *Korisnik* i *sistem* (program)

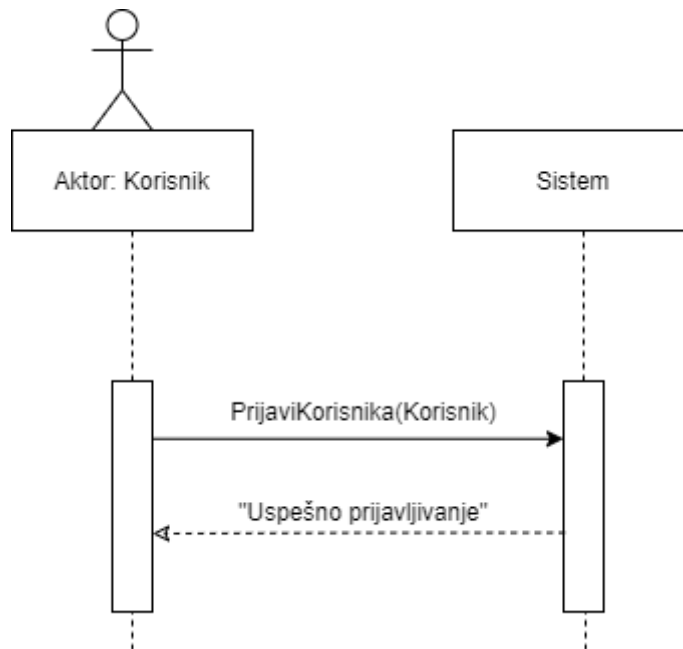
Preduslov: *Sistem* je uključen i prikazuje formu za prijavljivanje *korisnika*.

Osnovni scenario SK:

1. *Korisnik* unosi podatke za identifikaciju *korisnika*. (APUSO)
2. *Korisnik* poziva *sistem* da pronađe *korisnika* sa zadatim podacima. (APSO)
3. *Sistem* traži *korisnika* po zadatoj vrednosti. (SO)
4. *Sistem* prikazuje *korisniku* poruku "Uspešno prijavljivanje". (IA)

Alternativni scenario:

- 4.1. Ukoliko *sistem* nije mogao da pronađe *korisnika*, on prikazuje *korisniku* poruku: "Neuspešno prijavljivanje". (IA)



Slika 31: Dijagram sekvenci SK1, osnovni scenario

3.12.1.2. Slučaj korišćenja 2 – Pregled svih ispita u aktuelnom ispitnom roku

Naziv SK

Pregled svih *ispita* u aktuelnom ispitnom *roku*

Akteri SK

Korisnik

Učesnici SK: *Korisnik* i *sistem* (program)

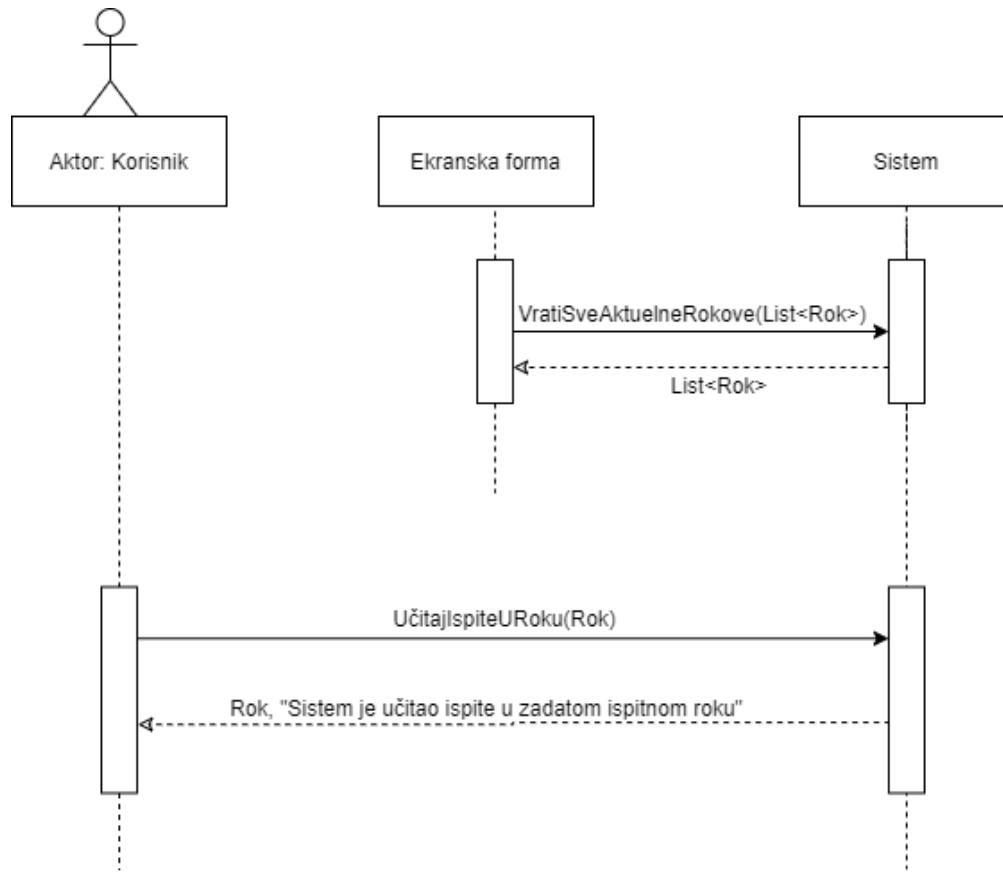
Preduslovi: *Sistem* je pokrenut i *korisnik* je prijavljen sa svojom šifrom. *Sistem* prikazuje formu za rad sa *rokovima*. Učitana je lista *aktuelnih rokova*.

Osnovni scenario SK:

1. *Korisnik* poziva *sistem* da učitava *ispite* u *roku*. (APSO)
2. *Sistem* traži *ispite* za ispitni *rok*. (SO)
3. *Sistem* prikazuje zaposlenom nađene *ispite* uz poruku “*Sistem* je učitao *ispite* u zadatom ispitnom roku “. (IA)

Alternativni scenariji:

- 3.1. Ukoliko *sistem* nije mogao da nađe *ispite*, on prikazuje *korisniku* poruku: “*Sistem* nije mogao da učitava *ispite* roku”. Prekida se izvršenje scenaria. (IA)



Slika 32: Dijagram sekvenci SK2, osnovni scenario

3.12.1.3. Slučaj korišćenja 3 – Pregled studentovih ispita u aktuelnom ispitnom roku

Naziv SK

Pregled svih studentovih *ispita* u aktuelnom ispitnom *roku*

Akteri SK

Student

Učesnici SK: *Student* i *sistem* (program)

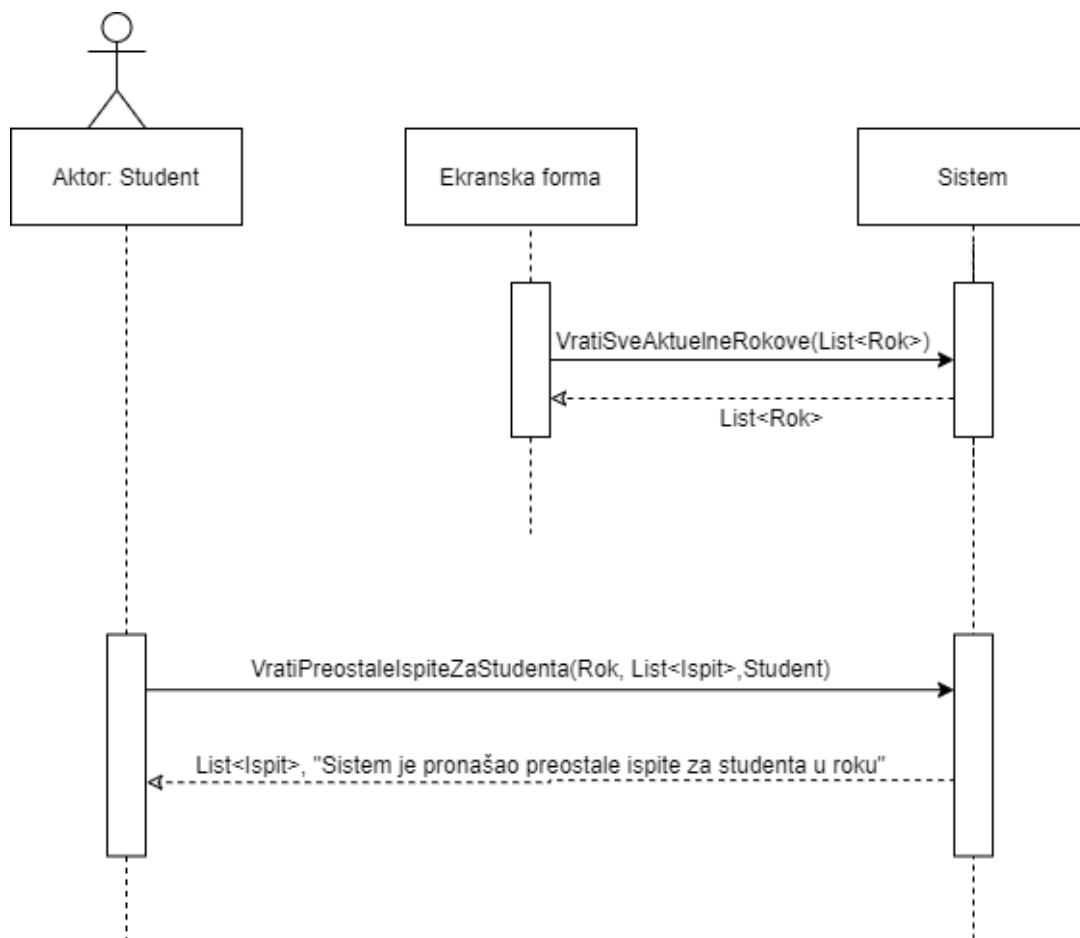
Preduslovi: *Sistem* je pokrenut i *student* je prijavljen sa svojom šifrom. *Sistem* prikazuje formu za rad sa *rokovima*. Učitana je lista *aktuelnih rokova*.

Osnovni scenario SK:

1. *Student* **poziva** *sistem* da vrati preostale *ispite* u zadatom *roku*. (APSO)
2. *Sistem* **traži** preostale *ispite* po zadatim vrednostima. (SO)
3. *Sistem* **prikazuje** zaposlenom nađene *ispite* uz poruku "*Sistem* je pronašao preostale *ispite* za *studenta* u *roku*". (IA)

Alternativni scenariji:

- 3.1. Ukoliko *sistem* nije mogao da nađe *ispite*, on prikazuje *studentu* poruku: "*Sistem* nije mogao da pronađe preostale *ispite* za *studenta*". Prekida se izvršenje scenaria. (IA)



Slika 33: Dijagram sekvenci SK3, osnovni scenario

3.12.1.4. Slučaj korišćenja 4 – Pregled uspešnih polaganja

Naziv SK

Pregled uspešnih *polaganja*

Akteri SK

Student

Učesnici SK

Student i *sistem* (program)

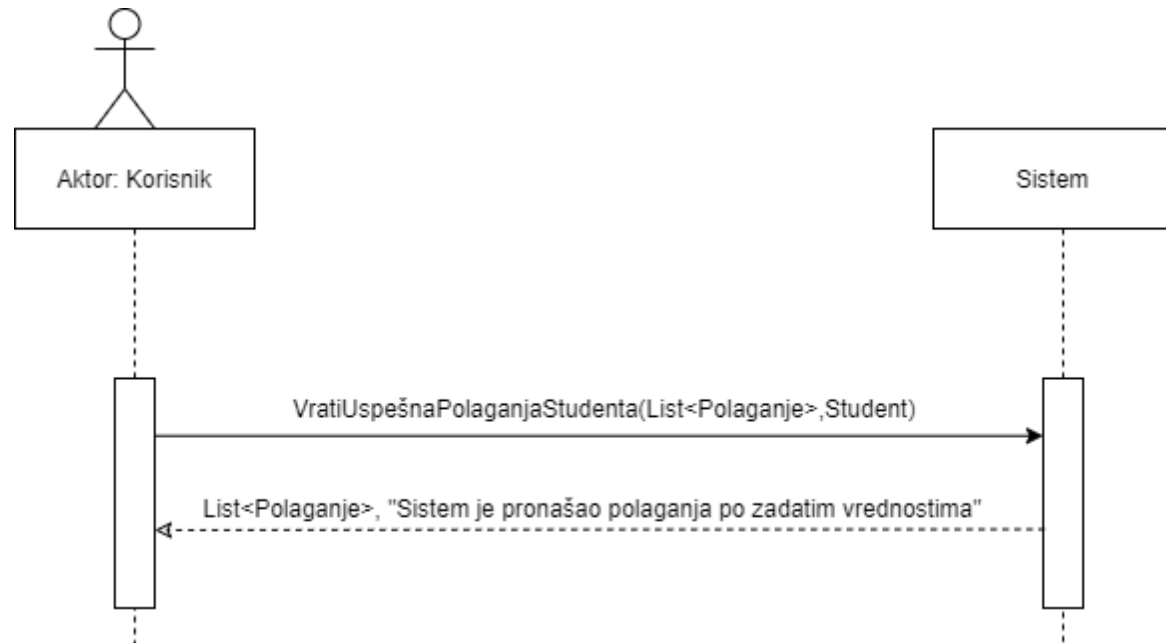
Preduslov: *Sistem* je pokrenut i *student* je prijavljen sa svojom šifrom.

Osnovni scenario SK:

1. *Student* poziva *sistem* da nađe uspešna *polaganja*. (APSO)
2. *Sistem* traži *polaganja* po zadatim vrednostima. (SO)
3. *Sistem* prikazuje *studentu* nađena *polaganja* uz poruku "*Sistem* je pronašao uspešna *polaganja* po zadatim vrednostima". (IA)

Alternativni scenarii:

3.1. Ukoliko *sistem* nije mogao da nađe *polaganja* za *unete podatke pretrage*, on prikazuje *studentu* poruku: "*Sistem* nije mogao da nađe uspešna *polaganja*". Prekida se izvršenje scenaria. (IA)



Slika 34: Dijagram sekvenci SK4, osnovni scenario

3.12.1.5. Slučaj korišćenja 5 – Prijavljivanje polaganja ispita

Naziv SK

Prijavljivanje *polaganja ispita*

Akteri SK

Student

Učesnici SK: *Student* i *sistem* (program)

Preduslovi: *Sistem* je uključen i *student* je prijavljen pod svojom šifrom. *Sistem* prikazuje formu za rad sa *prijavama*. Učitana je lista *neprijavljenih i postojećih prijava*.

Osnovni scenario SK:

1. *Student* bira ispit koji bi želeo da prijavi iz liste *neprijavljenih ispita*. (ANSO)
2. *Student* poziva *sistem* da zapamti *prijavu* za izabrani ispit. (APSO)
3. *Sistem* pamti podatke o *prijavi*. (SO)
4. *Sistem* prikazuje *studentu* poruku: "*Prijava* je uspešno sačuvana". (IA)

Alternativni scenarii:

- 4.1. Ukoliko *sistem* nije mogao da zapamti *prijavu* on prikazuje *studentu* poruku: "*Sistem* nije mogao da zapamti *prijavu*". (IA)

3.12.1.6. Slučaj korišćenja 6 – Odjavljivanje polaganja ispita

Naziv SK

Odjavljivanje *polaganja ispita*

Akteri SK

Student

Učesnici SK: *Student* i *sistem* (program)

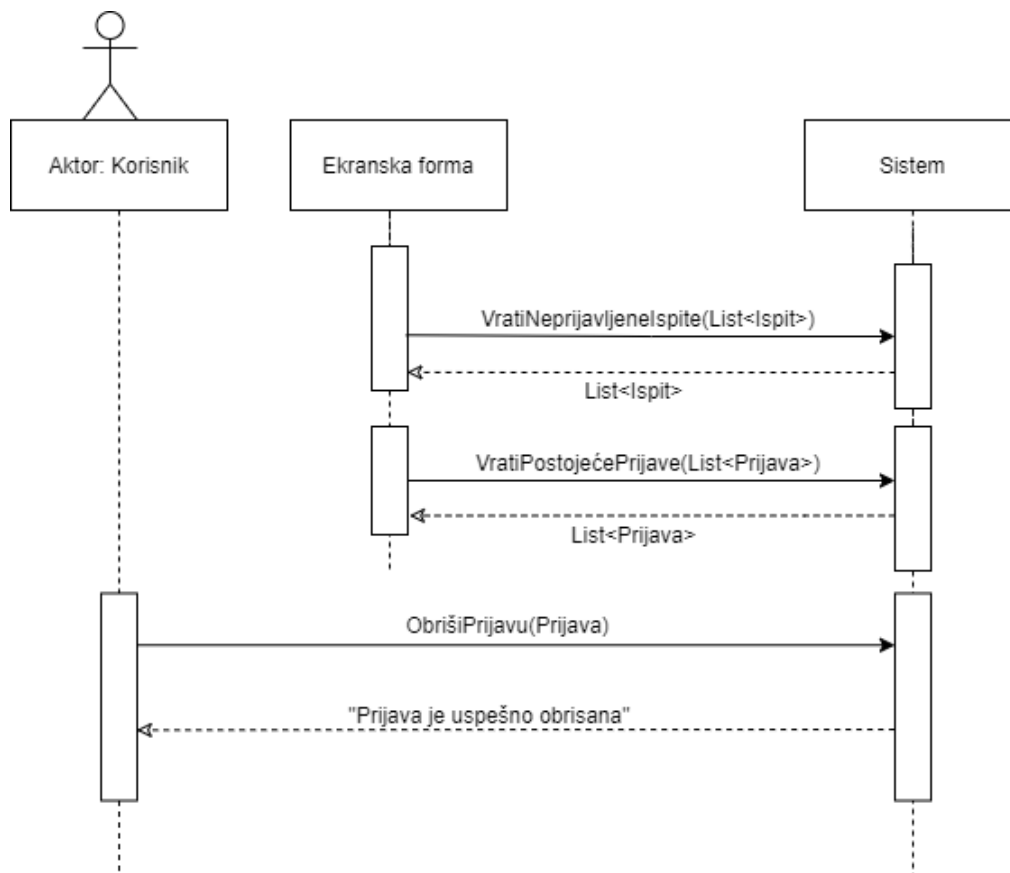
Preduslovi: *Sistem* je uključen i *student* je prijavljen pod svojom šifrom. *Sistem* prikazuje formu za rad sa *prijavama*. Učitana je lista *postojećih prijava*, učitana je lista *neprijavljenih ispita*.

Osnovni scenario SK:

1. *Student* bira ispit koji bi želeo da odjavi iz liste *postojećih prijava*. (ANSO)
2. *Student* poziva *sistem* da odjavi izabranu *prijavu ispita*. (APSO)
3. *Sistem* briše podatke o *prijavi*. (SO)
4. *Sistem* prikazuje *studentu* poruku: "*Prijava* je uspešno obrisana". (IA)

Alternativni scenarij:

4.1. Ukoliko *sistem* nije mogao da obriše *prijavu* on prikazuje *studentu* poruku: "*Sistem* nije mogao da obriše *prijavu*". (IA)



Slika 36: Dijagram sekvenci SK6, osnovni scenario

3.12.1.7 Slučaj korišćenja 7 – Ocenjivanje prijave studenata

Naziv SK

Ocenjivanje *prijave studenta*

Akteri SK

Admin

Učesnici SK: *Admin* i *sistem* (program)

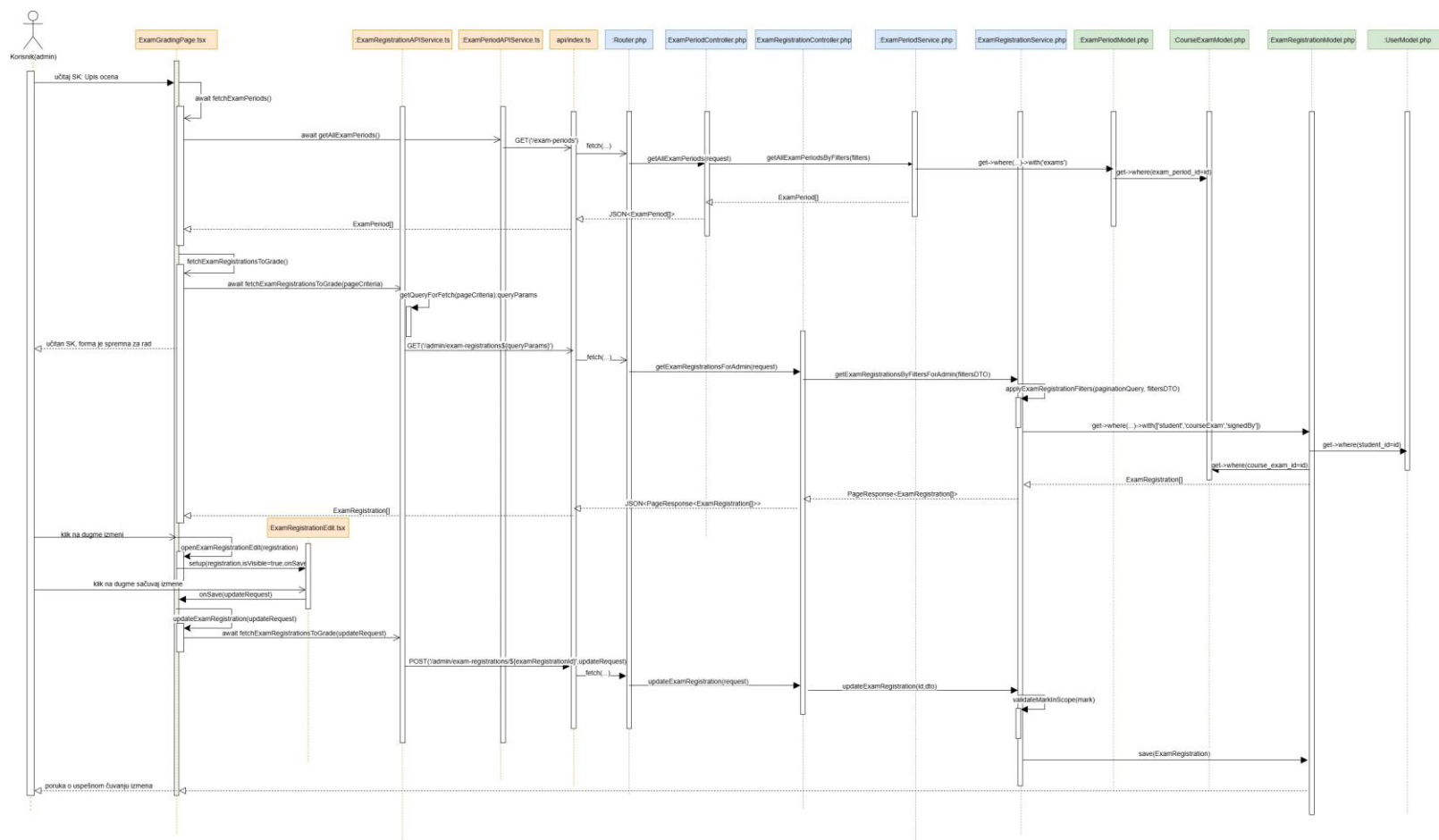
Preduslovi: *Sistem* je uključen i *admin* je prijavljen pod svojom šifrom. *Sistem* prikazuje formu za rad sa *prijavama*. Učitana je lista svih *postojećih ispitnih rokova sa ispitima i postojećih prijava* za tog *admina*.

Osnovni scenario SK:

1. *Admin* bira *prijavu* za koju želi da unese ocenu, komentar ili druge. (ANSO)
2. *Admin* unosi podatke za *prijavu*. (APUSO)
3. *Admin* poziva *sistem* da zapamti izmenjenu *prijavu*. (APSO)
4. *Sistem* pamti podatke o *prijavi*. (SO)
5. *Sistem* prikazuje *studentu* poruku: "*Prijava* je uspešno izmenjena". (IA)

Alternativni scenarii:

4.1. Ukoliko *sistem* nije mogao da zapamti izmene za *prijavu* on prikazuje *studentu* poruku: "*Sistem* nije mogao da zapamti *prijavu*". (IA)



Slika 37: Dijagram sekvenci SK7, osnovni scenario

3.12.1.8. Slučaj korišćenja 8 – Odjavljivanje korisnika

Naziv SK

Odjavljivanje *korisnika*

Akteri SK

Korisnik

Učesnici SK: *Korisnik* i *sistem* (program)

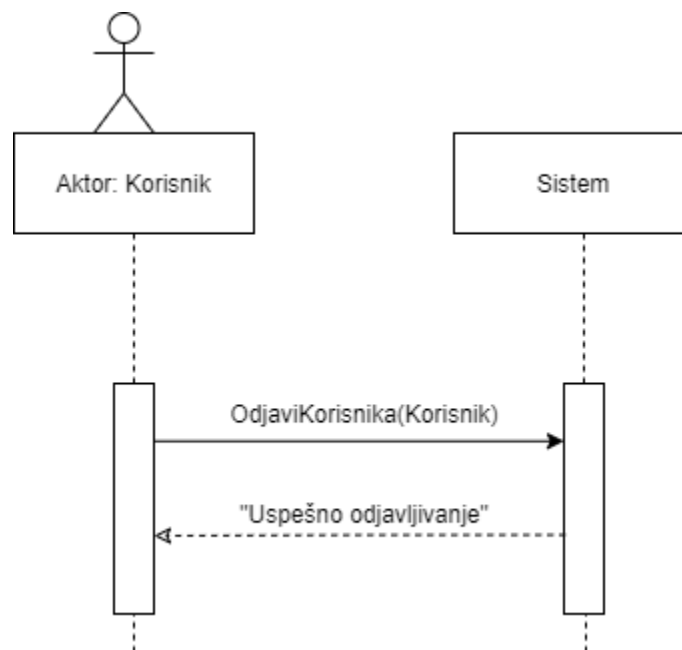
Preduslovi: *Sistem* je pokrenut i *korisnik* je prijavljen sa svojom šifrom.

Osnovni scenario SK:

1. *Korisnik* poziva *sistem* da vrati izloguje *korisnika*. (APSO)
2. *Sistem* pronalazi i odjavljuje prijavljenog *korisnika*. (SO)
3. *Sistem* prikazuje *korisniku* poruku "Uspešno ste odjavljeni". (IA)

Alternativni scenarij:

- 3.1. Ukoliko *sistem* nije mogao da pronađe *korisnika*, on prikazuje *korisniku* poruku: "Neuspešno odjavljivanje". (IA)



Slika 38: Dijagram sekvenci SK8, osnovni scenario

3.12.1.9. Slučaj korišćenja 9 – Pregled izveštaja ispita

Naziv SK

Pregled izveštaja *ispita*

Akteri SK

Korisnik

Učesnici SK: *Profesor* i *sistem* (program)

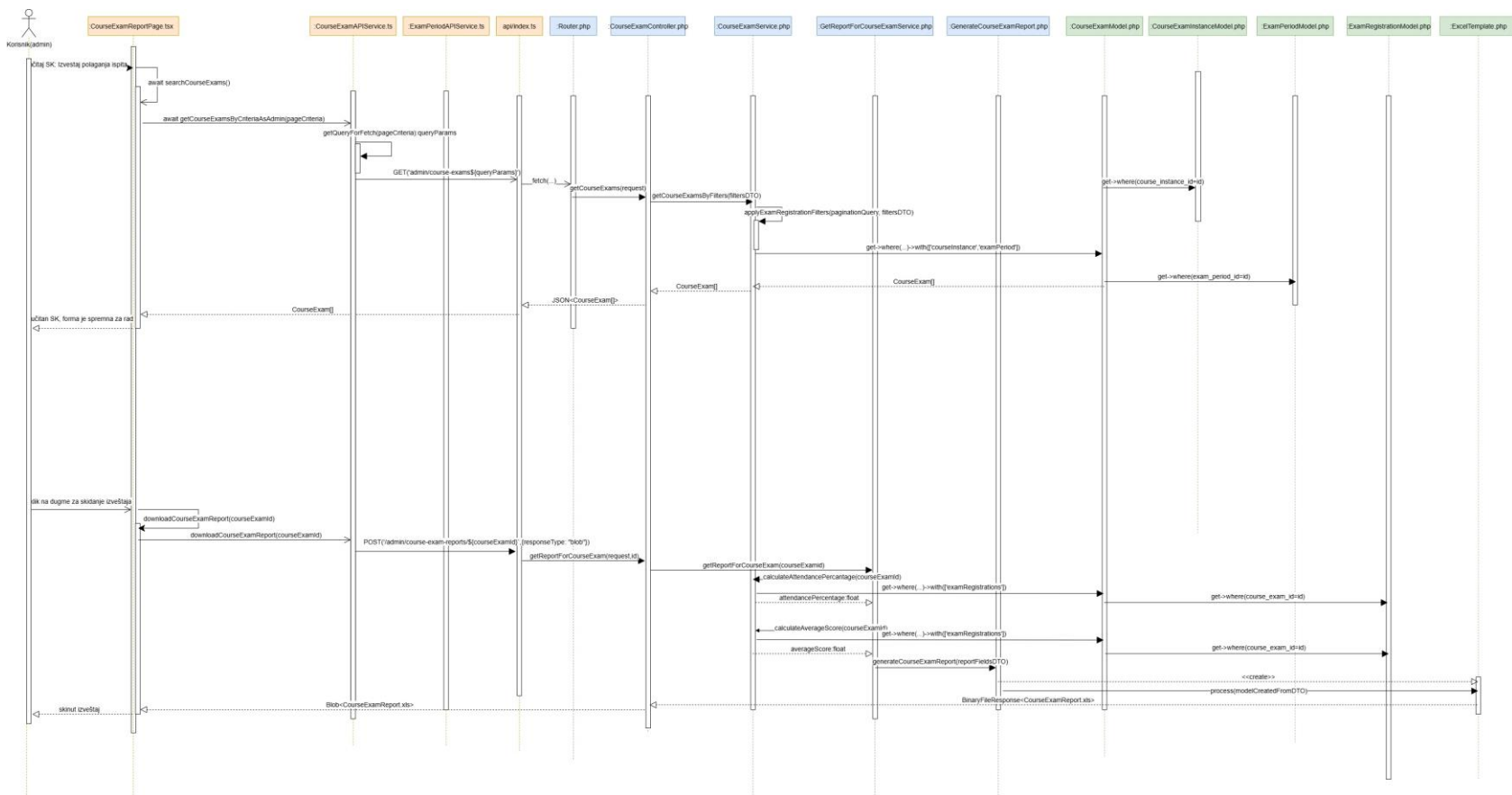
Preduslovi: *Sistem* je pokrenut i *profesor* je prijavljen sa svojom šifrom. *Sistem* prikazuje formu za rad sa *ispita*. Učitana je lista *ispita*.

Osnovni scenario SK:

1. *Profesor* poziva *sistem* da prikaže *izveštaj ispita* sa zadatim vrednostima. (APSO)
2. *Sistem* generiše *izveštaj ispita* po zadatim vrednostima. (SO)
3. *Sistem* prikazuje *profesoru izveštaj polaganja* uz poruku “*Sistem* je generisao *izveštaj ispita* po zadatim vrednostima”. (IA)

Alternativni scenarij:

3.1. Ukoliko *sistem* nije mogao da nađe *ispit* , on prikazuje *profesoru* poruku: “*Sistem* nije uspeo da generiše *izveštaj ispita* po zadatim vrednostima”. (IA)



Slika 39: Dijagram sekvenci SK9, osnovni scenario

3.12.1.10. Slučaj korišćenja 10 – Pregled izveštaja predmeta u školskoj godini

Naziv SK

Pregled izveštaja *predmeta u školskoj godini*

Akteri SK

Korisnik

Učesnici SK: *Profesor* i *sistem* (program)

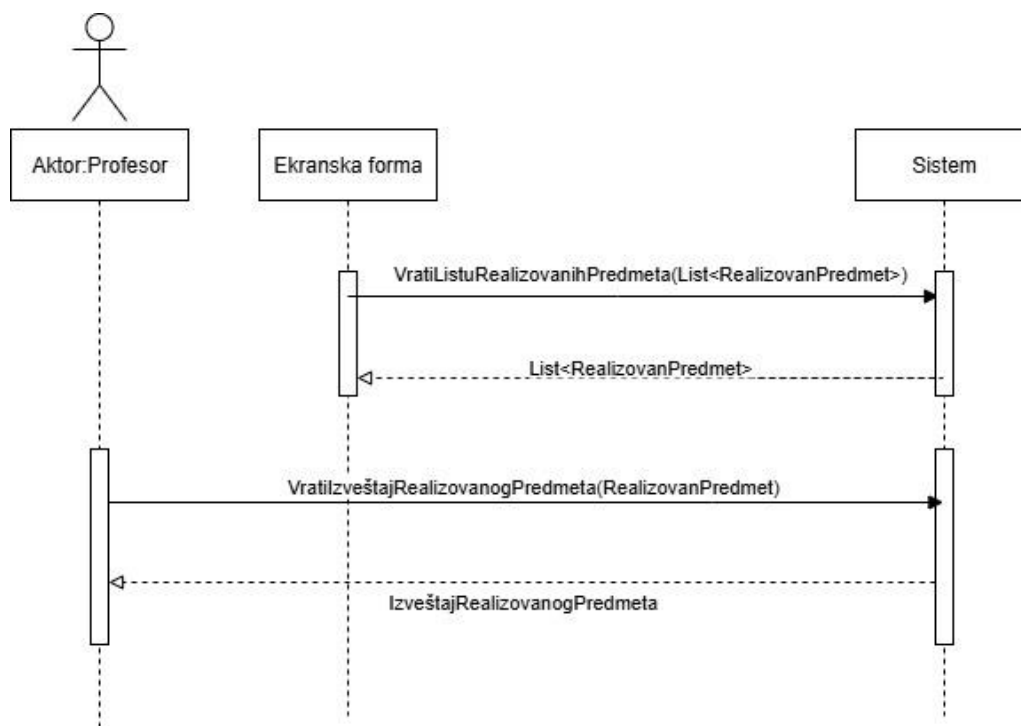
Preduslovi: *Sistem* je pokrenut i *profesor* je prijavljen sa svojom šifrom. *Sistem* prikazuje formu za rad sa *realizovanim predmetima*. Učitana je lista *realizovanih predmeta*.

Osnovni scenario SK:

1. *Profesor* poziva *sistem* da prikaže *izveštaj realizovanog predmeta* sa zadatim vrednostima. (APSO)
2. *Sistem* generiše *izveštaj realizovanog predmeta* po zadatim vrednostima. (SO)
3. *Sistem* prikazuje *profesoru izveštaj realizovanog predmeta* uz poruku “*Sistem* je generisao *izveštaj realizovanog predmeta* po zadatim vrednostima”. (IA)

Alternativni scenarij:

3.1. Ukoliko *sistem* nije mogao da nađe *realizovan predmet*, on prikazuje *profesoru* poruku: “*Sistem* nije uspeo da generiše *izveštaj realizovanog predmeta* po zadatim vrednostima”. (IA)



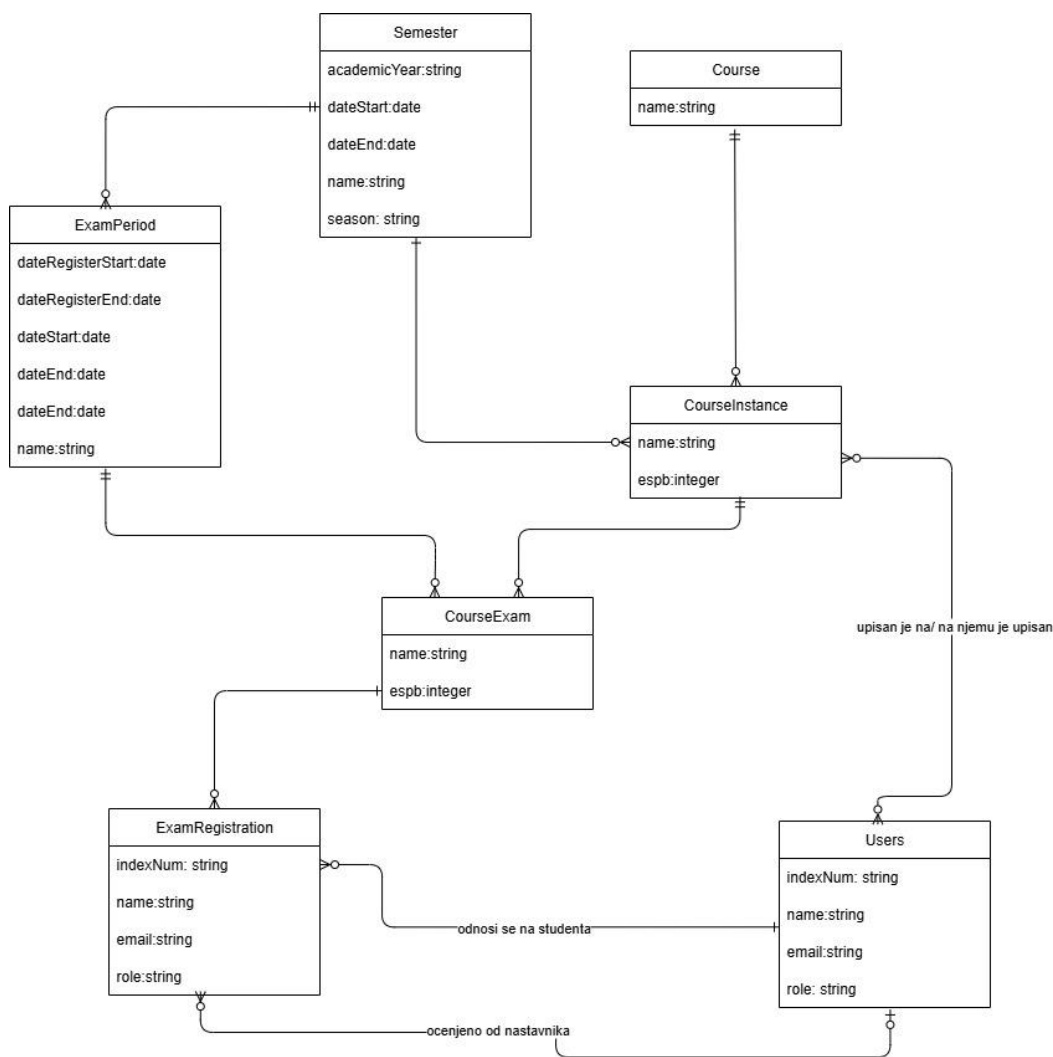
Slika 40: Dijagram sekvenci SK10, osnovni scenario

Sa navedenih sekvencnih dijagrama uočavaju se 15 sistemskih operacija koje treba projektovati:

1. signal **PrijaviKorisnika(Korisnik)**
2. signal **VratiSveAktuelneRokove(List<Rok>)**

3. signal **UčitajIspiteURoku(Rok)**
4. signal **VratiPreostaleIspiteZaStudenta(Rok, List<Ispit>,Student)**
5. signal **VratiUspešnaPolaganjaStudenta(List<Polaganje>,Student)**
6. signal **VratiNeprijavljenIspite(List<Ispit>)**
7. signal **VratiPostojećePrijava(List<Prijava>)**
8. signal **SačuvajPrijavu(Prijava)**
9. signal **ObrišiPrijavu(Prijava)**
10. signal **IzmeniPrijavu(Prijava)**
11. signal **OdjaviKorisnika(Korisnik)**
12. signal **VratiListuIspita(List<Ispit>)**
13. signal **VratilzveštajIspita(Ispit)**
14. signal **VratiListuRealizovanihPredmeta(List<RealizovaniPredmet>)**
15. signal **VratilzveštajRealizovanogPredmeta(RealizovaniPredmet)**

Struktura softverskog sistema je opisana preko konceptualnog dijagrama korišćenjem Crow's Foot ER notacije:



Slika 41: Konceptualni model softverskog sistema

3.13 Rečnik

Realizovan predmet – pojavljivanje predmeta(CourseInstace) u odredjenoj školskoj godini.

4. Studijski primer - prikaz realizovanog rešenja

4.1 Implementacija na klijentskoj strani

Klijentska strana je pisana u React.js-u korišćenjem višeslojne (*Page-Component-Service*) arhitekture. React.js održava sopstvenu kopiju pretraživačkog (browser) DOM-a (*Document Object Model*), poznatu kao *Virtual DOM*. Ova tehnologija postoji jer je direktno ažuriranje stvarnog DOM-a pri svakoj promeni

sporo i zahteva mnogo resursa. Umesto toga, React pretvara kod svojih komponenata u kod stvarnih DOM elemenata i inteligentno ažurira samo neophodne delove, što omogućava visoke performanse i obezbeđuje brzo i fluidno korisničko iskustvo u web aplikacijama.

Pored toga, React je *Single Page Application (SPA)*, što znači da se, za razliku od klasičnih HTML/CSS/JS stranica, celokupan sadržaj učitava samo jednom. Nakon toga, podaci se dinamički menjaju i ubacuju u istu stranicu, odnosno DOM, korišćenjem JavaScript-a, čime se postiže brža interakcija bez nepotrebnih osvežavanja stranice.

Unutar App.js u zavisnosti od izabrane rute, renderuje se odgovarajuća strana(page). BrowserRouter i Routes su react komponente koje presreću upite ka različitim stranicama i uslužuju rute korisniku. U slučaju da korisnik nije ulogovan, prosledjuje ga na login stranicu. U slučaju da jeste ulogovan UserProvider(moja custom context komponenta) će vratiti ulogovanog korisnika sa rolom radi customizovanja prikaza u AppLayout u zavisnosti od role. Npr student ne treba da može sam sebi da oceni prijavu, ili profesor ne treba da vidi svoj prosek i preostali broj ispita. AlertServiceProvider je globalno dostupna servis komponenta za prikaz grešaka korisniku.

```
src > App.tsx > App
import { LoginPage } from "@pages/LoginPage";
4 import { UserProvider } from "@context/UserContext";
5 import { AlertServiceProvider } from "@context/AlertServiceContext";
6 import RequireAuth from "@context/RequireAuth";
7 import { routes } from "@routes";
8
9 const App = () => {
10   return (
11     <div className="App">
12       <UserProvider>
13         <AlertServiceProvider>
14           <BrowserRouter basename="/estudent">
15             <Routes>
16               <Route path="/login" element={<LoginPage />} />
17               <Route path="/" element={<AppLayout />}>
18                 {routes.map(({ path, element }) => (
19                   <Route
20                     key={path}
21                     path={path}
22                     element={<RequireAuth>{element}</RequireAuth>}
23                   />
24                 ))}
25             </Route>
26             <Route path="*" element={<Navigate to="/login" replace />} />
27           </Routes>
28         </BrowserRouter>
29       </AlertServiceProvider>
30     </UserProvider>
31   </div>
32 );
33 };
34
```

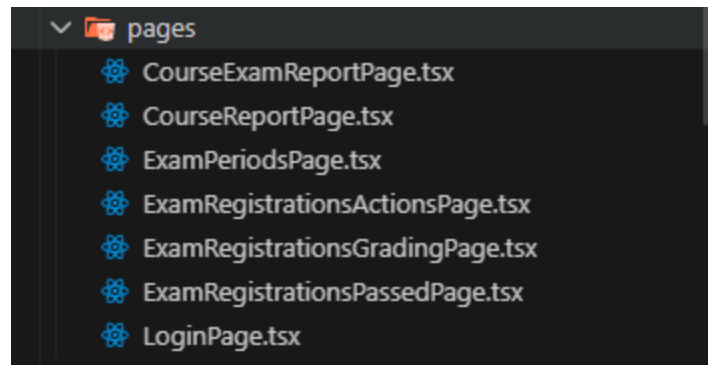
Slika 42: App.js, UserProvider,AlertServiceProvider


```

CourseReportPage.tsx ExamPeriodsPage.tsx course.ts examPeriods.ts courseExams.ts AlertServiceCon
src > layout > AppLayout.tsx > AppLayout
1  import { useState } from "react";
2  import React from "react";
3  import { Outlet, useNavigate } from "react-router-dom";
4  import logoFon from "@assets/logo-fon.png";
5  import {
6    NavigationDrawer,
7    List,
8    ListItem,
9  } from "@components/custom/NavigationDrawer";
10 import { useUser } from "@context/UserContext";
11
12 export const AppLayout = () => {
13   const navigate = useNavigate();
14   const [sideBarExpanded, setSideBarExpanded] = useState(false);
15   const { user, isAdmin, isStudent, logOut } = useUser();
16
17   return (
18     <>
19       <div className="wrapper align-items-stretch w-100 h-100">
20         { /* Sidebar */ }
21       <NavigationDrawer...
22     </NavigationDrawer>
23   <div
24     id="content"
25     className={`content p-4 p-md-4 ${sideBarExpanded ? "expanded" : ""}`>
26       <div className="d-flex flex-row justify-content-center align-items-center mb-5 header-row">
27         <div className="headerDiv flex-grow-1 text-center">
28           <span className="h4">
29             Studentski servisi - <i>Fakultet organizacionih nauka</i>
30           </span>
31         </div>
32         <ul className="nav header d-flex flex-row align-items-center ms-auto mb-0">
33           <li className="nav-item me-3 d-flex align-items-center">
34             <span className="fa fa-user secondary-darken-1 me-1"></span>
35             <span className="secondary">{user?.name}</span>
36           </li>
37           <li
38             className="nav-item logout-pointer d-flex align-items-center secondary-darken-1"
39             onClick={logOut}>
40             <span className="me-1">Izloguj me</span>
41             <span className="fa fa-power-off"></span>
42           </li>
43         </ul>
44       </div>
45       { /* Route-dependent content */ }
46       <Outlet />
47     </div>
48   </div>
49 </>
50 );
51 };

```

Slika 43: AppLayout



Slika 44: pages folder

```
src > routes > index.tsx > ...
1  import "typeface-poppins";
2  import ExamPeriodsPage from "@pages/ExamPeriodsPage";
3  import LoginPage from "@pages/LoginPage";
4  import ExamRegistrationsPassedPage from "@pages/ExamRegistrationsPassedPage";
5  import ExamRegistrationsActionsPage from "@pages/ExamRegistrationsActionsPage";
6  import CourseReportPage from "@pages/CourseReportPage";
7  import CourseExamReportPage from "@pages/CourseExamReportPage";
8  import ExamRegistrationsGradingPage from "@pages/ExamRegistrationsGradingPage";
9
10 export const routes = [
11   { path: "/", element: <ExamPeriodsPage /> },
12   { path: "/login", element: <LoginPage /> },
13   { path: "/passed-exams", element: <ExamRegistrationsPassedPage /> },
14   { path: "/exam-registration", element: <ExamRegistrationsActionsPage /> },
15   { path: "/course-report", element: <CourseReportPage /> },
16   { path: "/course-exam-report", element: <CourseExamReportPage /> },
17   { path: "/grade-entry", element: <ExamRegistrationsGradingPage /> },
18 ];
19
```

Slika 45: mapiranje ruta na page komponente

U stranicama su manje celine koda koje se ponavljaju poput tekstualnih inputa, tabela, dialoga grupisane u odvojene komponente, radi bolje organizacije. Reaktivnost se ostvaruje korišćenjem useState i useEffect react hookova. Pozivi ka serverskoj strani su izdvojeni u odvojene servis komponente.

```

1  /* eslint-disable react-hooks/exhaustive-deps */
2  import { useEffect, useState } from "react";
3  import { useUser } from "@context/UserContext";
4  import { useAlertService } from "@context/AlertServiceContext";
5  import { ExamPeriodAPIService } from "@api/examPeriods";
6  import { CourseExamAPIService } from "@api/courseExams";
7  import {
8    ExamPeriod,
9    ExamPeriodPresentation,
10   CourseExamPresentation,
11 } from "@types/items";
12 import { toExamPeriodPresentations } from "@utils/examPeriodUtils";
13 import { toCourseExamPresentations } from "@utils/courseExamsUtils";
14 import Container from "@components/custom/Container";
15 import Title from "@components/custom/Title";
16 import Info from "@components/custom/Info";
17 import Table from "@components/custom/Table";
18 import Button from "@components/custom/Button";
19 import log from "loglevel";
20
21 const ExamPeriodsPage = () => {
22   //region Consts and imports
23   const alertService = useAlertService();
24   const { user } = useUser();
25   //endregion Consts
26
27   //region useState---
28
29   //region API Functions
30   const fetchActiveExamPeriods = async () => { ...
31   };
32
33   const fetchRemainingCourseExams = async (examPeriod: ExamPeriod) => {
34     try {
35       const courseExams = await CourseExamAPIService.getRemainingCourseExams(
36         examPeriod
37       );
38       setRemainingCourseExams(toCourseExamPresentations(courseExams));
39     } catch (error) {
40       alertService.error("Došlo je do greške prilikom obrade zahteva.");
41       log.error(error);
42     }
43   };
44   //endregion API Functions
45
46   //region Other Handlers---
47
48   //region OnMount---
49
50   return (
51     <Container>
52       <Title>Rokovi</Title>
53       <div>
54         {examPeriods.length === 0 && isLoadingExamPeriods ? (...
55         ) : []}
56       </div>
57       <div>
58         {isAllCourseExamsVisible && (...
59         )}
60       </div>
61       <div>
62         {isRemainingCourseExamsVisible && (...
63         )}
64       </div>
65     </Container>
66   );
67 };
68
69 export default ExamPeriodsPage;

```

Slika 46: Pregled preostalih ispita studenta- ExamPeriodPage

```

src > api > courseExams.ts > CourseExamAPIService > getRemainingCourseExams
1  import {
2    BlobResponse,
3    CourseExam,
4    DocumentBlob,
5    ExamPeriod,
6  } from "@types/items";
7  import { CourseExamPageCriteria, PageResponse } from "@types/items";
8  import apiService from ".";
9  import { format } from "date-fns/format";
10
11 export class CourseExamAPIService {
12   static createQueryString(pageCriteria: CourseExamPageCriteria): string {
13     const queryParams: string[] = [];
14     queryParams.push(`page=${encodeURIComponent(pageCriteria.page)}`);
15     queryParams.push(`page-size=${encodeURIComponent(pageCriteria.pageSize)}`);
16     if (pageCriteria.searchText.length > 0)
17       queryParams.push(
18         `search-text=${encodeURIComponent(pageCriteria.searchText)}`
19       );
20     if (pageCriteria.dateFrom)
21       queryParams.push(
22         `date-from=${encodeURIComponent(
23           format(pageCriteria.dateFrom, "yyyy-MM-dd")
24         )}`
25       );
26
27     if (null !== pageCriteria.dateTo)
28       queryParams.push(
29         `date-to=${encodeURIComponent(pageCriteria.dateTo.toISOString())}`
30       );
31     return queryParams.length > 0 ? `?${queryParams.join("&")}` : "";
32   }
33
34   > static async getCourseExamsByCriteriaAsAdmin(...
42   }
43   static async getRemainingCourseExams(examPeriod: ExamPeriod) {
44     const response = await apiService.GET<CourseExam[]>(`
45       /course-exams/remaining-course-exams?for-exam-period-id=${examPeriod.id}`,
46       {}
47     );
48     return response;
49   }
50   static async getRegisterableCourseExams(examPeriod: ExamPeriod) {
51     const response = await apiService.GET<CourseExam[]>(`
52       /course-exams/registerable-course-exams?for-exam-period-id=${examPeriod.id}`
53     );
54     return response;
55   }
56   > static async downloadCourseExamReport(...
70   }
71
72

```

Slika 47: Pregled preostalih ispita studenta – servis courseExam.ts

```

19  async function GET<T>(  

20      path: string,  

21      headers: Record<string, string> = { "Content-Type": "application/json" }  

22  ): Promise<T> {  

23      const token = getAuthToken(path);  

24      const authHeaders: Record<string, string> = token  

25          ? { Authorization: `Bearer ${token}` }  

26          : {};  

27      const finalHeaders = {  

28          ...authHeaders,  

29          ...headers,  

30      };  

31      try {  

32          const response = await fetch(`${BASE_URL}${path}`, {  

33              method: "GET",  

34              headers: finalHeaders,  

35          });  

36          if (headers["responseType"] === "blob") return response as T;  

37          else return await handleResponse(response);  

38      } catch (error) {  

39          if (error instanceof EStudentError) throw error;  

40          else throw new Error(`GET request failed: ${(error as Error).message}`);  

41      }  

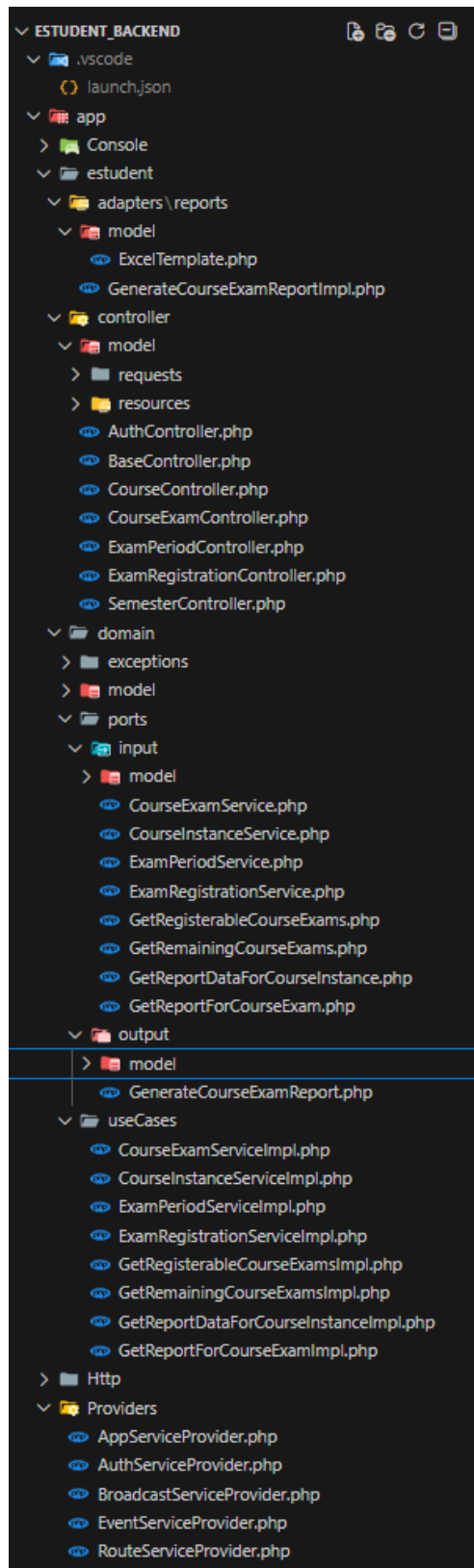
42  }

```

Slika 48: Implementacija GET funkcije korišćenjem axios biblioteke za HTTP pozive

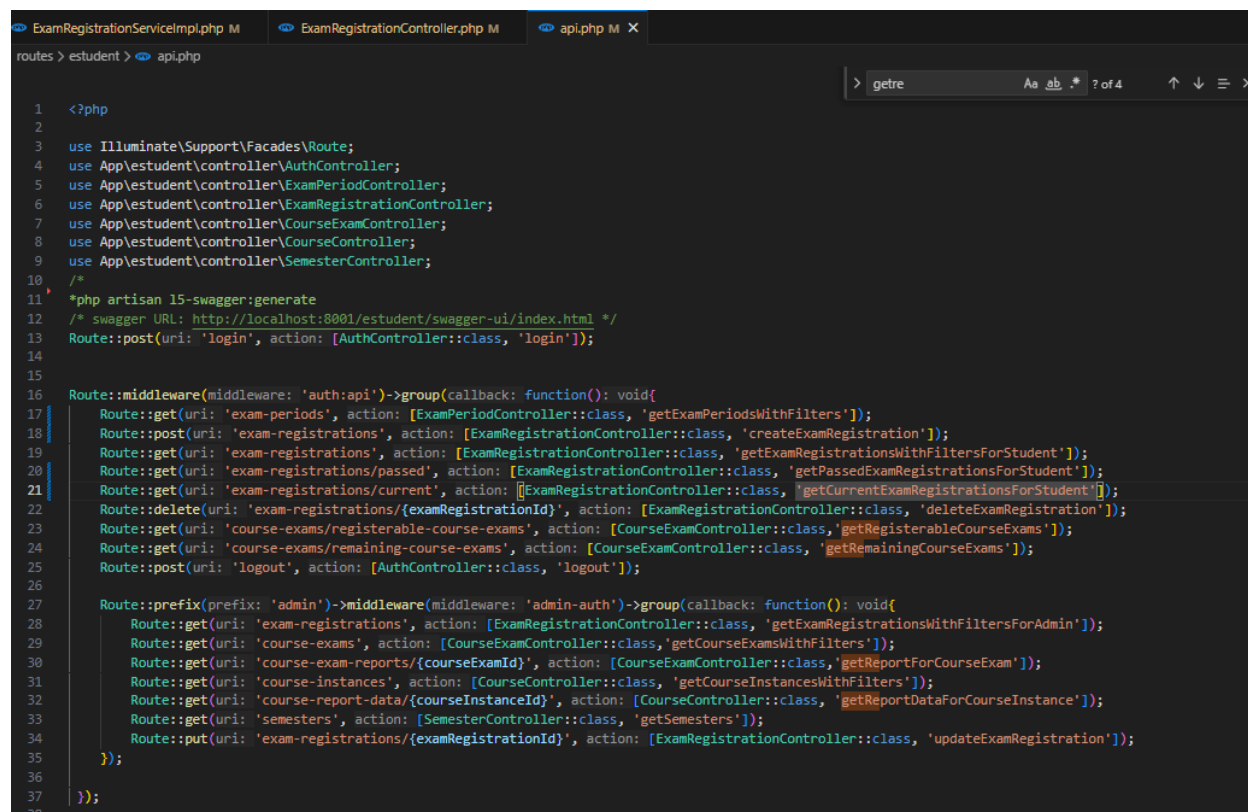
4.2 Implementacija na serverskoj strani

Rešenje na serverskoj strani je pisano u Laravel php frameworku korišćenjem hexagon arhitekture.



Slika 49: Organizacija koda prema Hexagon arhitekturi

Laravel na serverskoj strani prima zahteve na routeru api.php, poziva odgovarajući metodu u kontroleru koja korišćenjem Eloquent ORM-a nad Modelima fetch-uje i menja odgovarajuća stanja iz SQL Server baze podataka. Objekti klase modela ili kolekcije tih objekata se nakon toga konveruju u odgovarajući resurs i samo relevantni podaci se vraćaju klijentu kao odgovor. Pokreće se komandom php artisan serve. Implementacija biznis pravila će biti detaljnije objašnjena u odeljku 4.3 implementacija i korisničko uputstvo.

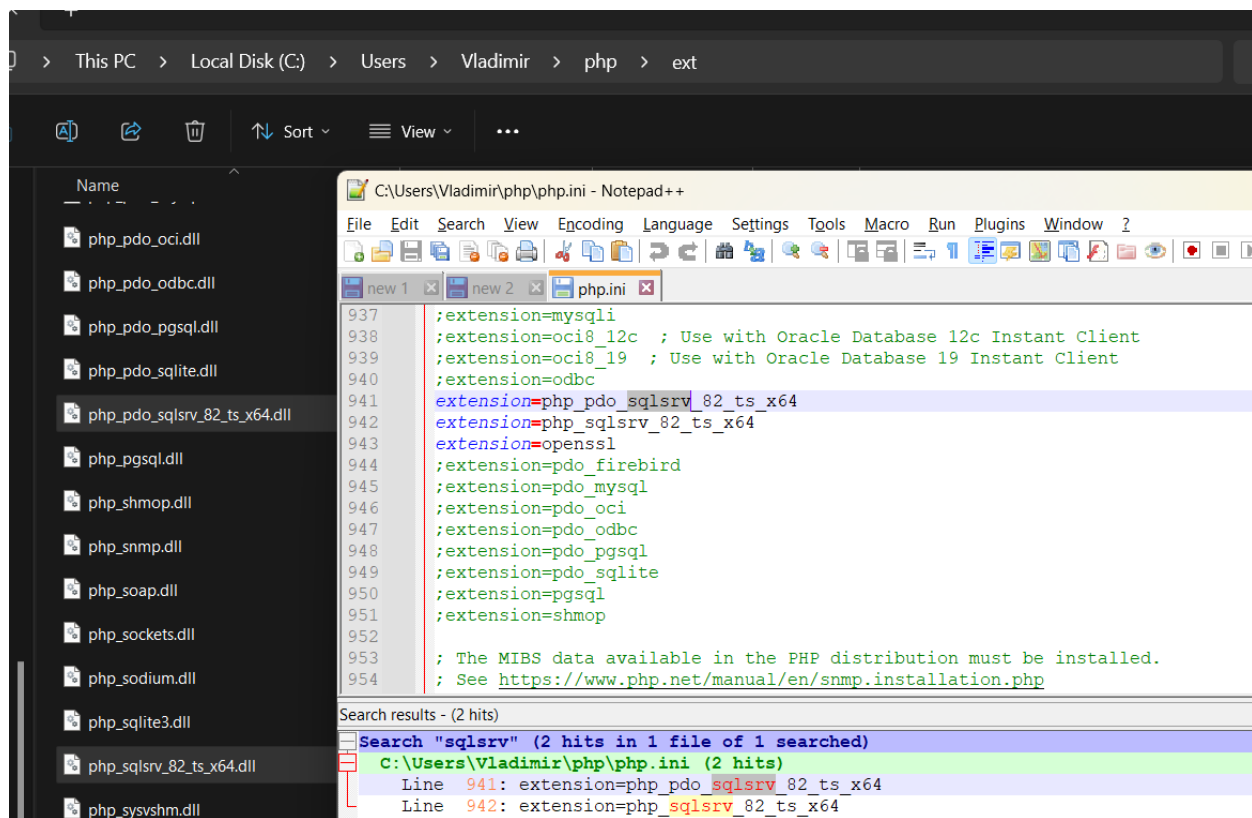


```
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use App\estudent\controller\AuthController;
5 use App\estudent\controller\ExamPeriodController;
6 use App\estudent\controller\ExamRegistrationController;
7 use App\estudent\controller\CourseExamController;
8 use App\estudent\controller\CourseController;
9 use App\estudent\controller\SemesterController;
10
11 /*
12  *php artisan 15-swagger:generate
13  * swagger URL: http://localhost:8001/estudent/swagger-ui/index.html */
14 Route::post(uri: 'login', action: [AuthController::class, 'login']);
15
16 Route::middleware(middleware: 'auth:api')->group(callback: function(): void{
17     Route::get(uri: 'exam-periods', action: [ExamPeriodController::class, 'getExamPeriodsWithFilters']);
18     Route::post(uri: 'exam-registrations', action: [ExamRegistrationController::class, 'createExamRegistration']);
19     Route::get(uri: 'exam-registrations', action: [ExamRegistrationController::class, 'getExamRegistrationsWithFiltersForStudent']);
20     Route::get(uri: 'exam-registrations/passed', action: [ExamRegistrationController::class, 'getPassedExamRegistrationsForStudent']);
21     Route::get(uri: 'exam-registrations/current', action: [ExamRegistrationController::class, 'getCurrentExamRegistrationsForStudent']);
22     Route::delete(uri: 'exam-registrations/{examRegistrationId}', action: [ExamRegistrationController::class, 'deleteExamRegistration']);
23     Route::get(uri: 'course-exams/registerable-course-exams', action: [CourseExamController::class, 'getRegisterableCourseExams']);
24     Route::get(uri: 'course-exams/remaining-course-exams', action: [CourseExamController::class, 'getRemainingCourseExams']);
25     Route::post(uri: 'logout', action: [AuthController::class, 'logout']);
26
27     Route::prefix(prefix: 'admin')->middleware(middleware: 'admin-auth')->group(callback: function(): void{
28         Route::get(uri: 'exam-registrations', action: [ExamRegistrationController::class, 'getExamRegistrationsWithFiltersForAdmin']);
29         Route::get(uri: 'course-exams', action: [CourseExamController::class, 'getCourseExamsWithFilters']);
30         Route::get(uri: 'course-exam-reports/{courseExamId}', action: [CourseExamController::class, 'getReportForCourseExam']);
31         Route::get(uri: 'course-instances', action: [CourseController::class, 'getCourseInstancesWithFilters']);
32         Route::get(uri: 'course-report-data/{courseInstanceId}', action: [CourseController::class, 'getReportDataForCourseInstance']);
33         Route::get(uri: 'semesters', action: [SemesterController::class, 'getSemesters']);
34         Route::put(uri: 'exam-registrations/{examRegistrationId}', action: [ExamRegistrationController::class, 'updateExamRegistration']);
35     });
36 });
37
38
```

Slika 50: Spisak ruta na serverskoj strani

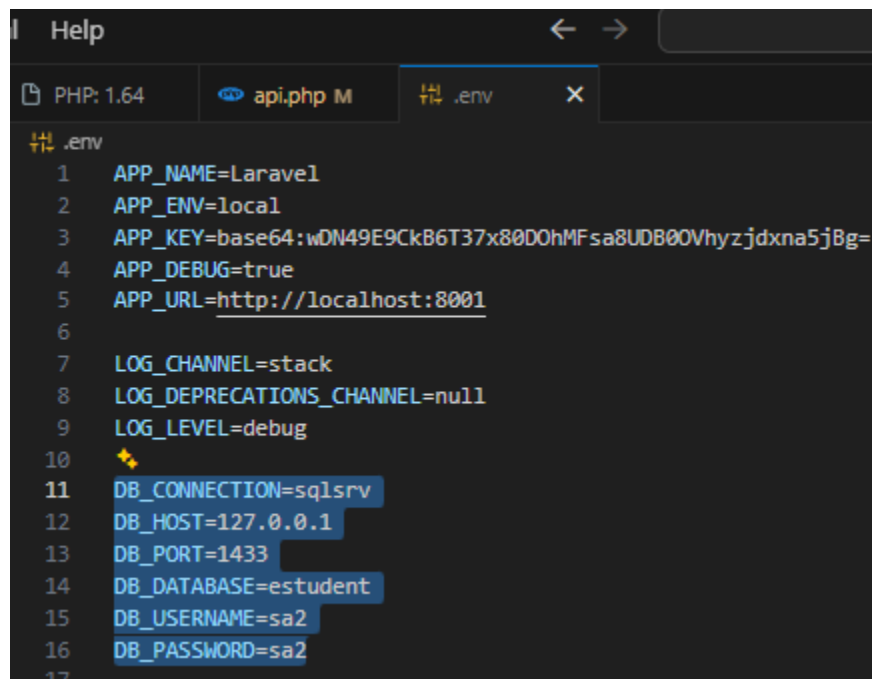
4.2.1 Konfiguracija na serverskoj strani: konekcije prema bazi i migracija dummy podataka

Pre pokretanja servera potrebno je konfigurisati .env fajl i bazu i instalirati php i composer. U slučaju konfiguracije za SQL Server 2019, potrebno je instalirati odgovarajući driver: [Download ODBC Driver for SQL Server - ODBC Driver for SQL Server | Microsoft Learn](#) . Drive je potrebno staviti u folder php\ext i u php.ini fajl dodati linije.



Slika 51: php/ext folder i php.ini

Nakon toga kroz Sql Server Management Studio potrebno je kreirati bazu sa imenom `estudent`, napraviti novog usera ili koristiti postojećeg koji ima adminska prava nad tom bazom.

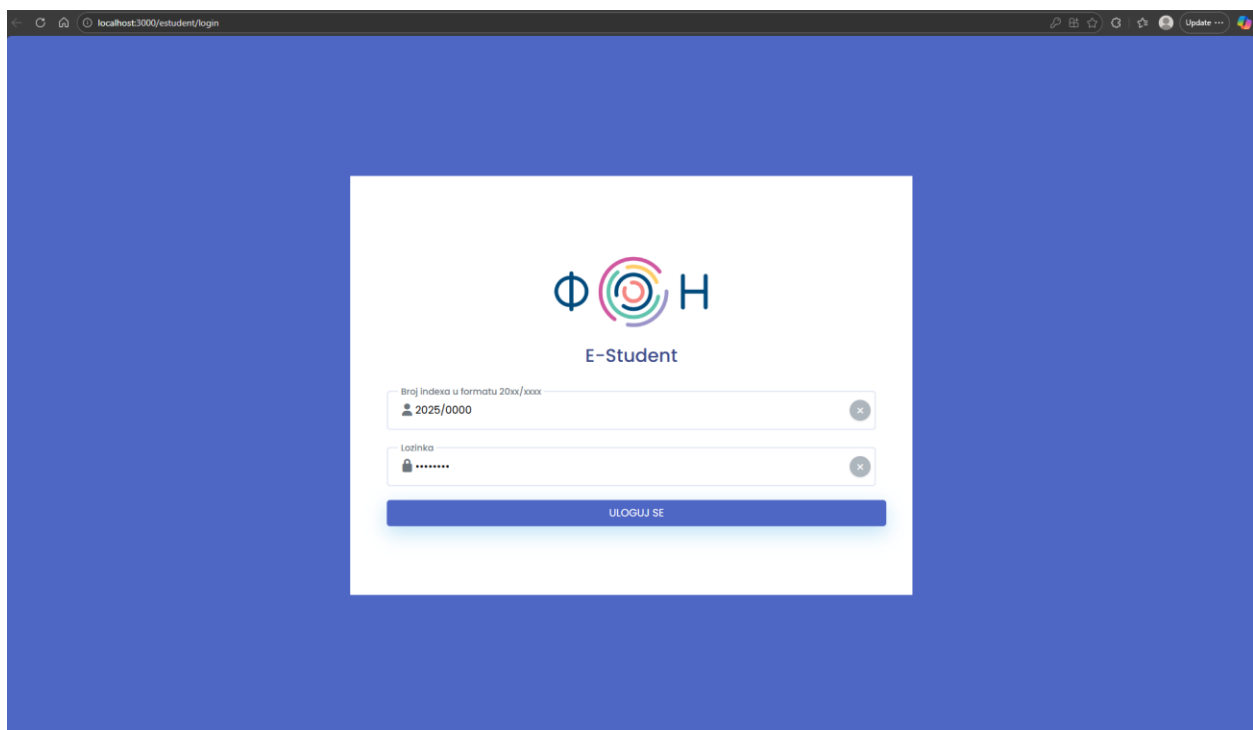


Slika 52: Konfiguracija Laravel - SQL Server konekcije

Radi efikasnosti git kopija pored source koda aplikacije, ne sadrži i konkretne pakete od kojih zavisi već samo njihove reference. Zato je potrebno instalirati sve potrebne pakete sa *\$composer install* komandom. Sa obzirom da aplikacija koristi Laravel Passport(oauth2) autentikaciju. Potrebno je kreirati tokene za laravel aplikaciju sto se radi sa komandom *\$php artisan key:generate* komandom. Nakon toga tabele i njihove veze se kreiraju korišćenjem *\$php artisan migrate* komande. Moguće je napuniti tabele sa dummy podacima korišćenjem *\$php artisan db:seed* komande. Server se na kraju pokreće sa komandom *\$php artisan serve*.

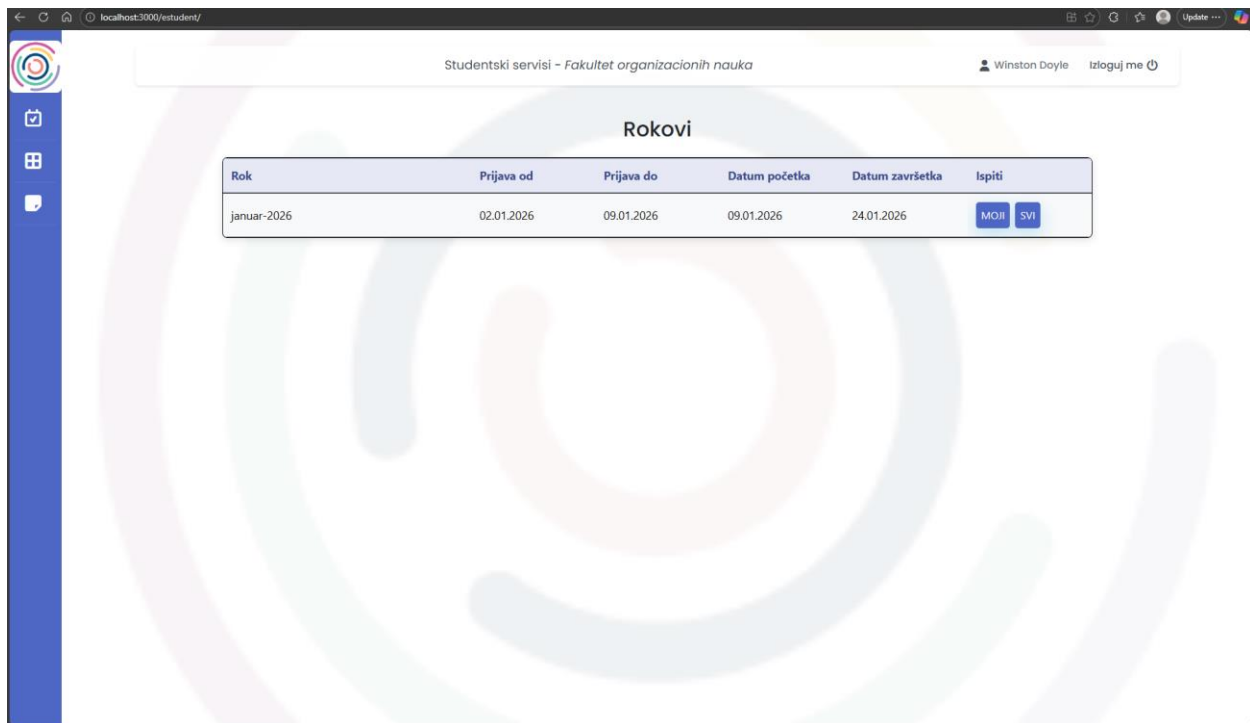
4.3 Implementacija i korisničko uputsvo

4.3.1 Uputstvo SK1: Prijavljivanje korisnika



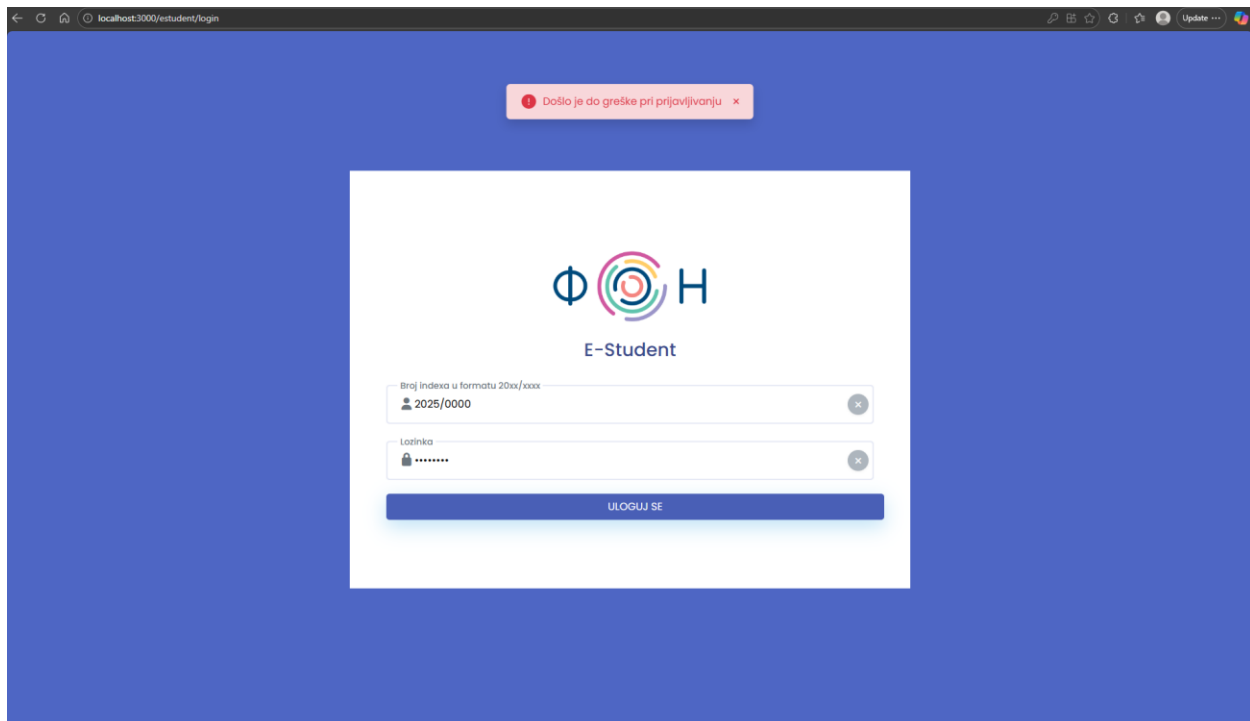
Slika 53: SK1:Izgled login forme

Opis: Nakon kucanja odgovarajućeg passworda i šifre klikom na dugme log in. Korisnik se loguje na system i prikazuje mu se HomePage komponenta. U slučaju da je poslat zahtev obrađen uspešno. Klijentu će biti prikazan HomePage.

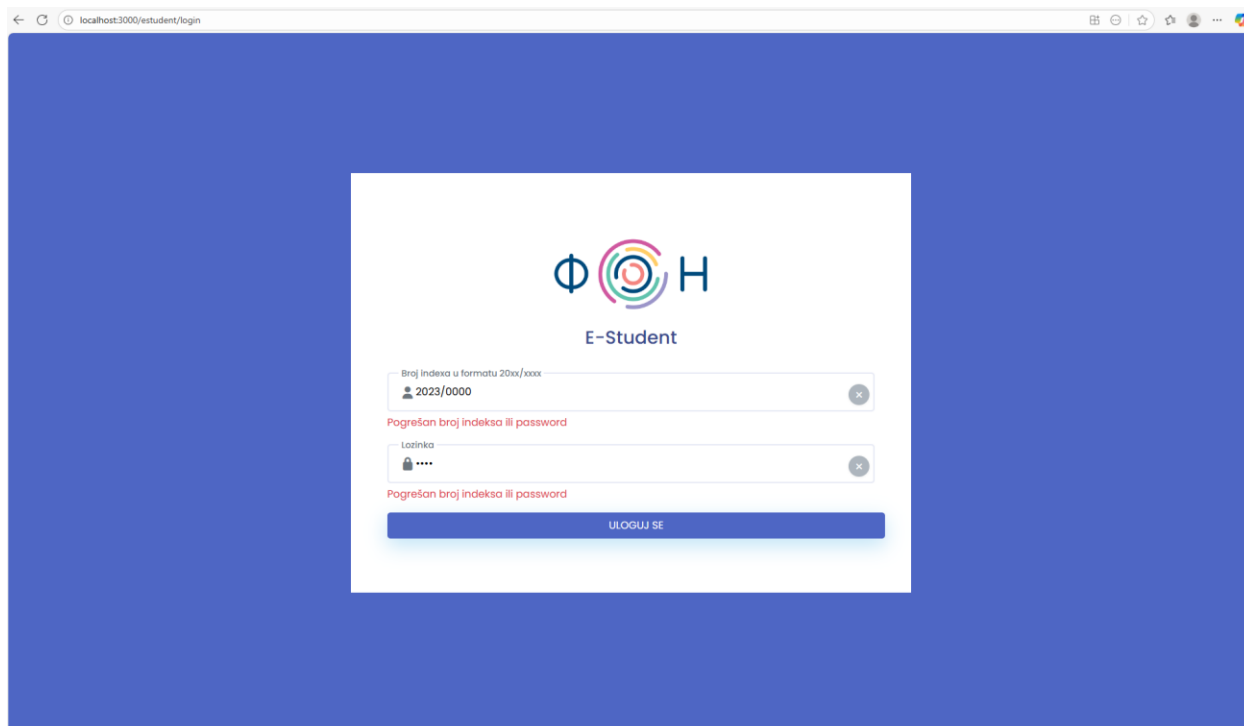


Slika 54: SK1: Uspešno logovanje

U slučaju da je zahtev poslat, a da server nije podignut klijentu će se poruka na 5 sekundi sa crvenim slovima: Došlo je do greške pri prijavljivanju.



Slika 55: SK1, poruka: Došlo je do greške pri prijavljivanju



Slika 56: SK1, poruka: Pogrešan broj indeksa ili password

```

/**
 * @OA\Post(
 *     path="/login",
 *     tags={"Common Routes"},
 *     summary="User login",
 *     @OA\RequestBody(
 *         required=true,
 *         @OA\JsonContent(
 *             required={"indexNum", "password"},
 *             @OA\Property(property="indexNum", type="string", example="2025/0000"),
 *             @OA\Property(property="password", type="string", example="password")
 *         )
 *     ),
 *     @OA\Response(
 *         response=200,
 *         description="User Login Successful"
 *     ),
 *     @OA\Response(
 *         response=401,
 *         description="Unauthorized"
 *     )
 * )
 */
1 reference | 0 overrides
public function login(Request $request): JsonResponse{
    $validator = Validator::make(data: $request->all(), rules: [
        'indexNum' => 'required|string',
        'password' => 'required|string',
    ]);

    if($validator->fails()){
        throw new BadRequestException(message: 'Bad Request, unauthorized');
    }
    if(Auth::attempt(credentials: [
        'indexNum'=>$request->indexNum,
        'password'=>$request->password])){
        $user = Auth::user();
        $result['id'] = $user->id;
        $result['indexNum'] = $user->indexNum;
        $result['name'] = $user->name;
        $result['email'] = $user->email;
        $result['role'] = $user->role;
        $result['token'] = $user->createToken('MyApp')->accessToken;

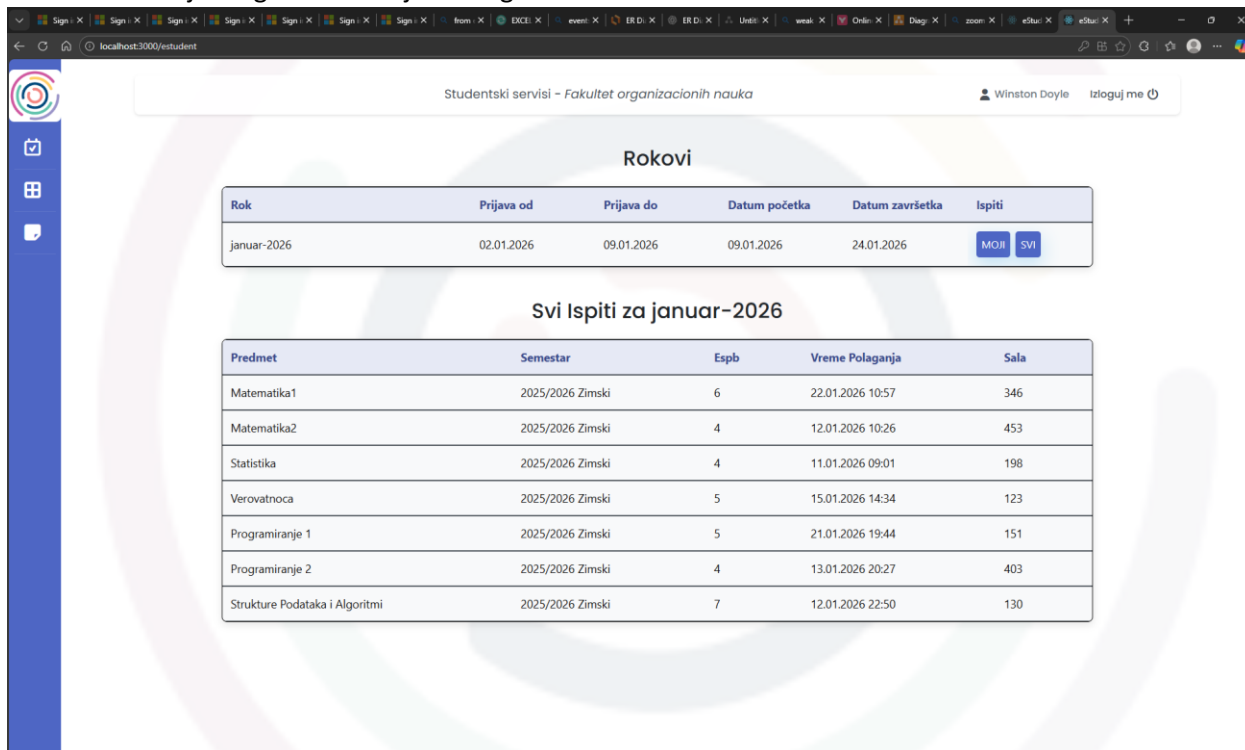
        return $this->createResponse(data: $result, message: "\nUser Login Successful!", code: 200);
    }else{
        throw new BadRequestException(message: 'Bad Credentials, unauthorized');
    }
}

```

Slika 57: SK1, Implementacija login metode

4.2.2 Uputstvo SK2: Pregled svih ispita u aktuelnom ispitnom roku

Uslov: Korisnik je ulogovan sa svojim nalogom.



Studentski servisi - Fakultet organizacionih nauka

Winston Doyle Izloguj me

Rokovi

Rok	Prijava od	Prijava do	Datum početka	Datum završetka	Ispiti
Januar-2026	02.01.2026	09.01.2026	09.01.2026	24.01.2026	MOJI SVI

Svi Ispiti za januar-2026

Predmet	Semestar	Espb	Vreme Polaganja	Sala
Matematika1	2025/2026 Zimski	6	22.01.2026 10:57	346
Matematika2	2025/2026 Zimski	4	12.01.2026 10:26	453
Statistika	2025/2026 Zimski	4	11.01.2026 09:01	198
Verovatnoca	2025/2026 Zimski	5	15.01.2026 14:34	123
Programiranje 1	2025/2026 Zimski	5	21.01.2026 19:44	151
Programiranje 2	2025/2026 Zimski	4	13.01.2026 20:27	403
Strukture Podataka i Algoritmi	2025/2026 Zimski	7	12.01.2026 22:50	130

Slika 58: SK2, Uspešno učitani svi ispiti u aktuelnom ispitnom roku

Opis: Klikom na dugme svi, korisniku se prikazuju svi ispiti u izabranom ispitnom roku. Ispitni rok je aktuelan ako je prijava ili njegovo održavanje trenutno u toku.

Svaki resurs ispitnog roka u sebi sadrži i sve ispite koji se mogu prijaviti u tom roku tako da postoji samo 1 api poziv, za učitavanje aktuelnih rokova. Gde se vraćaju samo rokovi čiji datum registracije je pre danasnjeg datuma, a datum završetka nakon danasnjeg datuma.

```

/**
 * @OA\Get(
 *   path="/exam-periods",
 *   tags={"Common Routes"},
 *   summary="Retrieve only currently active or all exam periods",
 *   security={
 *     {"passport": {*}}
 *   },
 *   @OA\Parameter(
 *     name="only-active",
 *     in="query",
 *     required=false,
 *     @OA\Schema(
 *       type="boolean"
 *     )
 *   ),
 *   @OA\Response(
 *     response=200,
 *     description="[Active]ExamPeriods retrieved successfully",
 *   ),
 * )
 */
0 references | 0 overrides
public function getExamPeriodsWithFilters(GetExamPeriodsRequest $request): JsonResponse
{
    $examPeriodFilters = $request->toDto();
    $examPeriods = $this->examPeriodService->getAllExamPeriodsWithFilters(filters: $examPeriodFilters);
    return $this->createResponse(data: ExamPeriodResource::collection(resource: $examPeriods),
    message: 'Exam periods retrieved successfully');
}

```

Slika 59: SK2, Controller metoda getExamPeriodsWithFilters

```

1 <?php
2 namespace App\estudent\domain\useCases;
3
4 use App\estudent\domain\ports\input\ExamPeriodService;
5 use App\estudent\domain\ports\input\model\ExamPeriodFilters;
6 use App\estudent\domain\model\ExamPeriod;
7 use Carbon\Carbon;
8 use Illuminate\Support\Collection;
9
10 4 references | 0 implementations
11 class ExamPeriodServiceImpl implements ExamPeriodService
12 {
13     1 reference | 0 overrides
14     public function getAllExamPeriodsWithFilters(ExamPeriodFilters $examPeriodFilters): Collection
15     {
16         if($examPeriodFilters->onlyActive)
17             return $this->active();
18         else{
19             return $this->all();
20         }
21     }
22
23     2 references | 0 overrides
24     public function active(): Collection
25     {
26         $currentDate = Carbon::now();
27         $examPeriods = ExamPeriod::with(relations: 'exams')->where(column: 'dateRegisterStart', operator: '<=', value: $currentDate)
28             ->where(column: 'dateEnd', operator: '>=', value: $currentDate)
29             ->get();
30
31         return $examPeriods;
32     }
33
34     1 reference
35     private function all(): Collection{
36         $examPeriods = ExamPeriod::with(relations: 'exams')->get();
37         return $examPeriods;
38     }
39
40     1 reference | 0 overrides
41     public function registerable(): Collection
42     {
43         $currentDate = Carbon::now();
44         $examPeriods = ExamPeriod::with(relations: 'exams')->where(column: 'dateRegisterStart', operator: '<=', value: $currentDate)
45             ->where(column: 'dateRegisterEnd', operator: '>=', value: $currentDate)
46             ->get();
47
48         return $examPeriods;
49     }
50 }

```

Slika 60: SK2, Vraćanje aktivnih rokova, metoda active()

4.2.3 Uputstvo SK3: Pregled studentovih ispita u aktuelnom ispitnom roku

Uslov: Student je ulogovan sa svojim nalogom.



📅

🏠

👤

Studentski servisi – Fakultet organizacionih nauka

Winston Doyle Izloguj me

Rokovi

Rok	Prijava od	Prijava do	Datum početka	Datum završetka	Ispiti
januar-2026	02.01.2026	09.01.2026	09.01.2026	24.01.2026	Moji SVI

Moji Ispiti za januar-2026

Predmet	Semestar	Espb	Vreme Polaganja	Sala
Matematika1	2025/2026 Zimski	6	22.01.2026 10:57	346
Statistika	2025/2026 Zimski	4	11.01.2026 09:01	198
Programiranje 2	2025/2026 Zimski	4	13.01.2026 20:27	403
Strukture Podataka i Algoritmi	2025/2026 Zimski	7	12.01.2026 22:50	130

Slika 61: SK3, Uspješno učitani samo studentovi ispiti u aktuelnom ispitnom roku

Opis: Klikom na dugme moji, studentu se prikazuju svi njegovi(nepoloženi) ispiti u izabranom ispitnom roku. Ispitni rok je aktuelan ako je prijava ili njegovo održavanje trenutno u toku. Preostali ispiti se dobijaju od endpointa /course-exams/remaining-course-exams/?for-exam-period-id.

Prvo se učitava rok i Id-evi svih kurseva na koji je student prijavljen (enrolledCourses). Zatim se ukrštaju svi ispiti koji se mogu polagati u tom roku sa svim ispitima na koji je student prijavljen kako bi se dobili svi ispiti koje student može da polaže (courseExams_where_enrolled). Na kraju se se uzimaju id-evi svih kurseva koje je student uspešno položio i oduzimaju iz skupa svih ispita da bi se na kraju dobila personalizovana lista ispita preostalih za tog studenta (\$courseExams_remaining).

```

2 references | 0 overrides
93 public function getRemainingCourseExams(int $examPeriodId): Collection
94
95     $examPeriod = ExamPeriod::with(relations: 'exams.courseInstance')->findOrFail(id: $examPeriodId);
96
97     $student = auth()->user();
98
99     $enrolledCourses = $student->courseInstances()->pluck('id')->toArray();
100
101     $courseExams_where_enrolled = $examPeriod->exams()->get()->reject(fn ($courseExam): bool=>
102         !in_array(needle: $courseExam->courseInstance->id, haystack: $enrolledCourses)
103     );
104
105     $examAttempts = ExamRegistration::with(relations: 'student', 'courseExam.courseInstance')
106     ->where(column: 'student_id', operator: $student->id->get());
107
108     $successfulAttempts = $examAttempts->count() > 0 ? $examAttempts->whereBetween(key: 'mark',
109     values: [6, 10])->pluck(value: 'courseExam.courseInstance.course_id')->toArray() : [];
110
111     $courseExams_remaining = $courseExams_where_enrolled->reject(fn ($courseExam): bool=>
112         in_array(needle: $courseExam->courseInstance->course_id, haystack: $successfulAttempts)
113     );
114     return $courseExams_remaining;
115
116

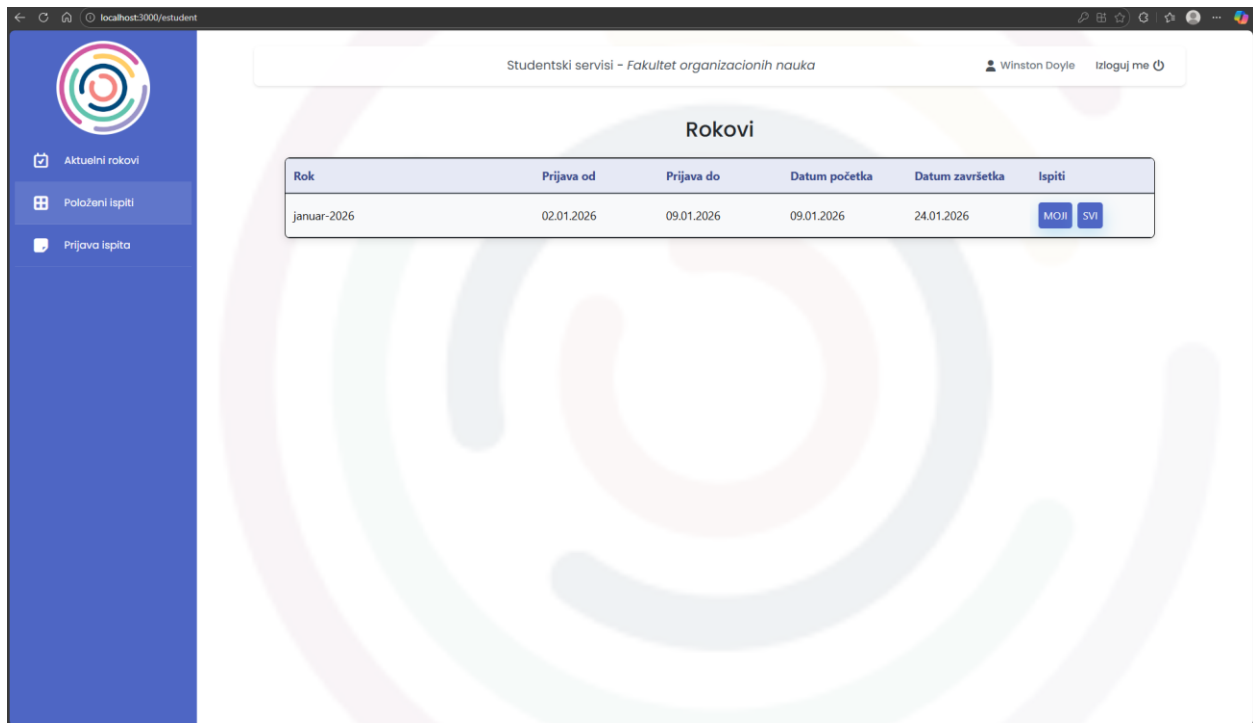
```

Slika 62: SK3, Metoda getRemainingCourseExams

4.2.4 Uputstvo SK4: Pregled uspešnih polaganja ispita

Uslov: Student je ulogovan sa svojim nalogom.

Opis: Nadnošenjem nad navigacioni bar, on se proširuje i potrebno je kliknuti Položeni ispiti učitavaju se svi položeni ispiti studenta i prikazuje se prosečna ocena i ukupan broj istvarenih espb.



The screenshot shows a web application interface for a student service portal. On the left is a blue sidebar with a logo and three menu items: 'Aktuelni rokovi', 'Položeni ispiti', and 'Prijava ispita'. The main content area has a header with the text 'Studentski servisi - Fakultet organizacionih nauka' and a user profile 'Winston Doyle' with a 'Izloguj me' button. Below the header is a section titled 'Rokovi' containing a table with exam dates.

Rok	Prijava od	Prijava do	Datum početka	Datum završetka	Ispiti
januar-2026	02.01.2026	09.01.2026	09.01.2026	24.01.2026	MOJI SVI

Slika 63: SK4, Navigacioni meni - student

R.Br	Naziv Ispita	Ocena	Esplb	Datum polaganja	Potpisao nastavnik	Dodatne informacije
1	Matematika2	6	4	08.01.2026	Vladimir Jecić	položio sa 51/100
2	Verovatnoca	7	5	08.01.2026	Vladimir Jecić	64/100
3	Programiranje 1	9	5	08.01.2026	Vladimir Jecić	položio 91/100
Prosečna ocena: 7.33, Ukupno ostvareno ESPB: 14						

Slika 64: SK4, Položeni ispiti studenta

Iako se API pozivi razlikuju, na serverskoj strani u se koristi ista logika za filtriranje samo položenih ispita (SK4), trenutnih prijava (SK5 i SK6) i pretragu prijava radi ocenjivanja (SK7). Pozivaoc može proslediti query parametre koji se konvertuju na serverskoj strani u `ExamRegistrationFilters` objekat. Rezultate je moguće paginirati (page i pageSize), moguće je vratiti bilo koju kombinaciju samo neocenjenih prijava, uspešnih i neuspešnih prijava, trenutno aktivnih (prijava iz ispitnog roka koji je još uvek aktuelan), te prijave koje se odnose na određenog studenta, iz određenog predmeta ili ispitnog roka.

```
Request.php → GetExamRegistrationsRequest.php M GetExamRegistrationsRequest.php M ExamRegistrationServiceImpl.php M
app > estudent > domain > ports > input > model > ExamRegistrationFilters.php > PHP > ExamRegistrationFilters

1  <?php
2  namespace App\estudent\domain\ports\input\model;
3
4  /**
5   * @OA\Schema(
6   *     schema="ExamRegistrationFiltersDTO",
7   *     type="object",
8   *     @OA\Property(property="search-text", type="string", example=""),
9   *     @OA\Property(property="page", type="integer", example=1),
10   *     @OA\Property(property="page-size", type="integer", example=10),
11   *     @OA\Property(property="include-not-graded", type="boolean", example=false),
12   *     @OA\Property(property="include-failed", type="boolean", example=false),
13   *     @OA\Property(property="include-passed", type="boolean", example=false),
14   *     @OA\Property(property="include-current", type="boolean", example=false),
15   *     @OA\Property(property="student-id", type="integer", example=123),
16   *     @OA\Property(property="exam-period-id", type="integer", example=1),
17   *     @OA\Property(property="course-exam-id", type="integer", example=1)
18   * )
19  */
20  class ExamRegistrationFilters
21  {
22      4 references
23      public int $page;
24      4 references
25      public int $pageSize;
26      3 references
27      public ?string $searchText;
28      2 references
29      public bool $includeNotGraded;
30      2 references
31      public bool $includeFailed;
32      3 references
33      public bool $includePassed;
34      3 references
35      public bool $includeCurrent;
36      6 references
37      public ?int $studentId;
38      3 references
39      public ?int $examPeriodId;
40      3 references
41      public ?int $courseExamId;
42
43      0 references | 0 overrides
44      public function __construct(array $data)
45      {
46          $this->page = $data["page"] ?? 1;
47          $this->pageSize = $data["page-size"] ?? 10;
48          $this->searchText = $data["search-text"] ?? null;
49          $this->includeNotGraded = $data["include-not-graded"] ?? false;
50          $this->includeFailed = $data["include-failed"] ?? false;
51          $this->includePassed = $data["include-passed"] ?? false;
52          $this->includeCurrent = $data["include-current"] ?? false;
53          $this->studentId = $data["student-id"] ?? null;
54          $this->examPeriodId = $data["exam-period-id"] ?? null;
55          $this->courseExamId = $data["course-exam-id"] ?? null;
56      }
57  }
```

Slika 65: SK4, Mogući parametri paginacije pretrage prijava ispita

Prvo se querybuilderom pravi odgovarajući upit idući redom kroz sve filter parametre koji su setovani i postavljajući dodatne uslove. U ovom slučaju korišćenja postavljen je samo filter includePassed što znači da će biti vraćene samo prijave kod kojih je ocena između 6 i 10, a paginacija se ne koristi u ovom slučaju pa je pageSize setovan na max int vrednost jer se ne očekuje veliki broj ispita, dok se u slučaju svih postojećih prijava ikada koje profesor može da pregleda pageSize ograničava na 10, kako se ne bi svaki put dovlačilo potencijalno nekoliko hiljada prijava.

```
94  /**
95  * Retrieve only passed exam registrations for the authenticated student
96  *
97  * @OA\Get(
98  *   path="/exam-registrations/passed",
99  *   tags={"Common Routes"},
100  *   summary="Retrieve only passed exam registrations for the authenticated student",
101  *   security={
102  *     {"passport": {}}
103  *   },
104  *   @OA\Response(
105  *     response=200,
106  *     description="Paginated list of exam registrations"
107  *   )
108  * )
109  */
110 1 reference | 0 overrides
111  public function getPassedExamRegistrationsForStudent(GetExamRegistrationsRequest $request): JsonResponse
112  {
113      $examRegistrationFilters = $request->toDto();
114      $examRegistrationFilters->studentId = auth()->id();
115      $examRegistrationFilters->includePassed = true;
116      $examRegistrationFilters->page = 1;
117      $examRegistrationFilters->pageSize = PHP_INT_MAX;
118      $paginatedExamRegistrations = $this->examRegistrationService->getAllExamRegistrationsWithFilters($filters: $examRegistrationFilters);
119      return $this->createResponse(
120          data: ExamRegistrationResource::collection(resource: $paginatedExamRegistrations->items())
121      );
122  }
```

Slika 66: SK4, Priprema parametara i poziv metode getAllExamRegistrationsWithFilters

```

ExamRegistrationServiceImpl.php M X
app > estudent > domain > useCases > ExamRegistrationServiceImpl.php > PHP > ExamRegistrationServiceImpl > applyExamRegistrationFilters()
2 references | 0 implementations
16 class ExamRegistrationServiceImpl implements ExamRegistrationService
17 {
18     2 references
19     private readonly ExamPeriodServiceImpl $examPeriodService;
20     0 references | 0 overrides
21     public function __construct(ExamPeriodServiceImpl $examPeriodService )
22     {
23         $this->examPeriodService = $examPeriodService;
24     }
25     4 references | 0 overrides
26     public function getAllExamRegistrationsWithFilters(ExamRegistrationFilters $examRegistrationFilters): LengthAwarePaginator
27     {
28         $query = ExamRegistration::with(relations: ['student', 'courseExam', 'signedBy']);
29         $this->applyExamRegistrationFilters(query: $query, examRegistrationFilters: $examRegistrationFilters);
30         $query->orderBy(column: 'updated_at', direction: 'desc');
31         return $query->paginate(perPage: $examRegistrationFilters->pageSize, page: $examRegistrationFilters->page);
32     }
33     1 reference | 0 overrides
34     public function createExamRegistration(SubmitExamRegistrationDTO $dto): ExamRegistration...
35     {
36     }
37     1 reference | 0 overrides
38     public function updateExamRegistration(int $id, UpdateExamRegistrationDTO $dto): ExamRegistration...
39     {
40     }
41     1 reference | 0 overrides
42     public function deleteExamRegistration(int $id): bool...
43     {
44     }
45     /**
46      * Applies filters to the ExamRegistration query based on the given filters object.
47      */
48     1 reference
49     private function applyExamRegistrationFilters($query, ExamRegistrationFilters $examRegistrationFilters): void
50     {
51         if ($examRegistrationFilters->studentId) {
52             $query->where('student_id', $examRegistrationFilters->studentId);
53         }
54         if ($examRegistrationFilters->courseExamId) {
55             $query->where('course_exam_id', $examRegistrationFilters->courseExamId);
56         }
57         if ($examRegistrationFilters->examPeriodId) {
58             $query->whereHas('courseExam', function($q) use ($examRegistrationFilters): void {
59                 $q->where('exam_period_id', $examRegistrationFilters->examPeriodId);
60             });
61         }
62         if ($examRegistrationFilters->includeCurrent) {
63             $currentDate = Carbon::now();
64             $query->whereHas('courseExam.examPeriod', function($q) use ($currentDate): void {
65                 $q->where('dateEnd', '>', $currentDate);
66             });
67         }
68         $marks = [];
69         if ($examRegistrationFilters->includePassed) {
70             $marks = array_merge($marks, [6,7,8,9,10]);
71         }
72         if ($examRegistrationFilters->includeFailed) {
73             $marks = array_merge($marks, [5]);
74         }
75         $query->where(function($q) use ($marks, $examRegistrationFilters): void {
76             if ($examRegistrationFilters->includeNotGraded) {
77                 $q->whereIn('mark', $marks)
78                     ->orWhereNull('mark');
79             } else {
80                 $q->whereIn('mark', $marks);
81             }
82         });
83         if ($examRegistrationFilters->searchText) {
84             $searchText = $examRegistrationFilters->searchText;
85             $query->where(function($q) use ($searchText): void {
86                 $q->whereHas('student', function($studentQ) use ($searchText): void {
87                     $studentQ->where('name', 'like', "%$searchText%")
88                         ->orWhere('indexNum', 'like', "%$searchText%");
89                 })
90                 ->orWhereHas('courseExam.courseInstance.course', function($courseQ) use ($searchText): void {
91                     $courseQ->where('name', 'like', "%$searchText%");
92                 });
93             });
94         }
95     }
96 }

```

Slika 67: SK4, Implementacija getAllExamRegistrationsWithFilters metode

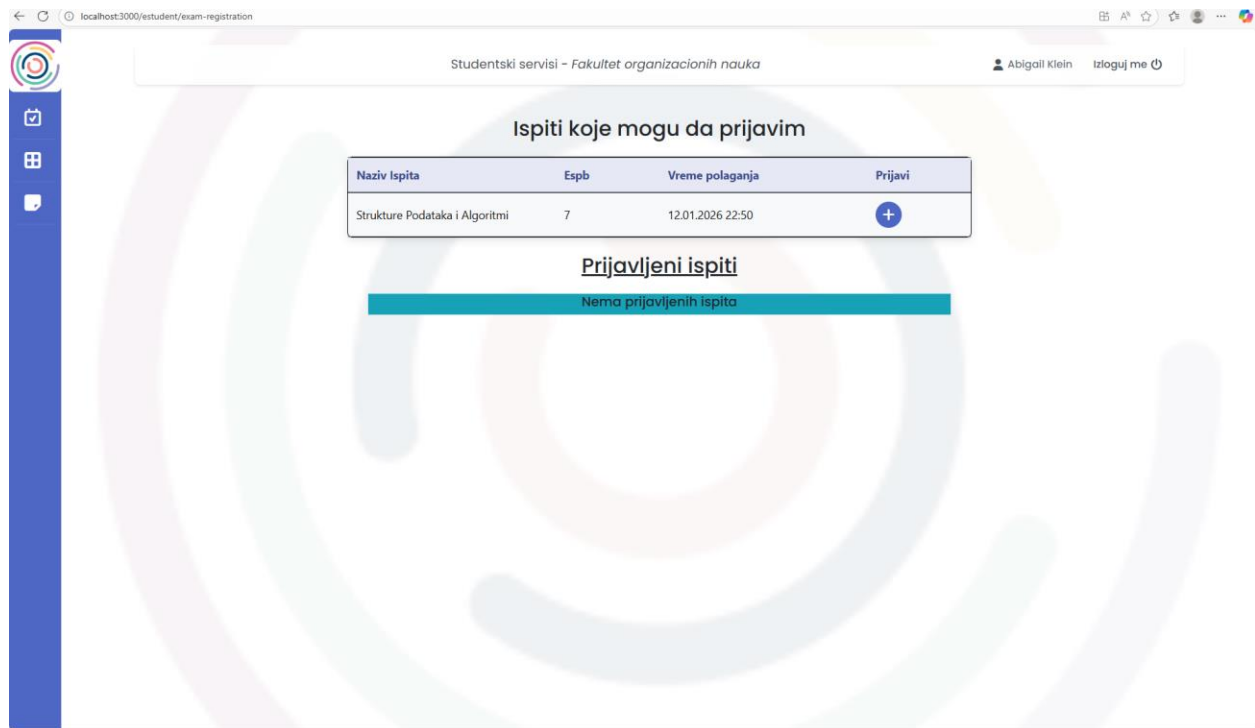
4.2.5 Uputstvo SK5: Prijavljivanje polaganja ispita

Uslov: Student je ulogovan sa svojim nalogom.

Opis: Student klikom na navigaciju “Prijava ispita” ([Slika 63: SK4, Navigacioni meni - student](#)) učitavaju se svi prijavljeni i neprijavljeni ispiti studenta. Student klikom na dugme prijavi poziva sistem da zabeleži prijavu ispita. Za ovaj primer to će biti ispit Strukture Podataka i Algoritmi.

```
121 1 reference | 0 overrides
122 public function getRegisterableCourseExams(): Collection
123 {
124     $student = auth()->user();
125     $activeExamPeriods = $this->examPeriodService->active();
126     if ($activeExamPeriods->isEmpty()) {
127         return collect();
128     }
129     $allRemainingCourseExams = collect();
130     foreach ($activeExamPeriods as $examPeriod) {
131         $remaining = $this->getRemainingCourseExams(examPeriodId: $examPeriod->id);
132         $allRemainingCourseExams = $allRemainingCourseExams->merge(items: $remaining);
133     }
134
135     $examRegistrationFilters = new ExamRegistrationFilters(data: []);
136     $examRegistrationFilters->studentId = $student->id;
137     $examRegistrationFilters->includeCurrent = true;
138     $examRegistrationFilters->includePassed = true;
139     $examRegistrationFilters->includeFailed = true;
140     $examRegistrationFilters->includeNotGraded = true;
141     $examRegistrationFilters->page = 1;
142     $examRegistrationFilters->pageSize = PHP_INT_MAX;
143
144     $paginatedExamRegistrations = $this->examRegistrationService
145         ->getAllExamRegistrationsWithFilters(filters: $examRegistrationFilters);
146     $currentExamRegistrationsIds = collect(value: $paginatedExamRegistrations->items())
147         ->pluck(value: 'course_exam_id')
148         ->toArray();
149     $registerableCourseExams = $allRemainingCourseExams->reject(
150         callback: fn (TValue $courseExam): bool => in_array(needle: $courseExam->id, haystack: $currentExamRegistrationsIds)
151     );
152
153     return $registerableCourseExams;
154 }
155 }
```

Slika 68: SK5, Učitavanje ispita koje student može da polaže



Slika 69: SK5, Prijava ispita

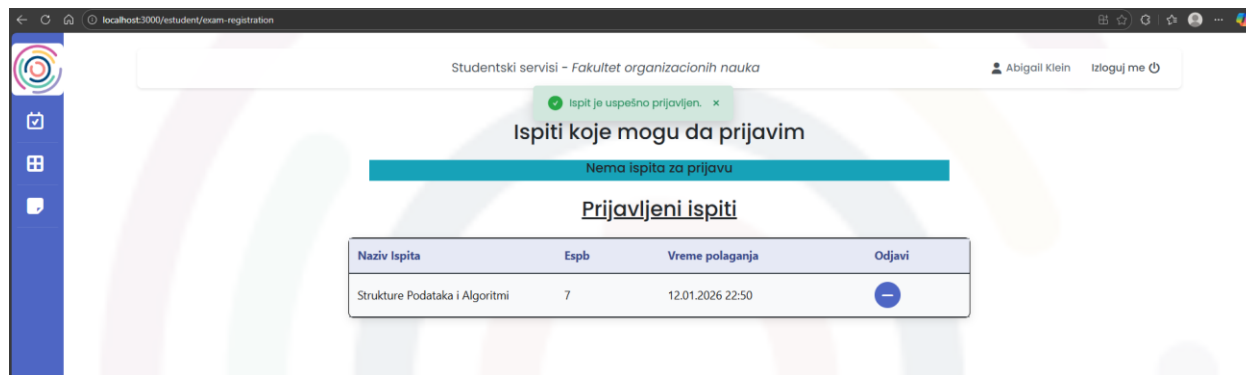
U slučaju da je period za prijavu istekao dugme za Prijavu će biti onemogućeno i hoverovanjem nad dugmetom će se prikazati poruka “Period za prijavu je istekao”. Klikom na dugme se okida asinhroni post poziv na rutu exam-registrations gde se nakon provere da je registracija ispita još uvek u toku pamti nova prijava za tog studenta.

```

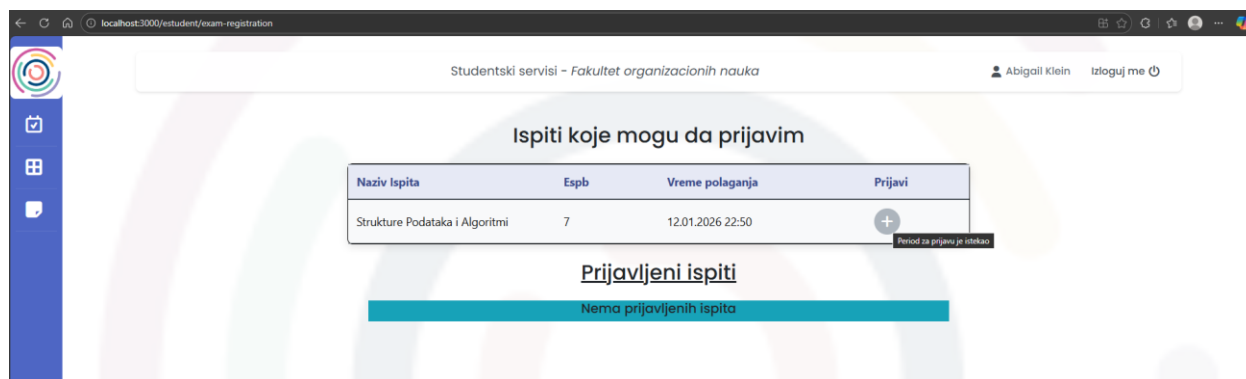
1 reference | 0 overrides
32 public function createExamRegistration(SubmitExamRegistrationDTO $dto): ExamRegistration
33 {
34     $courseExam = CourseExam::with(relations: 'examPeriod')->find(id: $dto->courseExamId);
35     $registerableExamPeriods = $this->examPeriodService->registerable();
36     if (!$registerableExamPeriods->contains(key: $courseExam->examPeriod)) {
37         throw new RegistrationNotInProgressException(message: 'Exam registration is not in progress for this exam period.');
```

Slika 70: SK5, Implementacija metode za kreiranje nove prijave

Nakon što je prijava uspešno zapamćena ona se pojavljuje u tabeli ispod (nakon posta se radi novi fetch svih trenutnih prijava za tog studenta, filter include-current, pogledati objašnjenje za [SK4](#))



Slika 71: SK5, Poruka ispit je uspešno prijavljen



Slika 72: SK5 - Period za prijavu je istekao

4.2.6 Uputstvo SK6: Odjavljivanje polaganja ispita

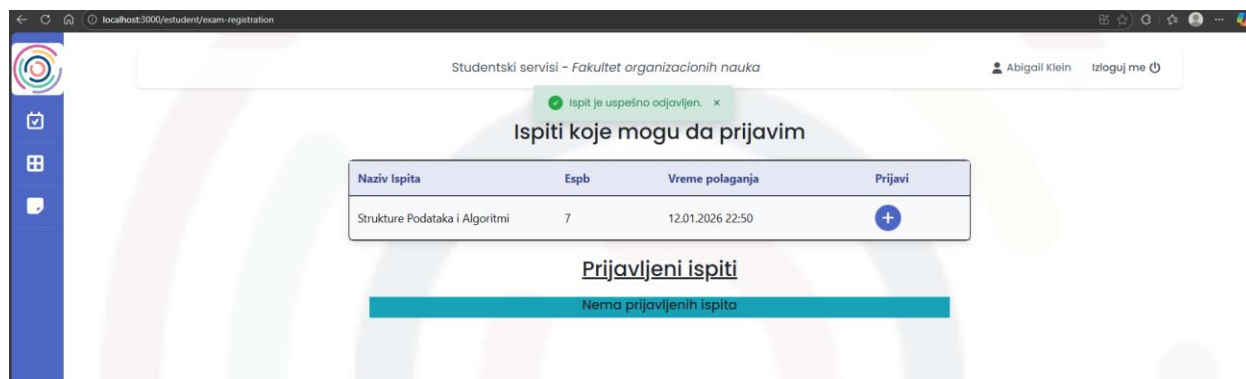
Uslov: Student je ulogovan sa svojim nalogom.

Opis: Student klikom na navigaciju "Prijava ispita" ([Slika 66: SK4, Navigacioni meni - student](#)) učitavaju se svi prijavljeni i neprijavljeni ispiti studenta. Klikom na dugme odjavi, student upućuje zahtev da ispit bude odjavljen.

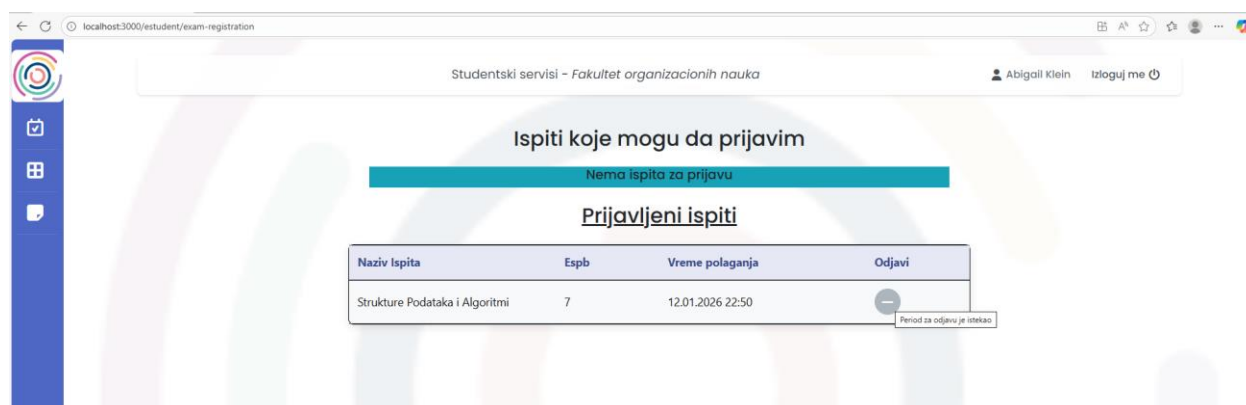
```
72 1 reference | 0 overrides
73 public function deleteExamRegistration(int $id): bool
74 {
75     $user = auth()->user();
76     $registration = ExamRegistration::find(id: $id);
77     if ($user->role === 'student' && $registration->student_id != $user->id) {
78         throw new UnauthorizedOperationException(message: 'Forbidden: You can only delete your own exam registrations.');
```

Slika 73: SK6, implementacija odjave ispita

Ako je odjavljivanje uspešno, posle nekoliko sekundi pregled će se osvežiti i ispit će nestati iz tabele Prijavljeni ispiti I pojaviti se u tabeli ispiti koje mogu da prijavim, u slučaju da je rok počeo, odjava ispita će biti onemogućena.



Slika 74: SK6, poruka, Ispit je uspešno objavljen



Slika 75: SK6, Period za prijavu je istekao

4.2.7 Uputstvo SK7: Ocenjivanje prijave studenta

Uslov: Korisnik je ulogovan sa administratorskim nalogom.

Opis: Kada se uloguje, za razliku od studenta njemu se po defaultu prikazuje interfejs za rad sa prijavama polaganja.

Broj indexa	Ime	Naziv Ispita	Rok	Prisustvovao	Ocena	Akcije
2025/7000	Abigail Klein	Strukture Podataka i Algoritmi	januar-2026	Da	1	
2025/2296	Winston Doyle	Matematika2	januar-2026	Da	6	
2025/2296	Winston Doyle	Statistika	januar-2026	Da	5	
2025/2296	Winston Doyle	Verovatnoca	januar-2026	Da	7	
2025/2296	Winston Doyle	Programiranje 1	januar-2026	Da	9	
2025/0800	Prof. Keanu Russel I	Matematika2	novembarsko-decembarski-apsolventski-2025	Da	8	
2025/9928	Delphia Bernhard	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	8	
2025/0800	Prof. Keanu Russel I	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	8	
2025/3037	Kirsten Bashirian	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	6	
2025/1308	Dante Mayer	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	5	

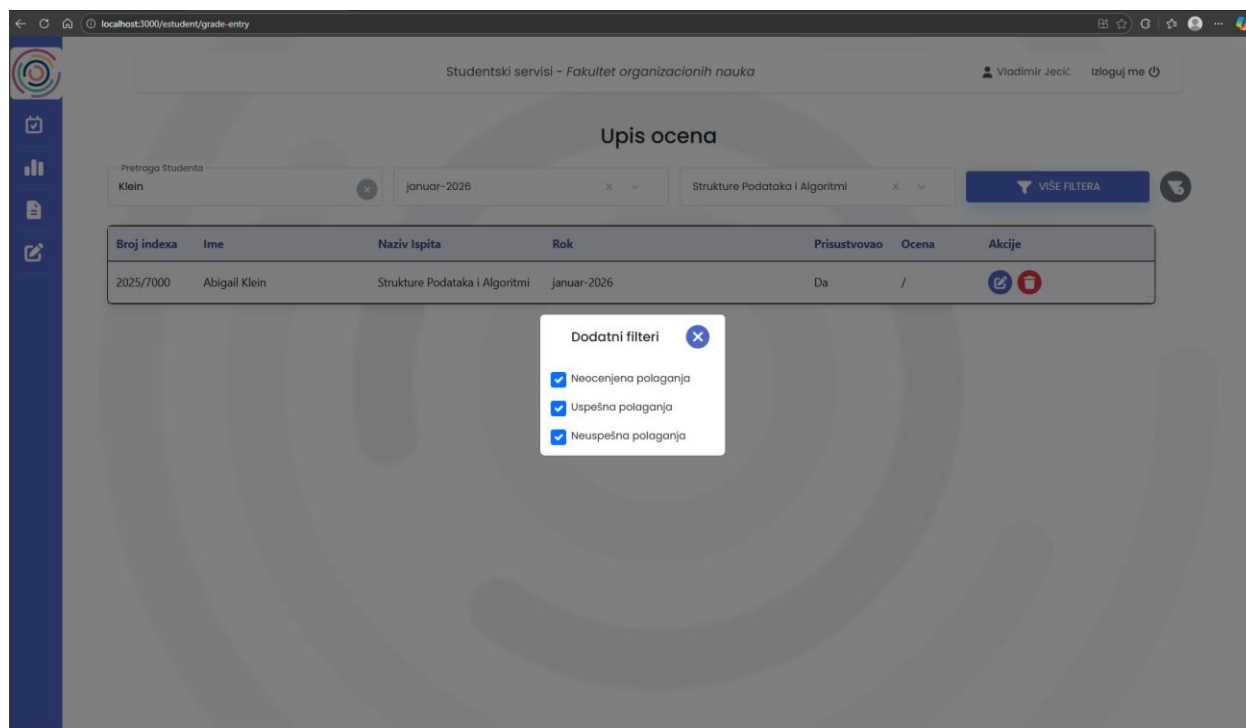
Slika 76: SK7: Upis ocena

Na formi vidimo prijavu studenta Abigail Klein, br. Indexa 2025/7000 koje je kreirana prilikom demonstriranja prijave ispita u SK5. Prijave je moguće filtrirati po student (full text search – po delu imena ili broju indexa), po roku u kojem je prijava napravljena i po ispitu u izabranom roku.

Broj indexa	Ime	Naziv Ispita	Rok	Akcije
2025/7000	Abigail Klein	Strukture Podataka i Algoritmi	januar-2026	

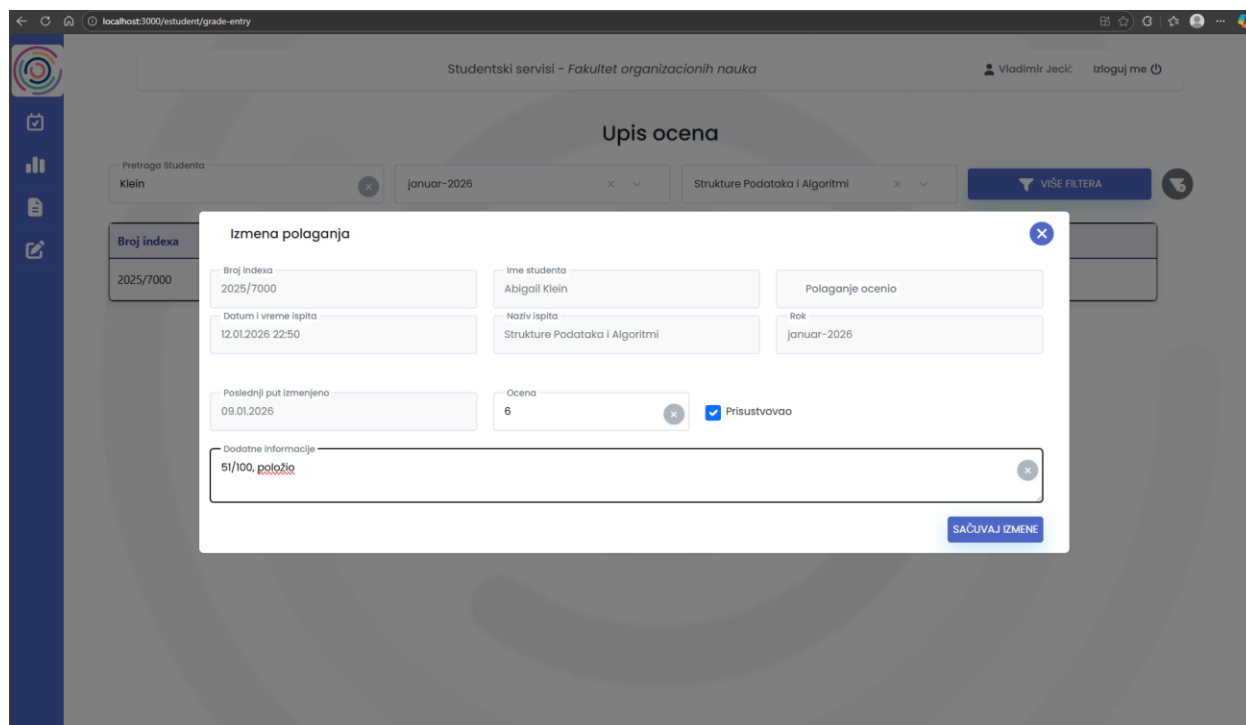
Slika 77: SK7, Pretraga po studentu i/ili ispitno roku i ispitu

Klikom na više filtera se otvara dijalog sa dodatnim filterima pretrage, svih 6 filtera je moguće očistiti odjednom klikom na sivo dugme pored više filtera koje postaje vidljivo u slučaju da je bilo koji od filtera aktivan:

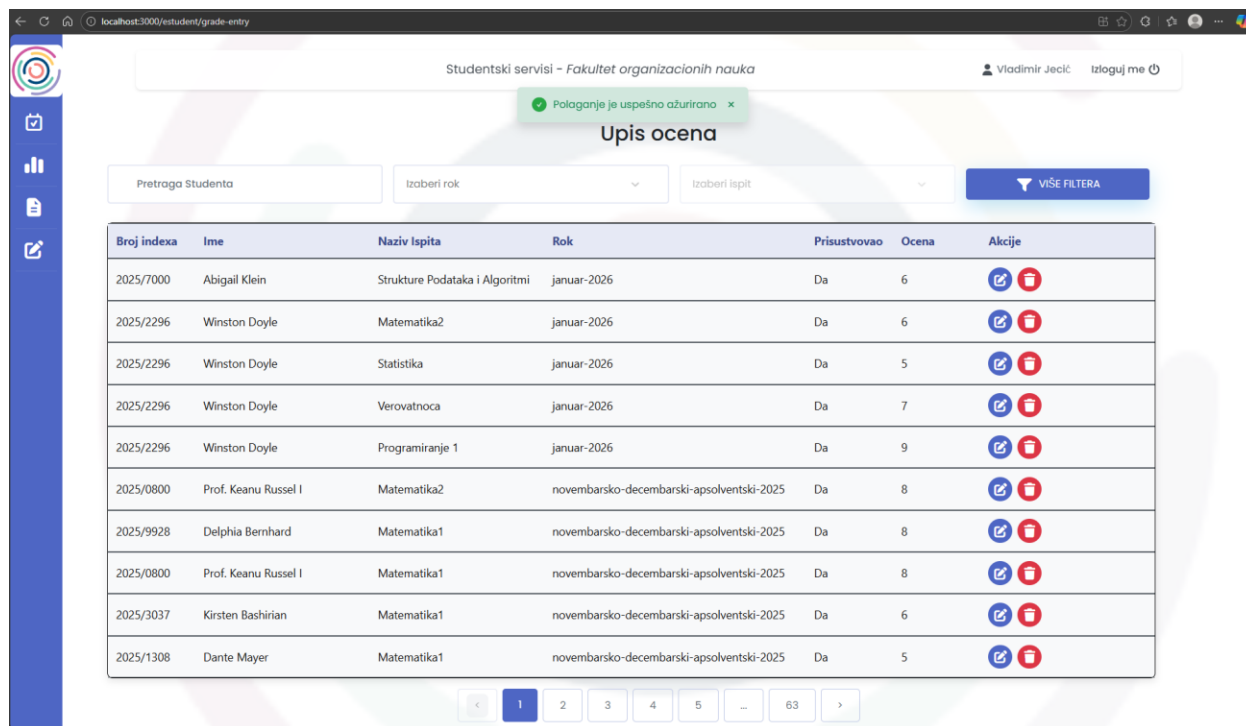


Slika 78: SK7, Dodatni filteri

Nastanak klikom na olovčicu otvara dijalog za izmenu prijave, gde može da upiše ocenu, prisustvo i ostavi komentar(opciono) nakon čega klikom na sačuvaj izmene poziva sistem da sačuva ocenjenu prijavu. Ako je prijava uspešno izmenjena prikazaće se poruka: "Polaganje je uspešno ažurirano".



Slika 79: SK7: Unos ocene



Slika 80: SK7, poruka, Polaganje je uspešno ažurirano

```

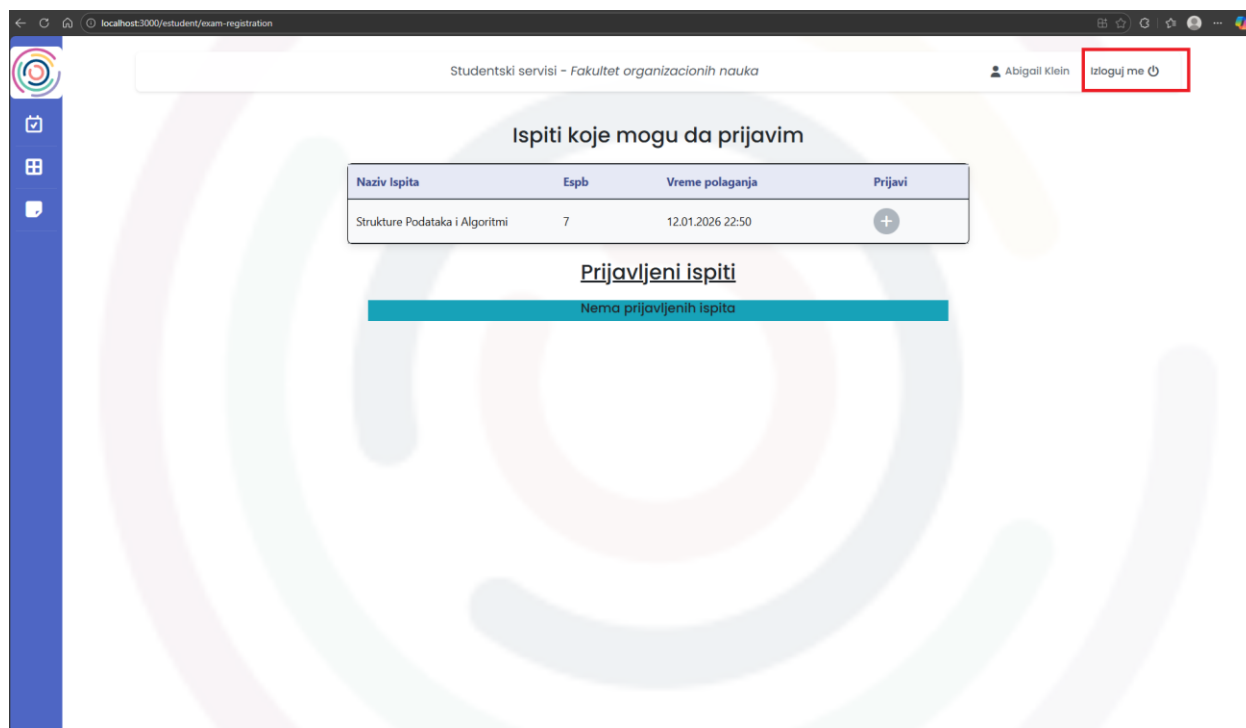
56 1 reference | 0 overrides
57 public function updateExamRegistration(int $id, UpdateExamRegistrationDTO $dto): ExamRegistration
58 {
59     $examRegistration = ExamRegistration::find(id: $id);
60     if ($dto->mark == null || $dto->mark < 5 || $dto->mark > 10) {
61         throw new BadRequestException(message: 'Invalid mark value.');
```

Slika 81: SK7, implementacija updateExamRegistration, prvog unosa ili izmena ocene

4.2.8 Uputstvo SK8: Odjavljivanje korisnika

Uslov: Korisnik je ulogovan sa svojim nalogom.

Opis: Korisnik klikće na dugme izloguj me pored imena. Nakon čega će korisnik biti vraćen na početnu login stranu kao na slici. Korisniku se briše OAuth jwt token(JSON Web Token) i da bi dobio novi mora da se ponovo uloguje.



Slika 82: SK8: Dugme izloguj me

4.2.9 Uputstvo SK9: Pregled izveštaja polaganja predmeta u ispitnom roku

Uslov: Korisnik je ulogovan sa svojim nalogom.

Opis: Admin Korisnik u navigacionom meniju bira opciju "Izveštaj polaganja ispita". Izveštaj je prvo moguće filtrirati po nazivu ispita ili roka kao i datumu od i datumu do gde ispit upada.

Studentski servisi - Fakultet organizacionih nauka

Vladimir Jecić Izloguj me

Upis ocena

Pretraga Studenta Izaberi rok Izaberi ispit VIŠE FILTERA

Broj indeksa	Ime	Naziv ispita	Rok	Prisustvovao	Ocena	Akcije
2025/7000	Abigail Klein	Strukture Podataka i Algoritmi	januar-2026	Da	6	 
2025/2296	Winston Doyle	Matematika2	januar-2026	Da	6	 
2025/2296	Winston Doyle	Statistika	januar-2026	Da	5	 
2025/2296	Winston Doyle	Verovatnoca	januar-2026	Da	7	 
2025/2296	Winston Doyle	Programiranje 1	januar-2026	Da	9	 
2025/0800	Prof. Keanu Russel I	Matematika2	novembarsko-decembarski-apsolventski-2025	Da	8	 
2025/9928	Delphia Bernhard	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	8	 
2025/0800	Prof. Keanu Russel I	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	8	 
2025/3037	Kirsten Bashirian	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	6	 
2025/1308	Dante Mayer	Matematika1	novembarsko-decembarski-apsolventski-2025	Da	5	 

< 1 2 3 4 5 ... 63 >

Slika 83: SK9, Navigacioni meni – admin

Studentski servisi - Fakultet organizacionih nauka

Vladimir Jecić Izloguj me

Izveštaj polaganja

Naziv ispita ili roka Datum od Datum do Oceni filtere

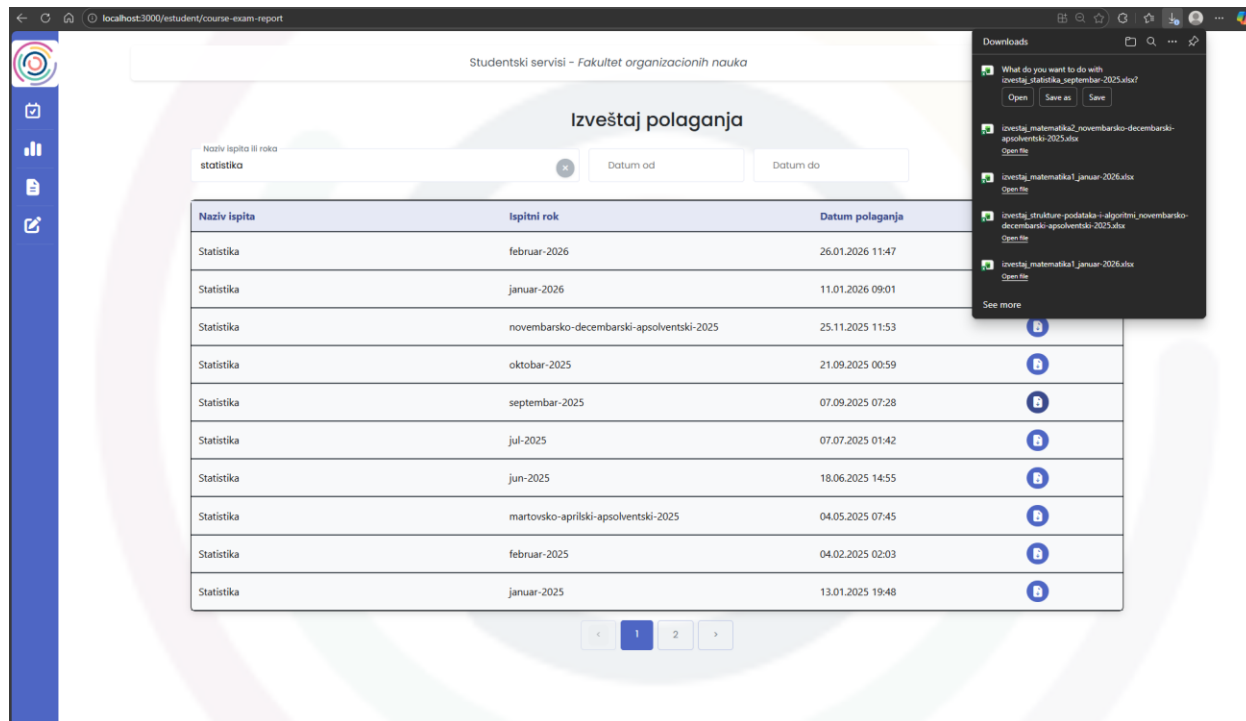
januar

Naziv ispita	Ispitni rok	Datum polaganja	Izveštaj
Matematika1	januar-2026	22.01.2026 10:57	
Programiranje 1	januar-2026	21.01.2026 19:44	
Verovatnoca	januar-2026	15.01.2026 14:34	
Programiranje 2	januar-2026	13.01.2026 20:27	
Strukture Podataka i Algoritmi	januar-2026	12.01.2026 22:50	
Matematika2	januar-2026	12.01.2026 10:26	
Statistika	januar-2026	11.01.2026 09:01	
Matematika1	januar-2025	19.01.2025 10:42	
Programiranje 2	januar-2025	19.01.2025 05:02	
Strukture Podataka i Algoritmi	januar-2025	17.01.2025 22:57	

< 1 2 >

Slika 84: SK9, Pretraga željenog izveštaja

Kada se nadje odgovarajući ispit, excel izveštaj se skida klikom na download ikonicu u tom redu. U izveštaju se može videti spisak studenata koji su bili prijavljeni na taj ispit kao i da li su izašli na ispit i koju ocenu su dobili kao i procentualna izlaznost i prosečna ocena.



Studentski servisi - Fakultet organizacionih nauka

Izveštaj polaganja

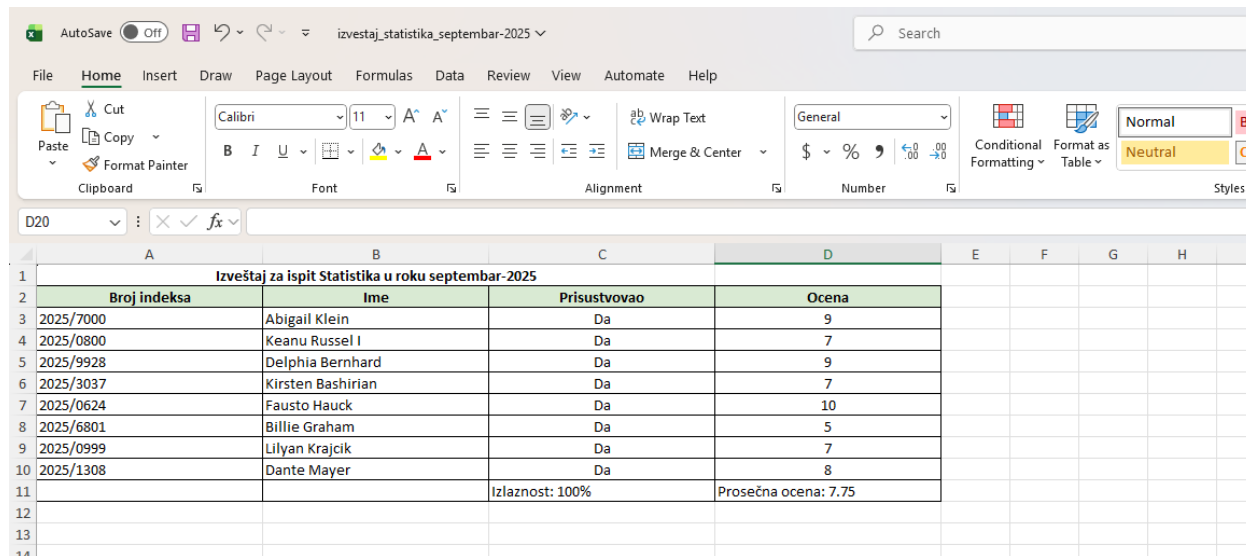
Naziv ispita ili roka: statistika

Datum od: Datum do:

Naziv ispita	Ispitni rok	Datum polaganja
Statistika	februar-2026	26.01.2026 11:47
Statistika	januar-2026	11.01.2026 09:01
Statistika	novembarsko-decembarski-apsolventski-2025	25.11.2025 11:53
Statistika	oktobar-2025	21.09.2025 00:59
Statistika	septembar-2025	07.09.2025 07:28
Statistika	jul-2025	07.07.2025 01:42
Statistika	jun-2025	18.06.2025 14:55
Statistika	martovsko-apriliški-apsolventski-2025	04.05.2025 07:45
Statistika	februar-2025	04.02.2025 02:03
Statistika	januar-2025	13.01.2025 19:48

Download dialog for 'izvestaj_statistika_septembar-2025.xlsx' is open.

Slika 85: SK9, Izveštaj ispita Statistika u roku septembar-2025



AutoSave Off | izvestaj_statistika_septembar-2025

File Home Insert Draw Page Layout Formulas Data Review View Automate Help

Clipboard: Paste, Copy, Format Painter

Font: Calibri, 11, Bold, Italic, Underline, Color, Background Color

Alignment: Left, Center, Right, Wrap Text, Merge & Center

Number: General, Percentage, Currency, Date, Time, Text, Fraction, Decimals

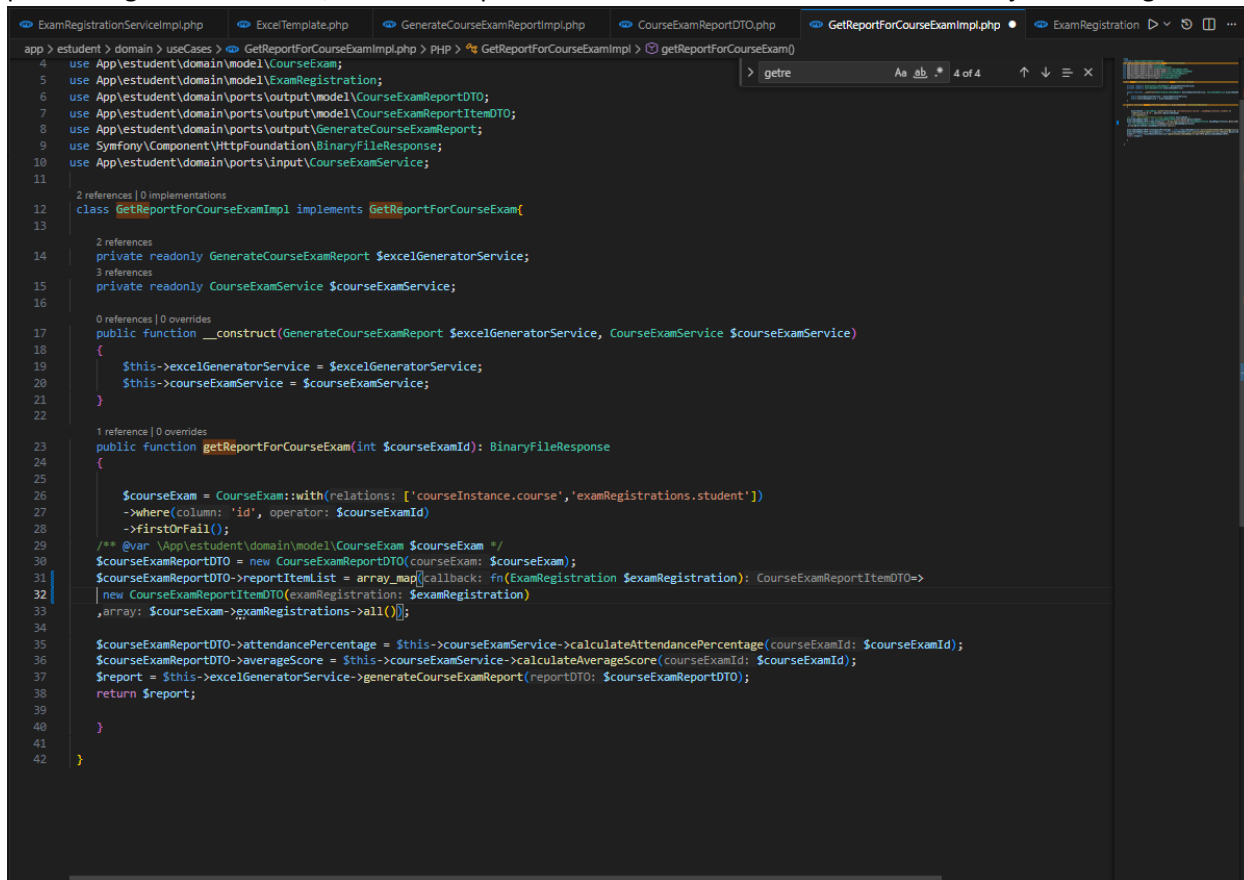
Styles: Normal, Neutral, Conditional Formatting, Format as Table

Izveštaj za ispit Statistika u roku septembar-2025			
Broj indeksa	Ime	Prisustvovao	Ocena
2025/7000	Abigail Klein	Da	9
2025/0800	Keanu Russel I	Da	7
2025/9928	Delphia Bernhard	Da	9
2025/3037	Kirsten Bashirian	Da	7
2025/0624	Fausto Hauck	Da	10
2025/6801	Billie Graham	Da	5
2025/0999	Lilyan Krajcik	Da	7
2025/1308	Dante Mayer	Da	8
		Izlaznost: 100%	Prosečna ocena: 7.75

Slika 86: SK9, Excel izveštaj

Implementacija ovog izveštaja na serverskoj strani izdvaja ulogu samog popunjavanja excela u odvojenu klasu van domena i poziva generisanje excela uz prosledjeno ime XML templejt-a i prethodno kalkulisane

podatke u transportnom DTO-u. Za samo generisanje excel je korišćena Maatwebsite php biblioteka. Front prima odgovor kao blob, dinamički pravi html element da bi skinuo izveštaj i zatim ga briše.



```
app > estudent > domain > useCases > GetReportForCourseExamImpl.php > PHP > % GetReportForCourseExamImpl > getReportForCourseExam()
4 use App\estudent\domain\model\CourseExam;
5 use App\estudent\domain\model\ExamRegistration;
6 use App\estudent\domain\ports\output\model\CourseExamReportDTO;
7 use App\estudent\domain\ports\output\model\CourseExamReportItemDTO;
8 use App\estudent\domain\ports\output\GenerateCourseExamReport;
9 use Symfony\Component\HttpFoundation\BinaryFileResponse;
10 use App\estudent\domain\ports\input\CourseExamService;
11
12 2 references | 0 implementations
13 class GetReportForCourseExamImpl implements GetReportForCourseExam{
14
15     2 references
16     private readonly GenerateCourseExamReport $excelGeneratorService;
17     3 references
18     private readonly CourseExamService $courseExamService;
19
20     0 references | 0 overrides
21     public function __construct(GenerateCourseExamReport $excelGeneratorService, CourseExamService $courseExamService)
22     {
23         $this->excelGeneratorService = $excelGeneratorService;
24         $this->courseExamService = $courseExamService;
25     }
26
27     1 reference | 0 overrides
28     public function getReportForCourseExam(int $courseExamId): BinaryFileResponse
29     {
30         $courseExam = CourseExam::with(['courseInstance.course', 'examRegistrations.student'])
31         ->where('column: 'id', operator: $courseExamId)
32         ->firstOrFail();
33
34         /** @var \App\estudent\domain\model\CourseExam $courseExam */
35         $courseExamReportDTO = new CourseExamReportDTO($courseExam);
36         $courseExamReportDTO->reportItemList = array_map(fn(ExamRegistration $examRegistration): CourseExamReportItemDTO=>
37         | new CourseExamReportItemDTO($examRegistration->examRegistration)
38         ,array: $courseExam->examRegistrations->all());
39
40         $courseExamReportDTO->attendancePercentage = $this->courseExamService->calculateAttendancePercentage($courseExamId);
41         $courseExamReportDTO->averageScore = $this->courseExamService->calculateAverageScore($courseExamId);
42         $report = $this->excelGeneratorService->generateCourseExamReport($reportDTO: $courseExamReportDTO);
43         return $report;
44     }
45 }
```

Slika 87: Pripremanje CourseExamReportDTO i poziv generateCourseExamReport


```

1  <?php
2
3  namespace App\estudent\adapters\reports;
4
5  use App\estudent\domain\ports\output\GenerateCourseExamReport;
6  use App\estudent\domain\ports\output\model\CourseExamReportDTO;
7  use App\estudent\adapters\reports\model\ExcelTemplate;
8  use Illuminate\Support\Str;
9  use Symfony\Component\HttpFoundation\BinaryFileResponse;
10
11 2 references | 0 implementations
12 class GenerateCourseExamReportImpl implements GenerateCourseExamReport
13 {
14     1 reference | 0 overrides
15     public function generateCourseExamReport(CourseExamReportDTO $reportDTO): BinaryFileResponse
16     {
17         $fileName = 'izveštaj_' . Str::slug(title: $reportDTO->courseExamName) . '_' . Str::slug(title: $reportDTO->examPeriodName) . '.xlsx';
18         $templatePath = config(key: 'reports.course_exam_template');
19         $excelTemplate = (new ExcelTemplate(pathToTemplate: $templatePath))->withFooter();
20         $model = [
21             'reportDTO' => $reportDTO,
22         ];
23         $download = $excelTemplate->process(model: $model, fileName: $fileName, columnCount: 4);
24         return $download;
25     }
26 }

```

Slika 88: SK09, kreiranje izveštaja na osnovu DTO-a i XML templejta

```

1  <table>
2  <thead>
3      <tr style="background-color: #e0e0ff;">
4          <th colspan="3" style="font-weight: bold; text-align: center;">
5              Izveštaj za ispit {{ $reportDTO->courseExamName }} u roku {{ $reportDTO->examPeriodName }}
6          </th>
7      </tr>
8      <tr style="background-color: #d9ead3; font-weight: bold; border: 1px solid #000;">
9          <th style="border: 1px solid #000; width: 25%;>Broj indeksa</th>
10         <th style="border: 1px solid #000; width: 40%;>Ime</th>
11         <th style="border: 1px solid #000; width: 15%;>Pristupovao</th>
12         <th style="border: 1px solid #000; width: 15%;>Ocena</th>
13     </tr>
14 </thead>
15 <tbody>
16     @foreach($reportDTO->reportItemList as $reportItemDTO)
17         <tr>
18             <td style="border: 1px solid #000;">{{ $reportItemDTO->studentIndexNum }}</td>
19             <td style="border: 1px solid #000;">{{ $reportItemDTO->studentName }}</td>
20             <td style="border: 1px solid #000; text-align: center;">{{ $reportItemDTO->hasAttended ? 'Da' : 'Ne' }}</td>
21             <td style="border: 1px solid #000; text-align: center;">{{ $reportItemDTO->mark }}</td>
22         </tr>
23     @endforeach
24 </tbody>
25 <tfoot>
26     <tr style="background-color: #f9f9f9; font-weight: bold;">
27         <td style="border: 1px solid #000;"></td>
28         <td style="border: 1px solid #000;"></td>
29         <td style="border: 1px solid #000;">Izlaznost: {{ $reportDTO->attendancePercentage }}%</td>
30         <td style="border: 1px solid #000;">Prosečna ocena: {{ $reportDTO->averageScore }}</td>
31     </tr>
32 </tfoot>
33 </table>
34

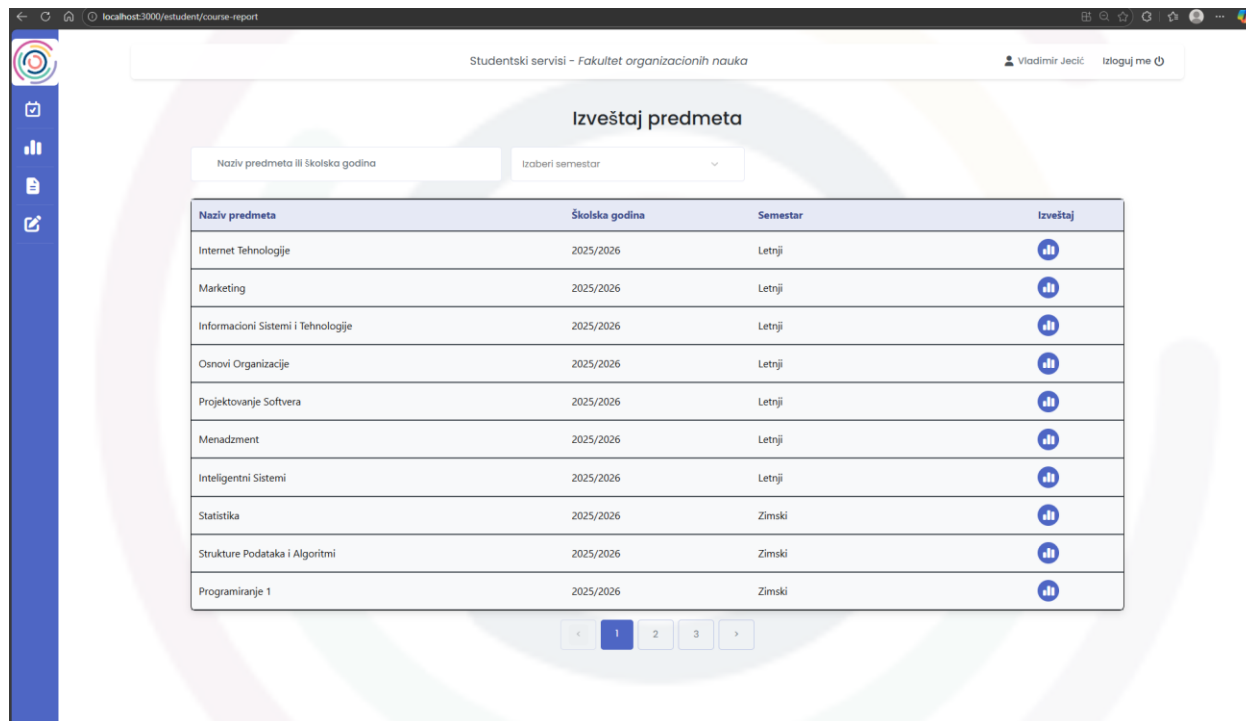
```

Slika 89: SK09, XML templejt za excel

4.2.10 Uputstvo SK10: Pregled izveštaja predmeta u školskoj godini

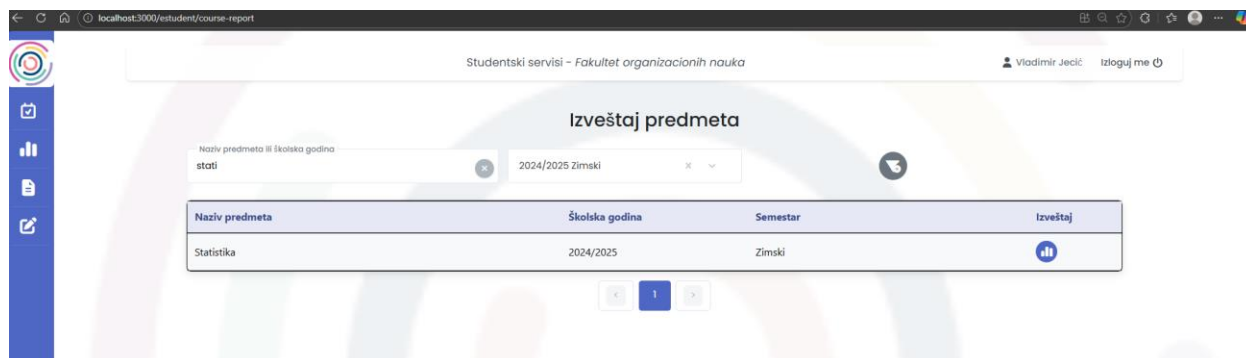
Uslov: Korisnik je ulogovan sa svojim nalogom.

Opis: Admin Korisnik u navigacionom meniju([Slika 83: SK9, Navigacioni meni – admin](#)) bira opciju “Izveštaj predmeta”



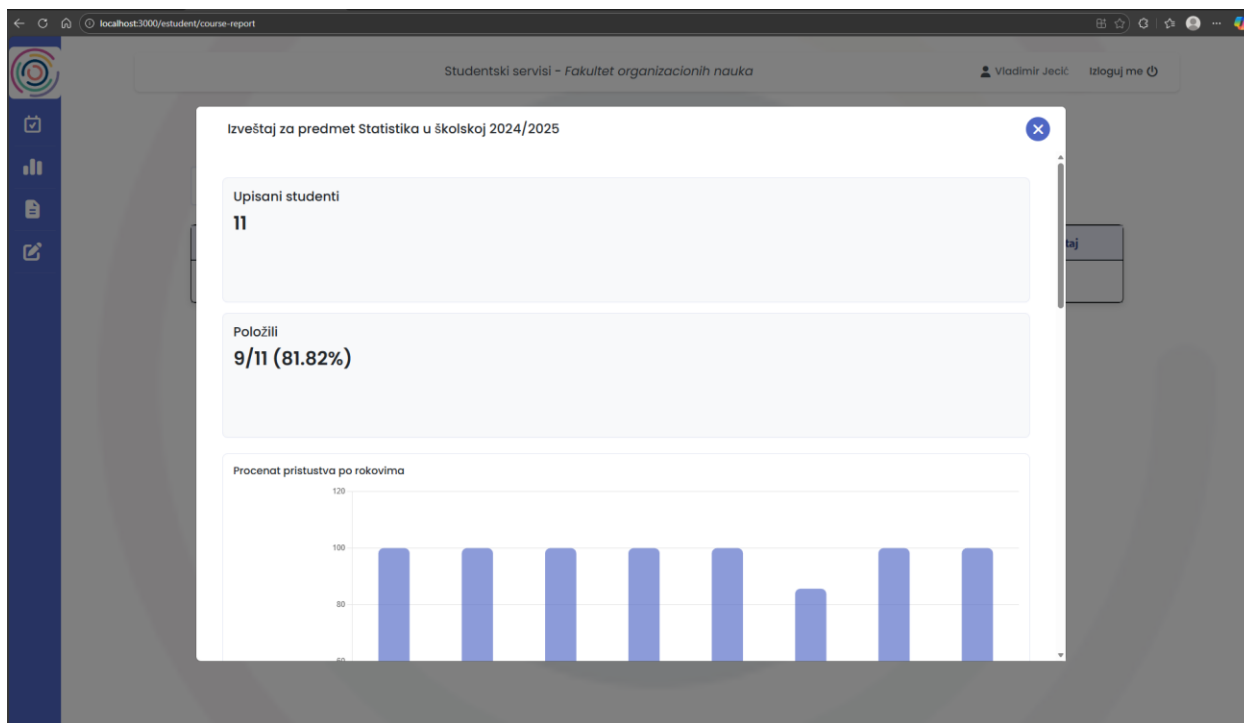
Slika 90: SK10, Forma za pregled Izveštaj predmeta

Korisnik može da pretražuje izveštaj po nazivu predmeta ili školskoj godini kao i semestru. Klikom na dugme sa bar chartom otvara se dijalog sa izveštajem tog predmeta.

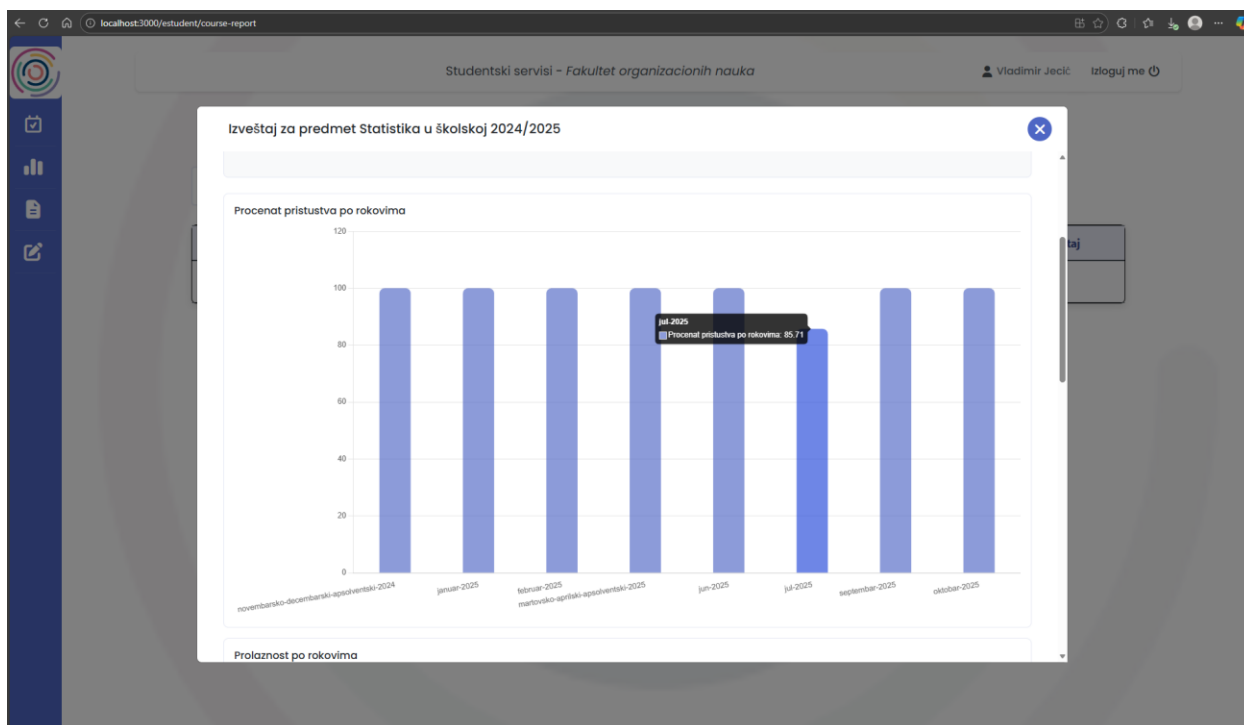


Slika 91: SK10, Pretraga predmeta

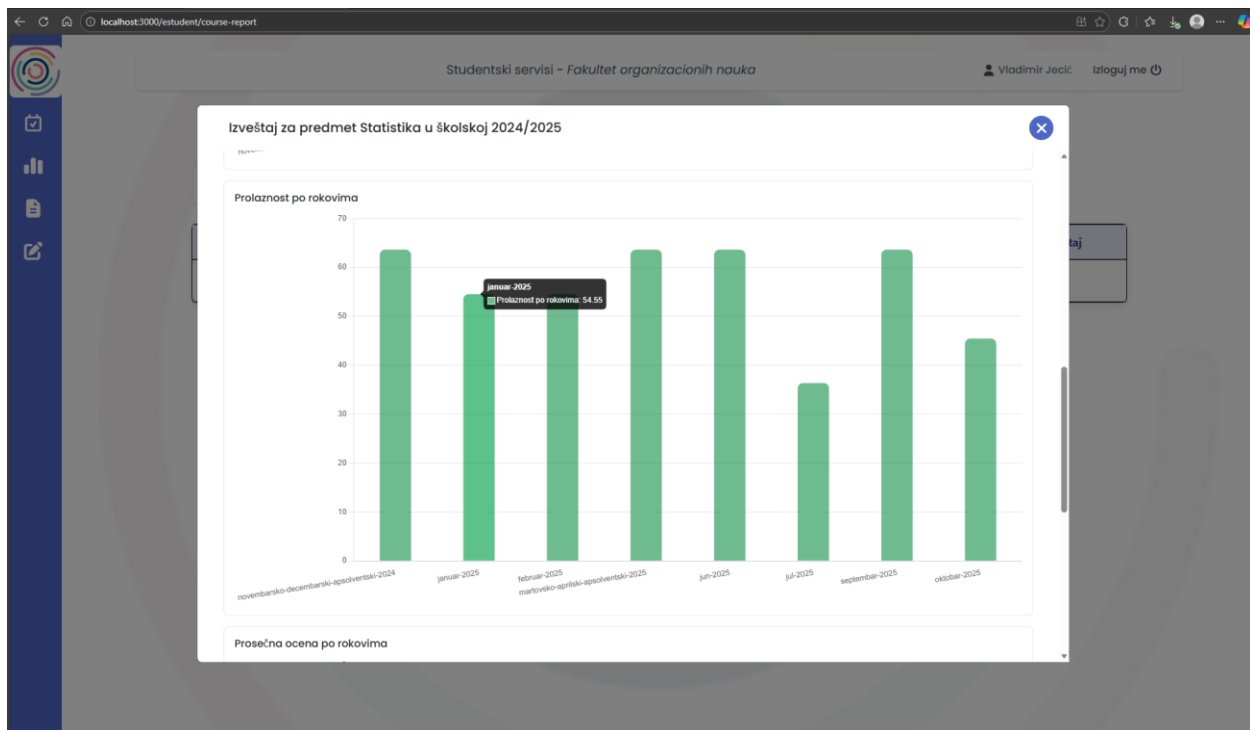
Podaci korišćeni za naredne dijagrame su napravljeni pomoću php faker-a i naredne slike su samo vizuelna reprezentacija podataka. Izveštaj je moguće pregledati kroz pretraživač, podaci se dovlače sa serverske strane dok je biblioteka na klijentskoj strani, chart.js, zaslužna za prikaz. Medjutim, zahvaljujuci jsPDF i html2canvas npm bibliotekama, html izveštaje je moguće sačuvati kao sliku i skinuti kao pdf zarad lakšeg pregleda i prenosa. Izveštaj se sastoji od 3 dijagrama procenat prisustva po rokovima, prolaznost po rokovima i prosečna ocena po rokovima.



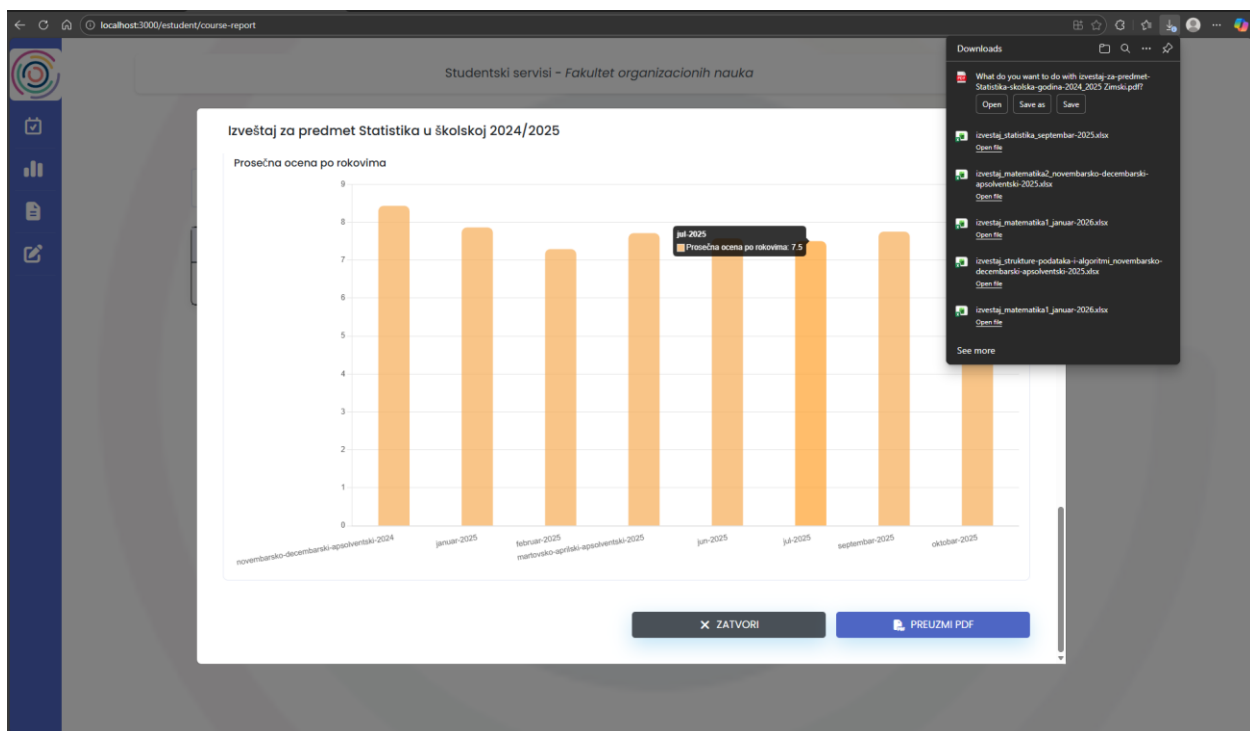
Slika 92: SK10, izveštaj predmeta prikaz u dijalogu



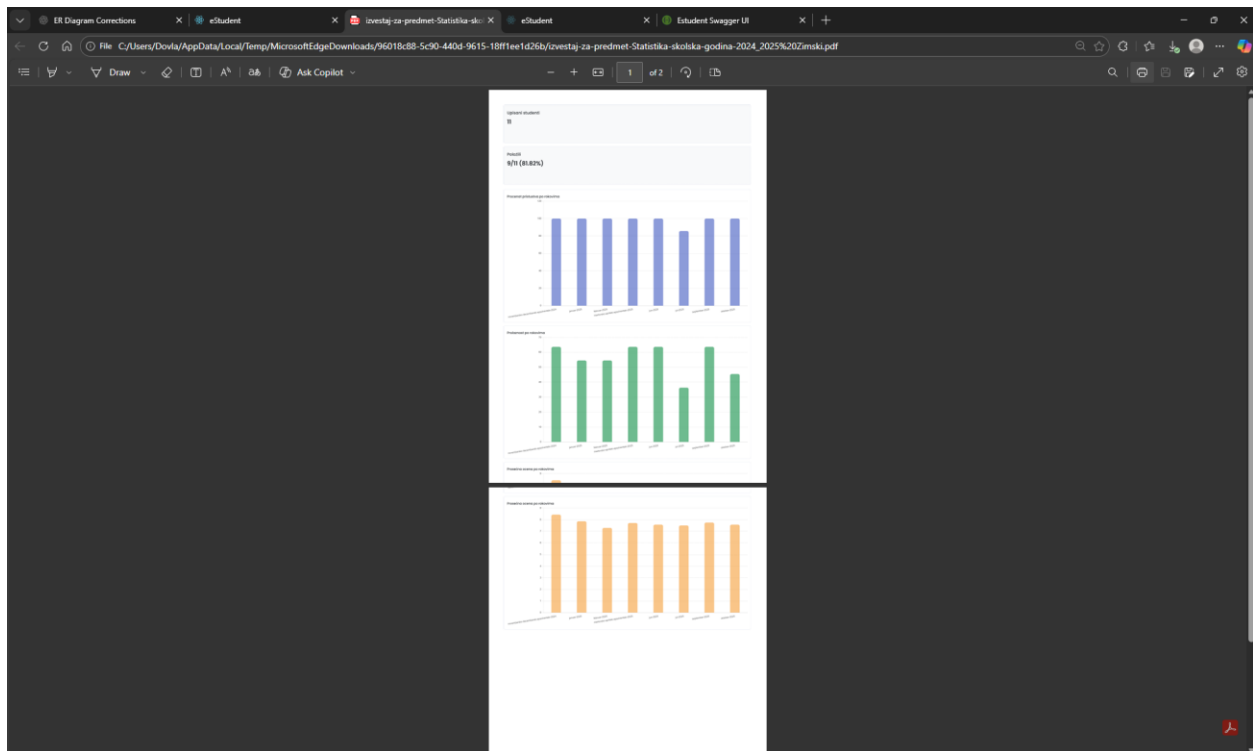
Slika 93: SK10, Procenat prisustva po rokovima



Slika 94: SK10, Prolaznost po rokovima



Slika 95: SK10, Dugme za preuzimanje PDF-a i prosečna ocena po rokovima



Slika 96: SK10, PDF izveštaj

Implementacija na serverskoj strani pakuje podatke koji će kasnije biti prezentovani tako što za zadati predmet u godini prolazi kroz sve rokove i za svaki računa prisutnost (\$attendancePercentage), prosečnu ocenu (\$averageGrade) i procenat prolaznosti (\$passagePercentage).

```

app > estudent > domain > useCases > GetReportDataForCourseInstanceImpl.php > PHP > GetReportDataForCourseInstanceImpl > getReportDataForCourseInstance()
11 class GetReportDataForCourseInstanceImpl implements GetReportDataForCourseInstance
12 {
13     public function getReportDataForCourseInstance(int $courseInstanceId): CourseSemesterReportDTO
14     {
15         $courseInstance = CourseInstance::with(relations: [
16             'course',
17             'semester',
18             'participants',
19             'courseExams.examPeriod',
20             'courseExams.examRegistrations'
21         ])->findOrFail(id: $courseInstanceId);
22
23         $title = 'Izveštaj za predmet ' . $courseInstance->course->name . ' u školskoj ' . $courseInstance->semester->academic_year;
24         $enrolledCount = $courseInstance->participants->count();
25
26         $attendanceSeries = [];
27         $passageSeries = [];
28         $averageGradesSeries = [];
29         $passedStudentIds = collect();
30
31         foreach ($courseInstance->courseExams as $courseExam) {
32             $examPeriod = $courseExam->examPeriod;
33             $allRegistrations = $courseExam->examRegistrations;
34
35             // Use service methods for calculations
36             $attendancePercentage = $this->courseExamService->calculateAttendancePercentage(courseExamId: $courseExam->id);
37             $averageGrade = $this->courseExamService->calculateAverageScore(courseExamId: $courseExam->id);
38             $passagePercentage = $this->calculatePassagePercentage(registrations: $allRegistrations, enrolledCount: $enrolledCount, passedStudentIds: $passedStudentIds);
39
40             $attendanceSeries[] = new ExamPeriodMetricPointDTO(
41                 examPeriodId: $examPeriod->id,
42                 examPeriodName: $examPeriod->name,
43                 value: $attendancePercentage
44             );
45             $passageSeries[] = new ExamPeriodMetricPointDTO(
46                 examPeriodId: $examPeriod->id,
47                 examPeriodName: $examPeriod->name,
48                 value: $passagePercentage
49             );
50             $averageGradesSeries[] = new ExamPeriodMetricPointDTO(
51                 examPeriodId: $examPeriod->id,
52                 examPeriodName: $examPeriod->name,
53                 value: $averageGrade
54             );
55
56             // Sort all series by examPeriod dateStart
57             $sortByDateStart = fn($a, $b): int => $this->getExamPeriodDateStart(dataPoint: $a) <=> $this->getExamPeriodDateStart(dataPoint: $b);
58             usort(array: &$attendanceSeries, callback: $sortByDateStart);
59             usort(array: &$passageSeries, callback: $sortByDateStart);
60             usort(array: &$averageGradesSeries, callback: $sortByDateStart);
61
62             $passedCount = $passedStudentIds->count();
63
64             return new CourseSemesterReportDTO(
65                 title: $title,
66                 attendanceSeries: $attendanceSeries,
67                 passageSeries: $passageSeries,
68                 averageGradesSeries: $averageGradesSeries,
69                 enrolledCount: $enrolledCount,
70                 passedCount: $passedCount
71             );
72         }
73     }
74 }

```

Slika 97: SK10, računanje prosečne prisutnosti, ocene i prolaznosti za izabrani predmet u svakom od rokova

4.3. API

Api se koristi za komunikaciju sa klijentskom stranom. Login ruta je javno dostupna, dok se za ostale rute prilaže Bearer Token u headeru, koji se proverava kasnije u Middlewaru kroz OAuth 2.0 autentikaciju korišćenjem Laravel Passport paketa. Takođe postoje rute koje su zaštićene dodatnim Admin Middleware-om, čijem pristupa imaju samo korisnici koje su administratori. Swagger dokumentacija na linku: [Estudent Swagger UI](#)

 Swagger
OpenAPI to SMART BEAR

http://localhost:8001/docs?api-docs.json

Explore

Estudent 1.0 OAS 3.0
<http://localhost:8001/docs?api-docs.json>Servers
http://localhost:8001/estudent/api - Rest API Server

Authorize

Filter by tag

Common Routes Common Routes

POST

/login

User login

POST

/logout

Logouts user, by deleting its token

GET

/course-exams/remaining-course-exams

Get all remaining course exams for currently logged-in student in provided exam period

GET

/course-exams/registerable-course-exams

Get all course-exams for current user that he did not register for yet in any active exam period

GET

/exam-periods

Retrieve only currently active or all exam periods

GET

/exam-registrations/passed

Retrieve only passed exam registrations for the authenticated student

GET

/exam-registrations/current

Retrieve only current exam registrations for the authenticated student

POST

/exam-registrations

Store new exam registration

DELETE

/exam-registrations/{examRegistrationId}

Delete existing exam registration

Admin Routes Admin Routes

GET

/admin/course-instances

Get paginated course instances with filters

GET

/admin/course-report-data/{courseInstanceId}

Get statistics report data for a course instance

GET

/admin/course-exams

Get paginated course exams with filters

GET

/admin/course-exam-reports/{courseExamId}

Download Excel report for a course exam identified by courseExamId

GET

/admin/exam-registrations

Retrieve exam registrations

PUT

/admin/exam-registrations/{examRegistrationId}

Update existing exam registration

GET

/admin/semesters

Get all semesters

Slika 98: Spisak Swagger ruta

5. Zaključak

U okviru ovog rada projektovano je i realizovano softversko rešenje za veb aplikaciju studentske službe Estudent. Rad je obuhvatio celokupan proces od teorijske analize metodologija za projektovanje arhitekture softvera, kroz njihovu primenu na konkretnom studijskom primeru, do implementacije funkcionalnog prototipa. Korišćena je arc42 struktura templejta koja je omogućila sistematičnu dokumentaciju svih aspekata arhitekture, od poslovnog konteksta i tehničkih ograničenja, preko pregleda modula i procesa, do detaljnog projektovanja interfejsa. Paralelno su analizirani 4+1 View Model i Software Architecture Inception Canvas, čime je obezbeđen sveobuhvatan pogled na rešenje.

Realizovano je ukupno 10 slučajeva korišćenja koji pokrivaju ključne funkcionalnosti sistema: prijavljivanje korisnika sa OAuth 2.0 autentifikacijom, pregledanje ispita u ispitnim rokovima, prijavljivanje i odjavljivanje polaganja ispita, ocenjivanje prijava studenata, kao i generisanje izveštaja u Excel i PDF formatu. Na klijentskoj strani korišćen je React.js sa Page-Component-Service arhitekturnim paternom, dok je na serverskoj strani korišćen Laravel framework organizovan po principima heksagonalne arhitekture. Implementirano je handlovanje grešaka putem custom exception klasa, logovanje grešaka, kao i Swagger dokumentacija API-a za lakše testiranje i integraciju. Rešenje zadovoljava definisane attribute kvaliteta: pouzdanost, fleksibilnost, sigurnost i operativnost, što je dokumentovano kroz stablo kvaliteta sa scenarijima kvaliteta.

Kao potencijalni pravci daljeg razvoja mogu se izdvojiti proširivanje sistema novim slučajevima korišćenja, poput upravljanja ispitnim rokovima od strane profesora, biranje izbornih predmeta od strane studenta ili automatskog obaveštavanja studenata o rezultatima ispita. Takođe, moguće je olakšati rad razvojem mobilne aplikacije. Sa aspekta arhitekture, razdvajanje sistema na nezavisne servise (mikroservisna arhitektura) bi mogao doprineti boljoj skalabilnosti sistema u produkcijskom okruženju, a što se pouzdanosti i održavanja rešenja tiče, pisanje unit testova bi značajno poboljšalo predvidivost ponašanja budućih verzija rešenja.

6. Reference

6.1. Linkovi ka Github repozitorijumima projekta

React: VladimirJecic/estudent_frontend: FronEnd fullstack aplikacije u Reactu. Imitacija webServisa E-Student sa nekim osnovnim funkcionalnostima. (github.com)

Laravel: VladimirJecic/estudent_frontend: FronEnd fullstack aplikacije u Reactu. Imitacija webServisa E-Student sa nekim osnovnim funkcionalnostima. (github.com)

6.2 Korišćena Literatura

¹ Hoare, C. A. R. (1974). Monitors: An operating system structuring concept. In P. Brinch Hansen (Ed.), *The origin of concurrent programming* (pp. 272–294). Springer.

² Kim, B., Kim, H. & Jeon, Y. (2018). Critical success factors of a design startup business. *Sustainability*, 10(9), 2981.

³ Zave, Pamela. (1997). Classification of Research Efforts in Requirements Engineering. *ACM Comput. Surv.* 29. 315-321. 10.1109/ISRE.1995.512563.

⁴ Radenković, M., Despotović-Zrakić, M., Bogdanović, Z., Barać, D., Labus, A., & Simić, M. (2024). *Razvoj softvera orijentisanog na procese, odabrana poglavlja: DevOps, Mikroservisi, Kontejneri (II izmenjeno i dopunjeno izdanje)*. Београд: Рачунарски факултет; Факултет организационих наука.
<https://rfos.fon.bg.ac.rs/handle/123456789/2877>

⁵ Educative. (n.d.). What are include and extend relationships in a use case diagram? Educative.
<https://www.educative.io/answers/what-are-include-and-extend-relationships-in-a-use-case-diagram>

⁶ Rainer, R. K., & Turban, E. (2009). *Uvod u informacione sisteme*. Beograd: Data Status.

⁷ AltexSoft Editorial Team. (n.d.). Nonfunctional requirements in software engineering: Examples, types, best practices. AltexSoft. <https://www.altexsoft.com/blog/non-functional-requirements/>

⁸ Bigelow, S. J. (2024, December 23). What are the types of APIs and their differences? TechTarget.
<https://www.techtarget.com/searchapparchitecture/tip/What-are-the-types-of-APIs-and-their-differences>

⁹ Postman. (n.d.). API design. Postman. <https://www.postman.com/api-platform/api-design/>

¹⁰ Zimmermann, O., Stocker, M., Lubke, D., Zdun, U. & Pautasso, C. (2022). *Patterns for API design: simplifying integration with loosely coupled message exchanges*. Addison-Wesley Professional.

¹¹ Vlajić, S. (2020). *Projektovanje softvera (Skripta – radni materijal – 2020)*. ResearchGate.
https://www.researchgate.net/publication/344459754_PROJEKTOVANE_SOFTVERA_SKRIPTA_-_RADNI_MATERIJAL_-_2020

¹² Red Hat. (2024, February 1). What is a webhook. Red Hat.
<https://www.redhat.com/en/topics/automation/what-is-a-webhook>

- ¹³ Starke, G., Roos, P., & arc42 contributors. (n.d.). arc42 template overview. arc42. <https://arc42.org/overview>
- ¹⁴ Starke, G., Roos, P., & arc42 contributors. (n.d.). arc42 quality model. arc42. <https://quality.arc42.org/articles/arc42-quality-model>
- ¹⁵ Zörner, S. (n.d.). Dokchess: arc42 template example. Dokchess. https://www.dokchess.de/en/00_overview/
- ¹⁶ Fakhroutdinov, K. (2009–2026). UML deployment diagrams. uml-diagrams.org. <https://www.uml-diagrams.org/deployment-diagrams.html>
- ¹⁷ Jayawardene, P. D. (2021, September 17). 4+1 architectural view model in software. Medium. <https://medium.com/javarevisited/4-1-architectural-view-model-in-software-ec407bf27258>
- ¹⁸ Starke, G., Roos, P., & arc42 contributors. (2025). Architecture inception canvas. arc42. <https://canvas.arc42.org/architecture-inception-canvas>
- ¹⁹ Kumar, S. (2024, April 7). Basics of software design. Scaler. <https://www.scaler.com/topics/software-engineering/what-is-software-design/>
- ²⁰ GeeksforGeeks. (2025, July 11). Software design process – software engineering. GeeksforGeeks. <https://www.geeksforgeeks.org/software-engineering/software-engineering-software-design-process/>
- ²¹ TutorialsPoint. (n.d.). Key principles of software architecture design. TutorialsPoint. https://www.tutorialspoint.com/software_architecture_design/key_principles.htm
- ²² TutorialsPoint. (n.d.). Architecture models. TutorialsPoint. https://www.tutorialspoint.com/software_architecture_design/architecture_models.htm
- ²³ Starke, G., Hruschka, P., & arc42 contributors. (n.d.). Example business context: HTMLSanityCheck. arc42. <https://docs.arc42.org/examples/business-context-1/>
- ²⁴ Tangly llc. (2016–2025). Cross-cutting concepts. tangly.net. <https://blog.tangly.net/docs/domains/architecture/08-crosscutting-concepts/>
- ²⁵ Starke, G. (2022, February 21). Brief introduction to arc42. Medium. <https://medium.com/@parserdigital/architecture-documentation-with-arc42-77662678aad8>
- ²⁶ API Patterns. (2019–2023). Patterns by Architectural Scope: Patterns for API Design. api-patterns.org. <https://api-patterns.org/patterns/byscope>